

Macacário

Dijkstravou

August 25, 2026

Contents

1	Template	7
1.1	Template código	7
1.2	Hash	7
1.3	Pragmas	8
2	Grafos	9
2.1	Fluxo	9
2.2	Bridges	15
2.3	Cut Vertex	15
2.4	SCC	15
2.4.1	Kosaraju	15
2.4.2	Tarjan	16
2.5	HLD	16
2.6	Matching	17
2.7	Single source shortest paths	18
2.7.1	Bellmanford	18
2.7.2	SPFA	19
2.7.3	01 BFS	20
2.7.4	1-K BFS	20
2.8	Outros	20
2.8.1	Aciclicidade de grafos	20
2.8.2	Pintando arestas de árvores	22
2.8.3	Conectividade de vértices e arestas	24
2.8.4	Segunda melhor MST	24
2.8.5	Número de spanning trees	26
2.8.6	Prufer code	26
2.8.7	MST direcionada	27

2.8.8	Árvore dominadora	29
2.8.9	Conectividade dinâmica	30
2.8.10	Centróide	32
2.8.11	Isomorfismo de árvores	34
2.8.12	Caminho euleriano	35
2.9	2-SAT	36
3	DP	38
3.1	Principais problemas	38
3.1.1	Knapsack 0-1	38
3.1.2	LIS	38
3.1.3	Edit Distance	38
3.2	Otimização	38
3.3	Outras técnicas	38
3.3.1	Convex Hull Trick	38
3.3.2	Divide and Conquer	39
3.3.3	Knuth	40
4	Geometria	42
4.1	Struct	42
4.2	Coisas gerais	42
4.2.1	Intersecção de reta com reta	42
4.2.2	Checando se segmentos intersectam	42
4.2.3	Equação da bissetriz	43
4.2.4	Distância de ponto a reta	43
4.2.5	Equação da reta a partir de dois pontos	43
4.2.6	Projeção de ponto em reta	43
4.2.7	Distância de ponto a segmento	43
4.2.8	Intersecção reta com círculo	43
4.2.9	Intersecção de círculo com círculo	44
4.2.10	Tangentes comuns a dois círculos	44
4.2.11	Miscelânea	44
4.3	Polígonos	45
4.3.1	Paralelograma e triângulo	45
4.3.2	Área de polígono	45
4.3.3	Checa se ponto está dentro de polígono	45
4.3.4	Soma de Minkowski	46
4.3.5	Teorema de Pick	47
4.3.6	Número de lattice points dentro de non-lattice polygon	47
4.4	Convex Hull	48

4.4.1	Graham Scan	48
4.4.2	Monotone Chain	49
4.4.3	Dinâmico	49
4.5	Line Sweep	52
4.5.1	Detecta se par de segmentos se intersecta	52
4.6	Grafos planares	53
4.6.1	Encontrar faces	53
4.6.2	Construir grafo planar a partir de segmentos	55
4.6.3	Point location	57
4.7	Miscelânea	61
4.7.1	Par mais próximo de pontos	61
4.7.2	Triangulação de Delaunay e Diagrama de Voronoi	62
4.7.3	Área da união de polígonos	65
4.7.4	Intersecção de half-plane	66
4.7.5	Distância de Manhattan máxima	68
4.7.6	Manhattan Minimum Spanning Tree	69
5	Estruturas de dados	70
5.1	Minimum Stack e Queue	70
5.2	Disjoint Set Union	71
5.3	Fenwick tree	73
5.4	Segmentation Tree	74
5.4.1	Seg Normal	74
5.4.2	Memória eficiente	75
5.4.3	Lazy	75
5.4.4	Persistent	76
5.4.5	Esparsa	76
5.5	Sqrt decomposition	77
5.5.1	Algoritmo de Mo	77
5.5.2	Mo com update	79
5.6	Wavelet tree	81
5.7	Policy Based Set	85
6	Strings	86
6.1	Hashing	86
6.1.1	Rabin-Karp para matching	87
6.2	Função prefixo e KMP	87
6.3	Função Z	88
6.4	Suffix Array	89
6.5	Algoritmo de Aho-Corasick e Tries	91

6.6	Algoritmo de Ukkonen	93
6.7	Autômato de sufixos	96
6.8	Fatoração de Lyndon	96
6.9	Extras	96
6.9.1	Parsing	96
6.9.2	Algoritmo de Manacher	99
6.9.3	Encontrando repetições	99
7	Álgebra e Teoria dos Números	102
7.1	Exponenciação binária	102
7.2	MDC e MMC	102
7.3	Algoritmo de Euclides Estendido	102
7.4	Equações Diofantinas Lineares	103
7.5	Números de fibonacci	104
7.5.1	Propriedades interessantes	104
7.5.2	Fibonacci Coding	104
7.5.3	Periodicidade	105
7.5.4	Problemas	105
7.6	Primos	105
7.6.1	Crivo de Eratóstenes	105
7.6.2	Testes de primalidade	107
7.6.3	Fatoração	108
7.7	Funções comuns de teoria dos números	109
7.7.1	Função totiente de Euler	109
7.7.2	Funções quantidade e soma de divisores	110
7.7.3	Função de mobius	110
7.7.4	Funções aritméticas multiplicativas	111
7.8	Inversos modulares	111
7.9	Congruências modulares	112
7.9.1	Teorema Chinês dos Restos	112
7.9.2	Algoritmo de Garner	113
7.9.3	Fatorial módulo p	114
7.9.4	Fórmula de Legendre	114
7.9.5	Logaritmo Discreto	114
7.9.6	Raízes primitivas	116
7.9.7	Raiz discreta	116
7.10	Sistemas Numéricos	118
7.10.1	Ternário balanceado	118
7.10.2	Código de Gray	118
7.11	Manipulação de bits	119

7.11.1	Iterar por todas as submáscaras de uma máscara	119
7.12	Aritmética de precisão arbitrária	119
7.12.1	IO do int128	120
7.13	FFT	120
7.13.1	Number theoretic transform	121
7.13.2	FFT 2D	125
7.13.3	Fast Walsh–Hadamard	126
7.14	Métodos numéricos	127
7.14.1	Busca ternária	127
7.14.2	Integração de Simsom	127
7.15	Miscelânea	128
7.15.1	Números harmônicos	128
7.15.2	Funções Multiplicativas II	128
8	Combinatória	133
8.1	Fatoriais e binomiais	133
8.2	Números de catalão	135
8.3	Princípio da Inclusão e Exclusão	135
8.4	Teoria dos grupos	137
8.5	K-Combinações	139
8.6	Extras	140
9	Teoria dos Jogos	144
9.1	Jogos em grafos arbitrários	144
9.2	Teorema de Sprague-Grundy e Nim	146
9.2.1	Nim	146
9.2.2	Teorema de Sprague-Grundy	146
10	Álgebra Linear	148
10.1	Equações lineares	148
10.2	Determinante e posto	148
10.2.1	Decomposição LU	149
10.3	Algelin em $\mathbb{F}_2$	150
11	Miscelânea	152
12	Apêndice	152
12.1	Matemática	152
12.1.1	Constantes	152
12.1.2	Fórmulas e Teoremas	152
12.1.3	Números primos	152

12.2 Big-O	154
12.3 Python...	154

1 Template

1.1 Template código

23f008

```
#include <bits/stdc++.h>

using namespace std;

typedef long long ll;
typedef pair<ll, ll> pii;
typedef long double ld;
typedef vector<ll> vi;
typedef vector<pii> vpi;

const ll maxn = 2e5 + 10;
const ll inf = 0x3f3f3f3f3f3f3f3f;

#define pb push_back
#define ppb pop_back
#define ff first
#define ss second
#define sza(x) ((int)(x).size())
#define all(a) (a).begin(), (a).end()
#define smax(s, v) s = max(s, (v))
#define smin(s, v) s = min(s, (v))

#define FFR(i, a, b, s) for(int i = a; i != b; i += s)
#define FF(i, n) FFR(i, 0, n, 1)
#define FF1(i, n) FFR(i, 1, n+1, 1)
#define FFD(i, n) FFR(i, n-1, -1, -1)
#define FFD1(i, n) FFR(i, n, 0, -1)

#define F(n) FF(i, n)
#define F1(n) FF1(i, n)
#define FD(n) FFD(i, n)
#define FD1(n) FFD1(i, n)
#define FR(a, b, s) FFR(i, a, b, s)

template<typename A, typename B> ostream& operator<<(ostream
```

```
&os, const pair<A, B> &p) { return os << "(" << p.ff <<
", " << p.ss << ")"; }
template<typename A> ostream& operator<<(ostream& os, const
vector<A>& v) { os << "["; for(const auto& x: v) os << x
<< ", "; return os << "]"; }
```

```
#ifdef LOCAL
void dbg_out() { cerr << endl; }
template<typename H, typename... T> void dbg_out(H h, T... t
) { cerr << " " << h; dbg_out(t...); }
#define dbg(...) cerr << "(" << #__VA_ARGS__ << "):",
dbg_out(__VA_ARGS__)
#else
#define dbg(...)
#endif
```

```
ll n, m, p, q, k;
ll a[maxn];
```

```
void solve() {
    cin >> n;
    F(n) {
        cin >> a[i];
    }
}
```

```
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
```

```
int t;
cin >> t;
```

```
F1(t) solve();
```

1.2 Hash

```
# Tirado de https://github.com/kth-competitive-programming/kactl/
# Entre as aspas não tem nenhum espaço.....
```

```
cpp -P -fpreprocessed $1 | tr -d '[:space:]' | md5sum | cut -c
-6
```

1.3 Pragas

Dois pragmas úteis: "(hashtag)pragma GCC optimize("opt1,opt2...")" e "(hashtag)pragma GCC target("t1,t2...")". Note que não tem espaço entre vírgulas.

Coloca isso no começo do código (antes dos includes). Lista de carinhas úteis:

- **optimize:** O3, Ofast (toma cuidado com math com esse), unroll-loops (já ta no O3 e Ofast), tree-vectorize (precisa disso para usar avx, se não tiver O(hashtag) ou Ofast ativado)
- **target:** avx2 (se nao der, tenta avx), popcnt (otimiza familia do popcount), lzcnt (otimiza familia do clz), abm, bmi, bmi2 (bmi não é subset do bmi2).

Se quiser ativar a otimização só em uma função, dá para por:

```
__attribute__((target("t1,t2..."), optimize("opt1,opt2..."))) void funcao() ....
```


2 Grafos

2.1 Fluxo

Problema 2.1.1: São dados um grafo $G = (V, E)$ direcionado, dois vértices s (source) e t (sink) e uma taxa máxima de fluxo (capacidade) pra cada aresta. Determinar o fluxo máximo.

Soluções:

1. Ford-Fulkerson: $O(EF)$, onde F é o fluxo máximo.
2. Edmonds-Karp: $O(VE^2)$. Ou use Dinic:
3. Dinic: $O(V^2E)$
4. Scaled Dinic: $O(VE \log F)$, onde F é o fluxo máximo.

É pra o scaled dinic sempre passar em qualquer problema... os outros boto mais como referência

Edmonds-Karp:

(Use uma dfs para virar Ford-Fulkerson)

```

291cf4
int n;
vector<vector<int>> capacity;
vector<vector<int>> adj;

int bfs(int s, int t, vector<int>& parent) {
    fill(parent.begin(), parent.end(), -1);
    parent[s] = -2;
    queue<pair<int, int>> q;
    q.push({s, INF});

    while (!q.empty()) {
        int cur = q.front().first;
        int flow = q.front().second;
        q.pop();

        for (int next : adj[cur]) {
            if (parent[next] == -1 && capacity[cur][next]) {
                parent[next] = cur;
            }
        }
    }

    return flow;
}

int maxflow(int s, int t) {
    int flow = 0;
    vector<int> parent(n);
    int new_flow;

    while (new_flow = bfs(s, t, parent)) {
        flow += new_flow;
        int cur = t;
        while (cur != s) {
            int prev = parent[cur];
            capacity[prev][cur] -= new_flow;
            capacity[cur][prev] += new_flow;
            cur = prev;
        }
    }

    return flow;
}

```

```

int new_flow = min(flow, capacity[cur][next]);
if (next == t) return new_flow;
q.push({next, new_flow});
    }
}

return 0;
}

int maxflow(int s, int t) {
    int flow = 0;
    vector<int> parent(n);
    int new_flow;

    while (new_flow = bfs(s, t, parent)) {
        flow += new_flow;
        int cur = t;
        while (cur != s) {
            int prev = parent[cur];
            capacity[prev][cur] -= new_flow;
            capacity[cur][prev] += new_flow;
            cur = prev;
        }
    }

    return flow;
}

```

Dinic:

```

b8d428
struct FlowEdge {
    int v, u;
    long long cap, flow = 0;
    FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(cap) {}
};

struct Dinic {

```

```

const long long flow_inf = 1e18;
vector<FlowEdge> edges;
vector<vector<int>> adj;
int n, m = 0;
int s, t;
vector<int> level, ptr;
queue<int> q;

Dinic(int n, int s, int t) : n(n), s(s), t(t) {
    adj.resize(n);
    level.resize(n);
    ptr.resize(n);
}

void add_edge(int v, int u, long long cap) {
    edges.emplace_back(v, u, cap);
    edges.emplace_back(u, v, 0);
    adj[v].push_back(m);
    adj[u].push_back(m + 1);
    m += 2;
}

bool bfs() {
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        for (int id : adj[v]) {
            if (edges[id].cap - edges[id].flow < 1)
                continue;
            if (level[edges[id].u] != -1) continue;
            level[edges[id].u] = level[v] + 1;
            q.push(edges[id].u);
        }
    }
    return level[t] != -1;
}

long long dfs(int v, long long pushed) {
    if (pushed == 0) return 0;
    if (v == t) return pushed;

```

```

        for (int& cid = ptr[v]; cid < (int)adj[v].size();
             cid++) {
            int id = adj[v][cid];
            int u = edges[id].u;
            if (level[v] + 1 != level[u] || edges[id].cap -
                edges[id].flow < 1) continue;
            long long tr = dfs(u, min(pushed, edges[id].cap -
                edges[id].flow));
            if (tr == 0) continue;
            edges[id].flow += tr;
            edges[id ^ 1].flow -= tr;
            return tr;
        }
        return 0;
    }

    long long flow() {
        long long f = 0;
        while (true) {
            fill(level.begin(), level.end(), -1);
            level[s] = 0;
            q.push(s);
            if (!bfs()) break;
            fill(ptr.begin(), ptr.end(), 0);
            while (long long pushed = dfs(s, flow_inf)) {
                f += pushed;
            }
        }
        return f;
    }
};

```

Scaled Dinic:

6ae100

```

/**
 * Author: chilli
 * Date: 2019-04-26
 * License: CC0
 * Source: https://cp-algorithms.com/graph/dinic.html
 * Description: Flow algorithm with complexity  $O(VE \log U)$ 

```

```

    where $U = \max |\text{cap}|$.
* $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$
  for bipartite matching.
* Status: Tested on SPOJ FASTFLOW and SPOJ MATCHING, stress
  -tested
*/

typedef vector<int> vi;

struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you
            need flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sza(adj[b]), c, c});
        adj[b].push_back({a, sza(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int& i = ptr[v]; i < sza(adj[v]); i++) {
            Edge& e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0;
        q[0] = s;
        FF(L, 31) do { // 'int L=30' maybe faster for random
            data

```

```

        lvl = ptr = vi(sza(q));
        int qi = 0, qe = lvl[s] = 1;
        while (qi < qe && !lvl[t]) {
            int v = q[qi++];
            for (Edge e : adj[v])
                if (!lvl[e.to] && e.c >> (30 - L)) q[qe
                    ++] = e.to, lvl[e.to] = lvl[v] + 1;
        }
        while (ll p = dfs(s, t, LLONG_MAX))
            flow += p;
    }
    while (lvl[t])
        ;
    return flow;
}
bool leftOfMinCut(int a) { return lvl[a] != 0; }
};

```

Problema 2.1.2: $s-t$ -cut é uma partição dos vértices em um conjunto que contém s e outro t . A capacidade de um cut é a soma das capacidades das arestas indo de um conjunto pra outro. É conhecido que a capacidade do max flow é igual a do min cut. Ache esse min-cut.

Solução: Faça o max-flow. Particione em: conjunto de vértices alcançados por s no grafo residual e o resto. Lembrando que, no código do Scaled Dinic, quando ele termina, os "e.c" são as capacidades residuais.

Problema 2.1.3: Achar as rotas que o fluxo máximo fez.

Solução: Depois de rodar aquele dinic, itera pelas arestas da struct e refaz um grafo colocando arestas $u \rightarrow v$ pra cada (u, v) flow > 0 . Daí, faz uma dfs que pega qualquer caminho (ao invés de tentar ir pra todos) e gg. Ou seja:

```

693a03
// lembrando que isso passou do Distinct Routes do CSES
// dentro do struct
void build_from_edges() {
    for (Edge e : in_sequence) {
        int from = adj[e.to][e.rev].to;
        if (e.flow() == 0 || e.oc == 0) continue;
        from_edges[from].push_back(e.to);
    }
}

```

```
// fora do struct
void the_dfs(int v) {
    path.push_back(v);
    if (d.from_edges[v].size()) {
        the_dfs(d.from_edges[v].back());
        d.from_edges[v].pop_back();
    }
}

// na main
cout << d.calc(0, n - 1) << "\n";
d.build_from_edges();
for (auto v : d.from_edges[0]) {
    path.clear();
    path.push_back(0);
    the_dfs(v);
    cout << path.size() << "\n";
    for (auto cara : path) {
        cout << cara + 1 << " ";
    }
    cout << "\n";
}
```

Problema 2.1.4: Fluxo com demandas. Para cada $e \in E$, queremos achar uma função f tal que $d(e) \leq f(e) \leq c(e)$ em que d, c são funções dadas.

Solução: Adicione uma source s' e um sink t' , uma aresta de s' para todos os outros vértices, uma nova aresta de cada vértice para o sink t' e uma aresta de t para s . Defina uma nova função capacidade tal que

1. $c'((s', v)) = \sum_{u \in V} d((u, v))$ para cada aresta (s', v) .
2. $c'((v, t')) = \sum_{w \in V} d((v, w))$ para cada aresta (v, t')
3. $c'((u, v)) = c((u, v)) - d((u, v))$ para cada aresta (u, v) na rede antigo.
4. $c'((t, s)) = \infty$

Se essa nova rede tiver um fluxo saturante (em que cada aresta saindo de s' está completamente preenchida, que dá no mesmo que as indo pra t' estarem), então a rede com demandas tem um fluxo válido. Basta usar algum algoritmo de max-flow para achar o saturating-flow.

Problema 2.1.5: Idêntico ao 2.1.4, mas queremos o fluxo mínimo.

Solução: Ao invés de usar $c'((t, s)) = \infty$, faça uma busca binária em $c'((t, s))$.

Problema 2.1.6: (min-cost flow) Para cada aresta, são dados a capacidade e o custo por unidade de fluxo. Encontre um fluxo de K e entre todos esses fluxos encontre o com menor custo.

Atenção: não funfa se tiver ciclo negativo. Pra maximizar o custo bota sinal de menos que dá bom. Repara também que a cada etapa ele faz um dijkstra e vai vendo o negócio no path de menor caminho.

Solução:

198327

```
/**
 * Author: Stanford
 * Date: Unknown
 * Source: Stanford Notebook
 * Description: Min-cost max-flow.
 * If costs can be negative, call setpi before maxflow, but
 * note that negative cost cycles are not
 * supported. To obtain the actual flow, look at positive
 * values only. Status: Tested on
 * kattis:mincostmaxflow, stress-tested against another
 * implementation Time:  $\mathcal{O}(F E \log(V))$  where
 *  $F$  is max flow.  $\mathcal{O}(VE)$  for setpi.
 */
#pragma once

// #include <bits/extc++.h> /// include-line, keep-include

const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vi seen;
    vector<ll> dist, pi;
    vector<edge*> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N)
```

```

N) {}

void addEdge(int from, int to, ll cap, ll cost) {
    if (from == to) return;
    ed[from].push_back(edge{from, to, sza(ed[to]), cap,
        cost, 0});
    ed[to].push_back(edge{to, from, sza(ed[from]) - 1,
        0, -cost, 0});
}

void path(int s) {
    fill(all(seen), 0);
    fill(all(dist), INF);
    dist[s] = 0;
    ll di;

    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vector<decltype(q)::point_iterator> its(N);
    q.push({0, s});

    while (!q.empty()) {
        s = q.top().second;
        q.pop();
        seen[s] = 1;
        di = dist[s] + pi[s];
        for (edge& e : ed[s])
            if (!seen[e.to]) {
                ll val = di - pi[e.to] + e.cost;
                if (e.cap - e.flow > 0 && val < dist[e.to]) {
                    dist[e.to] = val;
                    par[e.to] = &e;
                    if (its[e.to] == q.end())
                        its[e.to] = q.push({-dist[e.to],
                            e.to});
                    else
                        q.modify(its[e.to], {-dist[e.to],
                            e.to});
                }
            }
    }
}

```

```

}
F(N) pi[i] = min(pi[i] + dist[i], INF);
}

pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
        ll fl = INF;
        for (edge* x = par[t]; x; x = par[x->from])
            fl = min(fl, x->cap - x->flow);

        totflow += fl;
        for (edge* x = par[t]; x; x = par[x->from]) {
            x->flow += fl;
            ed[x->to][x->rev].flow -= fl;
        }
    }
    F(N) for (edge& e : ed[i]) totcost += e.cost * e.
        flow;
    return {totflow, totcost / 2};
}

// If some costs can be negative, call this before
// maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF);
    pi[s] = 0;
    int it = N, ch = 1;
    ll v;
    while (ch-- && it--)
        F(N)
        if (pi[i] != INF)
            for (edge& e : ed[i])
                if (e.cap)
                    if ((v = pi[i] + e.cost) < pi[e.to]) pi[
                        e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
}
};

```

Problema 2.1.7: Algoritmo húngaro com fluxo. Seleccionar n elementos de

uma matriz quadrada A de ordem n , com nenhuma fileira em comum, de modo a maximizar sua soma.

Solução:

```

bf4f7f
const int INF = 1000 * 1000 * 1000;

vector<int> assignment(vector<vector<int>> a) {
    int n = a.size();
    int m = n * 2 + 2;
    vector<vector<int>> f(m, vector<int>(m));
    int s = m - 2, t = m - 1;
    int cost = 0;
    while (true) {
        vector<int> dist(m, INF);
        vector<int> p(m);
        vector<bool> inq(m, false);
        queue<int> q;
        dist[s] = 0;
        p[s] = -1;
        q.push(s);
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            inq[v] = false;
            if (v == s) {
                for (int i = 0; i < n; ++i) {
                    if (f[s][i] == 0) {
                        dist[i] = 0;
                        p[i] = s;
                        inq[i] = true;
                        q.push(i);
                    }
                }
            }
            else {
                if (v < n) {
                    for (int j = n; j < n + n; ++j) {
                        if (f[v][j] < 1 && dist[j] > dist[v]
                            + a[v][j - n]) {
                            dist[j] = dist[v] + a[v][j - n];
                            p[j] = v;
                        }
                    }
                }
            }
        }
    }
}

```

```

        if (!inq[j]) {
            q.push(j);
            inq[j] = true;
        }
    }
}
else {
    for (int j = 0; j < n; ++j) {
        if (f[v][j] < 0 && dist[j] > dist[v]
            - a[j][v - n]) {
            dist[j] = dist[v] - a[j][v - n];
            p[j] = v;
            if (!inq[j]) {
                q.push(j);
                inq[j] = true;
            }
        }
    }
}
}
}
}

int curcost = INF;
for (int i = n; i < n + n; ++i) {
    if (f[i][t] == 0 && dist[i] < curcost) {
        curcost = dist[i];
        p[t] = i;
    }
}
if (curcost == INF) break;
cost += curcost;
for (int cur = t; cur != -1; cur = p[cur]) {
    int prev = p[cur];
    if (prev != -1) f[cur][prev] = -(f[prev][cur] = 1);
}

vector<int> answer(n);
for (int i = 0; i < n; ++i) {

```

```

        for (int j = 0; j < n; ++j) {
            if (f[i][j + n] == 1) answer[i] = j;
        }
    }
    return answer;
}

```

2.2 Bridges

```

c335fc
int tin[maxn], low[maxn];
int timer;
vector<pii> res;

void dfs(int v, int p=-1) {
    tin[v] = low[v] = ++timer;
    bool pskip = false;
    for(int u: g[v]) {
        if(u == p && !pskip) {
            pskip = true;
            continue;
        }
        if(tin[u] != 0) smin(low[v], tin[u]);
        else {
            dfs(u, v);
            smin(low[v], low[u]);
            if(low[u] > tin[v]) res.pb({v, u});
        }
    }
}

void find_bridges() {
    Fl(n) if(tin[i] == 0) dfs(i);
}

```

Para fazer online, dá para pensar em componentes sem pontes (2-edge). Faz meio que um DAG dessas comps, e daí é só pensar um pouquinho.

2.3 Cut Vertex

Mesma coisa do de aresta, só com algumas modificações:

```

14192b
// PODE INSERIR O MSM VERTICE > 1 VEZ
if(low[u] >= tin[v] && p != -1) res.pb(v);

// Fim da dfs
if(p == -1 && children > 1) res.pb(v);

// Sempre que o nó v chama dfs em um filho, ++children (
    local)

```

2.4 SCC

2.4.1 Kosaraju

```

ca46d5
bool vis[maxn];
int nc;
int root[maxn];

void dfs(int v, vi* g, vi &out) {
    vis[v] = true;
    for(auto u: g[v]) if(!vis[u]) dfs(u, g, out);
    out.pb(v);
}

void scc() {
    vi ord;
    memset(vis, false, sizeof(vis));
    Fl(n) if(!vis[i]) dfs(i, g, ord);
    memset(vis, false, sizeof(vis));
    reverse(all(ord));
    for(auto v: ord) if(!vis[v]) {
        vi comp;
        dfs(v, gt, comp);
        int rt = ++nc;
        for(auto u: comp) root[u] = rt;
    }
}

```

2.4.2 Tarjan

Muito parecido com o de bridges.

c9e926

```
int tin[maxn], low[maxn];
int timer;
int nc;
int root[maxn];

void dfs(int v, vi &st) {
    tin[v] = low[v] = ++timer;
    st.pb(v);

    for(int u: g[v]) {
        if(tin[u] == 0) dfs(u, st);
        if(root[u] == 0) smin(low[v], low[u]);
    }

    if(low[v] == tin[v]) {
        int lst;
        ++nc;
        do {
            lst = st.back();
            st.ppb();
            root[lst] = nc;
        } while(lst != v);
    }
}

F1(n) if(tin[i] == 0) {
    vi st;
    dfs(i, st);
}
```

2.5 HLD

Divide a árvore em "segmentos". Pra cada vértice não folha, digo que a aresta que leva ao filho com sub-árvore maior é "pesada".

Divido minha árvore em heavy paths: zero ou mais arestas pesadas mais uma leve (tem uma exceção que são paths envolvendo a raiz)

Se eu fizer isso e salvar quem é o cara mais de cima de cada path e ir subindo, no máximo vou subir $\log n$ vezes.

Além disso, dá pra associar a cada vértice identificadores de modo que caras num mesmo heavy path correspondam a um intervalo contíguo. Então dá pra fazer uma árvore de segmentos.

No seguinte código, indexei no zero. Queries levam $O(\log^2)$ (porque é possível que eu passe por $\log$ segmentos, cada um demorando $\log$ pra query da seg)

O vetor heavy diz pra cada vértice não folha, qual criança corresponde à aresta pesada

O vetor head diz pra cada vértice o líder do seu heavy path. O vetor pos diz qual o id dele pra botar na seg.

97d989

```
// Lembra de declarar o n dentro da seg
// Aqui ta como point update
// LEMBRA DE CHAMAR O BUILD DA HLD

// Intervalo da seg eh [0, n)
#define INI 1, 0, n-1

struct Hld {
    int n, head;
    vi verts;
    Seg s;

    void build() {
        n = sza(verts);
        head = *verts.rbegin();

        // Inicializa valores da seg aqui
        vi sa(n);
        F(n) sa[i] = a[verts[i]];
        s = Seg(n);
        s.build(sa, INI);
    }

    ll query(int ql, int qr) { return s.query(ql, qr, INI); }
    ll query_end(int ql) { return s.query(ql, n-1, INI); }
    void update(int pos, ll v) { s.update(pos, v, INI); }
};
```



```

pii hldid[maxn];
int sztree[maxn], anc[maxn];
vector<Hld> hld;

void dfshld(int v, int p=0) {
    anc[v] = p;
    sztree[v] = 1;

    for(auto u: g[v]) if(u != p) {
        dfshld(u, v);
        sztree[v] += sztree[u];
    }

    int heavy = -1;
    for(auto u: g[v]) if(u != p) {
        if(2*sztree[u] >= sztree[v])
            heavy = u;
    }

    if(heavy == -1) {
        hldid[v] = {sza(hld), 0};
        hld.pb(Hld());
    } else {
        hldid[v] = hldid[heavy];
        hldid[v].ss++;
    }

    hld[hldid[v].ff].verts.pb(v);
}

void hld_update(int v, ll qtd) {
    auto [hid, hpos] = hldid[v];
    hld[hid].update(hpos, qtd);
}

// Inclusivo
ll hld_query(int v, int fim) {
    auto [hid, hpos] = hldid[v];
    auto [hidfim, hposfim] = hldid[fim];

```

```

    if(hid == hidfim) return hld[hid].query(hpos, hposfim);
    ll res = hld[hid].query_end(hpos);

    // Lembra de mudar aqui tambem, dependendo de como une
    int prox = anc[hld[hid].head];
    if(prox) res += hld_query(prox, fim);
    return res;
}

// NA MAIN
dfshld(1);
for(auto &h: hld) h.build();

```

2.6 Matching

Problema 2.6.1: Determinar se um grafo eh bipartido ou não

Soluções:

1. Roda uma DFS colorindo o grafo de 2 cores.
2. Se algum vértice tiver 2 cores em algum momento, o grafo não eh bipartido:

Problema 2.6.2: Dado um grafo bipartido de n vértices e m arestas, ache o matching máximo, isto eh, selecione o máximo de arestas possiveis do grafo tal que nenhuma destas tenham um vertice em comum.

Algoritmo de Hopcroft-Karp:

```

3de371
// recebe uma edge list das arestas entre os dois conjuntos
// de vertices do grafo bipartido, indexado no 0
// complexidade O(sqrt(V)E)
vector<pair<int, int>> hopcroft_karp(vector<pair<int, int>>
    > e) {
    int n = -1, m = -1;
    shuffle(e.begin(), e.end(), mt19937(random_device{}()));
    for(auto &[u, v]: e)
        n = max(n, u + 1), m = max(m, v + 1);
    vector<int> deg(n + 1);
    for(auto &[u, v]: e) deg[u]++;
    partial_sum(deg.begin(), deg.end(), deg.begin());

```

```

vector<int> g(e.size()), l(n, -1), r(m, -1), a, p, q(n);
for (auto &[u, v]: e) g[--deg[u]] = v;
while (true) {
    a.assign(n, -1), p.assign(n, -1);
    int t = 0;
    for (int i = 0; i < n; i++)
        if (l[i] == -1) q[t++] = a[i] = p[i] = i;
    bool match = false;
    for (int i = 0; i < t; i++) {
        int x = q[i];
        if (l[a[x]] != -1) continue;
        for (int j = deg[x]; j < deg[x + 1]; j++) {
            int y = g[j];
            if (r[y] == -1) {
                while (y != -1)
                    r[y] = x, swap(l[x], y), x = p[x];
                match = true;
                break;
            }
            if (p[r[y]] == -1)
                q[t++] = y = r[y], p[y] = x, a[y] = a[x];
        }
    }
    if (!match) break;
}
vector<pair<int, int>> ret;
for (int i = 0; i < n; i++)
    if (l[i] != -1) ret.emplace_back(i, l[i]);
return ret;
}

```

2.7 Single source shortest paths

2.7.1 Bellmanford

Grafo direcionado com pesos, n vértices, m arestas. Para um vértice especificado, diz a menor distância de todos os outros até ele. Funciona com peso negativo.

Claramente, se tiver ciclo negativo não existe esse caminho. Então dá pra detectar ciclo negativo também.

Roda em $O(nm)$

4b26aa

```

void solve() {
    vector<int> d(n, INF);
    d[v] = 0;
    vector<int> p(n, -1);

    for (;;) {
        bool any = false;
        for (Edge e : edges)
            if (d[e.a] < INF)
                if (d[e.b] > d[e.a] + e.cost) {
                    d[e.b] = d[e.a] + e.cost;
                    p[e.b] = e.a;
                    any = true;
                }
        if (!any) break;
    }

    if (d[t] == INF)
        cout << "No path from " << v << " to " << t << ".";
    else {
        vector<int> path;
        for (int cur = t; cur != -1; cur = p[cur])
            path.push_back(cur);
        reverse(path.begin(), path.end());

        cout << "Path from " << v << " to " << t << ": ";
        for (int u : path)
            cout << u << ' ';
    }
}

// CICLO NEGATIVO
// SE NA ENESIMA RELAXACAO OCORRER ALGUMA COISA
// E PQ TEM CICLO NEGATIVO

void solve() {
    vector<int> d(n, INF);
    d[v] = 0;

```

```

vector<int> p(n, -1);
int x;
for (int i = 0; i < n; ++i) {
    x = -1;
    for (Edge e : edges)
        if (d[e.a] < INF)
            if (d[e.b] > d[e.a] + e.cost) {
                d[e.b] = max(-INF, d[e.a] + e.cost);
                p[e.b] = e.a;
                x = e.b;
            }
}

if (x == -1)
    cout << "No negative cycle from " << v;
else {
    int y = x;
    for (int i = 0; i < n; ++i)
        y = p[y];

    vector<int> path;
    for (int cur = y;; cur = p[cur]) {
        path.push_back(cur);
        if (cur == y && path.size() > 1) break;
    }
    reverse(path.begin(), path.end());

    cout << "Negative cycle: ";
    for (int u : path)
        cout << u << ' ';
}
}

```

2.7.2 SPFA

Melhoramento do Bellmanford que toma vantagem do fato de que nem todas as tentativas de relaxação vão funcionar. Também dá pra usar pra detectar ciclo negativo.

O pior caso é $O(nm)$ também. Acontece que, na prática, é bem mais rápido, apesar de ser fácil de criar contra-exemplos em que roda em $O(nm)$. Mas tem gente

que diz que em média roda em $O(m)$.

03ab94

```

const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

bool spfa(int s, vector<int>& d) {
    int n = adj.size();
    d.assign(n, INF);
    vector<int> cnt(n, 0);
    vector<bool> inqueue(n, false);
    queue<int> q;

    d[s] = 0;
    q.push(s);
    inqueue[s] = true;
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        inqueue[v] = false;

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                if (!inqueue[to]) {
                    q.push(to);
                    inqueue[to] = true;
                    cnt[to]++;
                    if (cnt[to] > n) return false; //
                        negative cycle
                }
            }
        }
    }
    return true;
}

```

2.7.3 01 BFS

Imagina que os pesos são 0 ou 1. Dá pra resolver o problema com dijkstra em $O(|E|\log|V|)$. Mas aproveitando que os pesos são zero ou um tem como fazer isso em $O(|E|)$.

6bf989

```
vector<int> d(n, INF);
d[s] = 0;
deque<int> q;
q.push_front(s);
while (!q.empty()) {
    int v = q.front();
    q.pop_front();
    for (auto edge : adj[v]) {
        int u = edge.first;
        int w = edge.second;
        if (d[v] + w < d[u]) {
            d[u] = d[v] + w;
            if (w == 1)
                q.push_back(u);
            else
                q.push_front(u);
        }
    }
}
```

2.7.4 1-K BFS

Também conhecida como algoritmo de Dial. Usado quando os pesos são das arestas vão de 1 até um K pequeno. Tem complexidade de tempo $O(nk + m)$.

Aqui, bfs é uma array de $K + 1$ filas.

cf89f1

```
bfs[0].push(start);
d[start] = 0;
int pos = 0,
    num = 1; // I recommend to define a variable num — the
              number of vertexes that are in the queues
while (num > 0) {
    while (bfs[pos % (k + 1)].empty()) {
        ++pos;
```

```
    }
    int u = bfs[pos % (k + 1)].front();
    bfs[pos % (k + 1)].pop();
    --num;
    if (used[u]) // used vertex can be in other queues
    {
        continue;
    }
    used[u] = true;
    for (pair<int, int> edge : g[u]) {
        int cost = edge.first, w = edge.second;
        if (d[u] + cost < d[w]) {
            d[w] = d[u] + cost;
            bfs[d[w] % (k + 1)].push(w);
            ++num;
        }
    }
}
```

2.8 Outros

2.8.1 Aciclicidade de grafos

Ideia: vértice fica preto quando sai, cinza enquanto ainda não terminou ou branco se não tiver sido visitado.

bfb6d6

```
// PARA GRAFO DIRECIONADO

int n;
vector<vector<int>>> adj;
vector<char> color;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v) {
    color[v] = 1;
    for (int u : adj[v]) {
        if (color[u] == 0) {
            parent[u] = v;
            if (dfs(u)) return true;
        }
    }
    color[v] = 2;
    return false;
}
```

```

        } else if (color[u] == 1) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
    }
    color[v] = 2;
    return false;
}

void find_cycle() {
    color.assign(n, 0);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (color[v] == 0 && dfs(v)) break;
    }

    if (cycle_start == -1) {
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);
        reverse(cycle.begin(), cycle.end());

        cout << "Cycle found: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}

// PARA GRAFO NAO DIRECIONADO
int n;

```

```

vector<vector<int>> adj;
vector<bool> visited;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v, int par) { // passing vertex and its parent
    vertex
    visited[v] = true;
    for (int u : adj[v]) {
        if (u == par) continue; // skipping edge to parent
        vertex
        if (visited[u]) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
        parent[u] = v;
        if (dfs(u, parent[u])) return true;
    }
    return false;
}

void find_cycle() {
    visited.assign(n, false);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && dfs(v, parent[v])) break;
    }

    if (cycle_start == -1) {
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);
    }
}

```

```

        cout << "Cycle found: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}

```

2.8.2 Pintando arestas de árvores

Dada árvore G de n vértices, temos duas queries: em uma, pintamos uma aresta. Em outra, queremos saber o número de arestas pintadas entre dois vértices. Dá pra fazer cada uma dessas queries em $O(\log n)$. A etapa de pré-processamento leva $O(n)$.

```

df67ed
const int INF = 1000 * 1000 * 1000;

typedef vector<vector<int>> graph;

vector<int> dfs_list;
vector<int> edges_list;
vector<int> h;

void dfs(int v, const graph& g, const graph& edge_ids, int
    cur_h = 1) {
    h[v] = cur_h;
    dfs_list.push_back(v);
    for (size_t i = 0; i < g[v].size(); ++i) {
        if (h[g[v][i]] == -1) {
            edges_list.push_back(edge_ids[v][i]);
            dfs(g[v][i], g, edge_ids, cur_h + 1);
            edges_list.push_back(edge_ids[v][i]);
            dfs_list.push_back(v);
        }
    }
}

vector<int> lca_tree;
vector<int> first;

```

```

void lca_tree_build(int i, int l, int r) {
    if (l == r) {
        lca_tree[i] = dfs_list[l];
    } else {
        int m = (l + r) >> 1;
        lca_tree_build(i + i, l, m);
        lca_tree_build(i + i + 1, m + 1, r);
        int lt = lca_tree[i + i], rt = lca_tree[i + i + 1];
        lca_tree[i] = h[lt] < h[rt] ? lt : rt;
    }
}

void lca_prepare(int n) {
    lca_tree.assign(dfs_list.size() * 8, -1);
    lca_tree_build(1, 0, (int)dfs_list.size() - 1);

    first.assign(n, -1);
    for (int i = 0; i < (int)dfs_list.size(); ++i) {
        int v = dfs_list[i];
        if (first[v] == -1) first[v] = i;
    }
}

int lca_tree_query(int i, int tl, int tr, int l, int r) {
    if (tl == l && tr == r) return lca_tree[i];
    int m = (tl + tr) >> 1;
    if (r <= m) return lca_tree_query(i + i, tl, m, l, r);
    if (l > m) return lca_tree_query(i + i + 1, m + 1, tr, l,
        , r);
    int lt = lca_tree_query(i + i, tl, m, l, m);
    int rt = lca_tree_query(i + i + 1, m + 1, tr, m + 1, r);
    return h[lt] < h[rt] ? lt : rt;
}

int lca(int a, int b) {
    if (first[a] > first[b]) swap(a, b);
    return lca_tree_query(1, 0, (int)dfs_list.size() - 1,
        first[a], first[b]);
}

```

```

vector<int> first1 , first2;
vector<char> edge_used;
vector<int> tree1 , tree2;

void query_prepare(int n) {
    first1.resize(n - 1, -1);
    first2.resize(n - 1, -1);
    for (int i = 0; i < (int)edges_list.size(); ++i) {
        int j = edges_list[i];
        if (first1[j] == -1)
            first1[j] = i;
        else
            first2[j] = i;
    }

    edge_used.resize(n - 1);
    tree1.resize(edges_list.size() * 8);
    tree2.resize(edges_list.size() * 8);
}

void sum_tree_update(vector<int>& tree, int i, int l, int r,
    int j, int delta) {
    tree[i] += delta;
    if (l < r) {
        int m = (l + r) >> 1;
        if (j <= m)
            sum_tree_update(tree, i + i, l, m, j, delta);
        else
            sum_tree_update(tree, i + i + 1, m + 1, r, j,
                delta);
    }
}

int sum_tree_query(const vector<int>& tree, int i, int tl,
    int tr, int l, int r) {
    if (l > r || tl > tr) return 0;
    if (tl == l && tr == r) return tree[i];
    int m = (tl + tr) >> 1;
    if (r <= m) return sum_tree_query(tree, i + i, tl, m, l,
        r);

```

```

        if (l > m) return sum_tree_query(tree, i + i + 1, m + 1,
            tr, l, r);
        return sum_tree_query(tree, i + i, tl, m, l, m) +
            sum_tree_query(tree, i + i + 1, m + 1, tr, m + 1,
                r);
    }

int query(int v1, int v2) {
    return sum_tree_query(tree1, 1, 0, (int)edges_list.size()
        - 1, first[v1], first[v2] - 1) -
        sum_tree_query(tree2, 1, 0, (int)edges_list.size()
            - 1, first[v1], first[v2] - 1);
}

int main() {
    // reading the graph
    int n;
    scanf("%d", &n);
    graph g(n), edge_ids(n);
    for (int i = 0; i < n - 1; ++i) {
        int v1, v2;
        scanf("%d%d", &v1, &v2);
        --v1, --v2;
        g[v1].push_back(v2);
        g[v2].push_back(v1);
        edge_ids[v1].push_back(i);
        edge_ids[v2].push_back(i);
    }

    h.assign(n, -1);
    dfs(0, g, edge_ids);
    lca_prepare(n);
    query_prepare(n);

    for (;;) {
        if () {
            // request for painting edge x;
            // if start = true, then the edge is painted,
            // otherwise the painting
            // is removed

```

```

    edge_used[x] = start;
    sum_tree_update(tree1, 1, 0, (int)edges_list.
        size() - 1, first1[x], start ? 1 : -1);
    sum_tree_update(tree2, 1, 0, (int)edges_list.
        size() - 1, first2[x], start ? 1 : -1);
} else {
    // query the number of colored edges on the path
    // between v1 and v2
    int l = lca(v1, v2);
    int result = query(l, v1) + query(l, v2);
    // result - the answer to the request
}
}
}

```

2.8.3 Conectividade de vértices e arestas

A **conectividade de arestas** λ de um grafo é o número mínimo de arestas que precisa ser deletado para que o grafo fique desconexo. Dizemos que um conjunto S de arestas separa os vértices s e t quando, ao remover todas as arestas contidas em S do grafo G , eles ficam em componentes distintas. É claro que a conectividade é igual ao menor tamanho desse conjunto que separa s e t variando por todos s e t .

A **conectividade de vértices** κ de um grafo é o menor número de vértices tal que, ao removê-los, o grafo fica desconexo. De maneira análoga, dizemos que um conjunto T separa os vértices s e t se, ao remover todos os vértices de T do grafo, os vértices ficam em componentes distintas. Novamente, é só ir variando s e t e ir pegando o mínimo.

As desigualdades de Whitney dão uma relação entre κ , λ e δ (o menor grau dos vértices): $\kappa \leq \lambda \leq \delta$.

Para achar a conectividade de arestas, vai variando por todos os pares de vértices, bota s como source, t como sink, bota capacidade de 1 pra cada aresta e pega o fluxo desse grafo.

Para achar a conectividade de vértices, itera novamente por s e t . Mas antes disso, divide cada vértice $x \notin \{s, t\}$ em dois vértices x_1 e x_2 e bota uma aresta de x_1 até x_2 com capacidade 1. Além disso, substitua arestas (u, v) por duas arestas direcionadas (u_2, v_1) e (v_2, u_1) , ambas com 1 de capacidade. Aí é só pegar o fluxo nesse grafo.

2.8.4 Segunda melhor MST

Faz a MST normal. Pra cada aresta que não tá, tento botar ela da seguinte forma. Primeiro, bota ela forçadamente, criando um ciclo. Depois, pega o máximo desse ciclo não incluindo a aresta original. Remove a aresta correspondente a esse máximo temporariamente e lembra da diferença de peso entre a que eu botei e a que removi. Lembra esse peso junto com a aresta que eu botei. Daí é só ir repetindo isso e lembrar da menor diferença de peso.

Complexidade $O(E \log V)$

d2bd48

```

struct edge {
    int s, e, w, id;
    bool operator<(const struct edge& other) { return w <
        other.w; }
};
typedef struct edge Edge;

const int N = 2e5 + 5;
long long res = 0, ans = 1e18;
int n, m, a, b, w, id, l = 21;
vector<Edge> edges;
vector<int> h(N, 0), parent(N, -1), size(N, 0), present(N,
    0);
vector<vector<pair<int, int>>> adj(N), dp(N, vector<pair<int,
    int>>(1));
vector<vector<int>> up(N, vector<int>(1, -1));

pair<int, int> combine(pair<int, int> a, pair<int, int> b) {
    vector<int> v = {a.first, a.second, b.first, b.second};
    int topTwo = -3, topOne = -2;
    for (int c : v) {
        if (c > topOne) {
            topTwo = topOne;
            topOne = c;
        } else if (c > topTwo && c < topOne) {
            topTwo = c;
        }
    }
    return {topOne, topTwo};
}

```



```

void dfs(int u, int par, int d) {
    h[u] = 1 + h[par];
    up[u][0] = par;
    dp[u][0] = {d, -1};
    for (auto v : adj[u]) {
        if (v.first != par) {
            dfs(v.first, u, v.second);
        }
    }
}

pair<int, int> lca(int u, int v) {
    pair<int, int> ans = {-2, -3};
    if (h[u] < h[v]) {
        swap(u, v);
    }
    for (int i = 1 - 1; i >= 0; i--) {
        if (h[u] - h[v] >= (1 << i)) {
            ans = combine(ans, dp[u][i]);
            u = up[u][i];
        }
    }
    if (u == v) {
        return ans;
    }
    for (int i = 1 - 1; i >= 0; i--) {
        if (up[u][i] != -1 && up[v][i] != -1 && up[u][i] !=
            up[v][i]) {
            ans = combine(ans, combine(dp[u][i], dp[v][i]));
            u = up[u][i];
            v = up[v][i];
        }
    }
    ans = combine(ans, combine(dp[u][0], dp[v][0]));
    return ans;
}

int main(void) {
    cin >> n >> m;

```

```

    for (int i = 1; i <= n; i++) {
        parent[i] = i;
        size[i] = 1;
    }
    for (int i = 1; i <= m; i++) {
        cin >> a >> b >> w; // 1-indexed
        edges.push_back({a, b, w, i - 1});
    }
    sort(edges.begin(), edges.end());
    for (int i = 0; i <= m - 1; i++) {
        a = edges[i].s;
        b = edges[i].e;
        w = edges[i].w;
        id = edges[i].id;
        if (unite_set(a, b)) {
            adj[a].emplace_back(b, w);
            adj[b].emplace_back(a, w);
            present[id] = 1;
            res += w;
        }
    }
    dfs(1, 0, 0);
    for (int i = 1; i <= 1 - 1; i++) {
        for (int j = 1; j <= n; ++j) {
            if (up[j][i - 1] != -1) {
                int v = up[j][i - 1];
                up[j][i] = up[v][i - 1];
                dp[j][i] = combine(dp[j][i - 1], dp[v][i -
                    1]);
            }
        }
    }
    for (int i = 0; i <= m - 1; i++) {
        id = edges[i].id;
        w = edges[i].w;
        if (!present[id]) {
            auto rem = lca(edges[i].s, edges[i].e);
            if (rem.first != w) {
                if (ans > res + w - rem.first) {
                    ans = res + w - rem.first;
                }
            }
        }
    }
}

```

```

    }
    } else if (rem.second != -1) {
        if (ans > res + w - rem.second) {
            ans = res + w - rem.second;
        }
    }
}
cout << ans << "\n";
return 0;
}

```

2.8.5 Número de spanning trees

Dado grafo representado como lista de adjacências, encontrar o número de árvores geradoras distintas.

Seja A a matriz de adjacência do grafo G . Seja D a matriz de graus do grafo i.e. $D_{u,u}$ é o grau do vértice u . A matriz laplaciana do grafo é $L = D - A$. Pelo **teorema de Kirchhoff**, todos os cofatores dessa matriz são iguais e também são iguais ao número de árvores geradores distintos.

Então é só tirar a última linha e coluna de L , calcular o determinante e tirar o valor absoluto. Isso pode ser feito em $O(n^3)$ com eliminação gaussiana.

2.8.6 Prufer code

É uma maneira de codificar uma árvore rotulada em uma sequência de números de maneira única. É uma bijeção entre as árvores geradoras de um grafo completo e as sequências numéricas de $n - 2$ inteiros no intervalo $[0, n - 1]$.

Ajuda a resolver problemas combinatórios.

Aqui, é mostrado o número de maneiras de adicionar arestas a um grafo de modo que ele fique conexo. Não são consideradas árvores de um único vértice.

Construção: selecionamos a folha da árvore com o menor número. Remove ele da árvore. Escreve o número do vértice que tava conectado à folha. Depois de $n - 2$ iterações, sobrarão 2 vértices e o algoritmo acaba. Aqui um jeito de fazer isso linearmente.

```

be7dce
vector<vector<int>> adj;
vector<int> parent;

void dfs(int v) {

```

```

    for (int u : adj[v]) {
        if (u != parent[v]) {
            parent[u] = v;
            dfs(u);
        }
    }
}

vector<int> pruefer_code() {
    int n = adj.size();
    parent.resize(n);
    parent[n - 1] = -1;
    dfs(n - 1);

    int ptr = -1;
    vector<int> degree(n);
    for (int i = 0; i < n; i++) {
        degree[i] = adj[i].size();
        if (degree[i] == 1 && ptr == -1) ptr = i;
    }

    vector<int> code(n - 2);
    int leaf = ptr;
    for (int i = 0; i < n - 2; i++) {
        int next = parent[leaf];
        code[i] = next;
        if (--degree[next] == 1 && next < ptr) {
            leaf = next;
        } else {
            ptr++;
            while (degree[ptr] != 1)
                ptr++;
            leaf = ptr;
        }
    }

    return code;
}

```

Propriedades:

1. Depois de construir, dois vértices faltam. Um deles é o maior $n - 1$. Mas nada pode ser dito sobre o outro.
2. Todo vértice aparece no código de Prufer o grau dele menos uma vez.

Dá pra restaurar a árvore a partir do código de Prufer linearmente

```

350d67
vector<pair<int, int>> pruefer_decode(vector<int> const&
code) {
    int n = code.size() + 2;
    vector<int> degree(n, 1);
    for (int i : code)
        degree[i]++;

    int ptr = 0;
    while (degree[ptr] != 1)
        ptr++;
    int leaf = ptr;

    vector<pair<int, int>> edges;
    for (int v : code) {
        edges.emplace_back(leaf, v);
        if (--degree[v] == 1 && v < ptr) {
            leaf = v;
        } else {
            ptr++;
            while (degree[ptr] != 1)
                ptr++;
            leaf = ptr;
        }
    }
    edges.emplace_back(leaf, n - 1);
    return edges;
}

```

Fórmula de Cayley: O número de árvores geradoras num grafo completo é trivialmente n^{n-2} pelo código de Prufer.

Número de maneiras de tornar um grafo conexo: Dado um grafo de k componentes, quero o número de maneiras de adicionar $k - 1$ arestas de modo que o grafo fique conexo. É como se tivéssemos um grafo de k componentes e quiséssemos

aplicar a fórmula de Cayley. Sejam d_i os graus. Temos que $\sum_{i=1}^k d_i = 2k - 2$. Isso quer dizer que o vértice i aparece $d_i - 1$ vezes no código de Prufer. O código de Prufer de uma árvore de k vértices tem tamanho $k - 2$, então o número de maneiras de escolher um código com $k - 2$ números em que o número i aparece exatamente $d_i - 1$ vezes é igual ao coeficiente multinomial

$$\binom{k-2}{d_1-1, \dots, d_k-1}$$

O fato de que cada aresta adjacente ao vértice i multiplica a resposta por s_i (a quantidade de caras na componente) implica que a resposta é

$$\sum_{d_i \geq 1, \sum d_i = 2k-2} s_1^{d_1} \dots s_k^{d_k} \binom{k-2}{d_1-1, \dots, d_k-1}$$

Trivialmente, isso é

$$s_1 \dots s_k (s_1 + \dots + s_k)^{k-2} = s_1 \dots s_k n^{k-2}$$

Acidentalmente essa fórmula também funciona para $k = 1$.

2.8.7 MST direcionada

```

179d68
/** Source:
 * https://github.com/kth-competitive-programming/kactl/blob
 * /master/content/graph/DirectedMST.h */

/** Finds a minimum spanning tree/arborescence rooted at
 some node of a directed graph */

struct RollbackUF { // RollBack union find, O(logn)
    vector<int> e;
    vector<pii> st;
    RollbackUF(int n) : e(n, -1) {}
    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : find(e[x]); }
    int time() { return st.size(); }
    void rollback(int t) {
        for (int i = time(); i > t;)

```

```

        e[st[i].first] = st[i].second;
        st.resize(t);
    }
    bool merge(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        st.push_back({a, e[a]});
        st.push_back({b, e[b]});
        e[a] += e[b];
        e[b] = a;
        return true;
    }
};

struct Edge {
    int a, b;
    ll w;
};

struct Node { /// lazy skew heap node
    Edge key;
    Node *l, *r;
    ll delta;
    void prop() {
        key.w += delta;
        if (l) l->delta += delta;
        if (r) r->delta += delta;
        delta = 0;
    }
    Edge top() {
        prop();
        return key;
    }
};

Node* merge(Node* a, Node* b) {
    if (!a || !b) return a ? b;
    a->prop(), b->prop();
    if (a->key.w > b->key.w) swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));

```

```

        return a;
    }
    void pop(Node*& a) {
        a->prop();
        a = merge(a->l, a->r);
    }

pair<ll, vector<int>>> dmst(int n, int r, vector<Edge> const&
    g) { // O(E log V)
    RollbackUF uf(n);
    vector<Node*> heap(n);
    for (Edge e : g)
        heap[e.b] = merge(heap[e.b], new Node{e});
    ll res = 0;
    vector<int> seen(n, -1), path(n), par(n);
    seen[r] = r;
    vector<Edge> Q(n), in(n, Edge{-1, -1, 0}), comp;
    deque<tuple<int, int, vector<Edge>>>> cys;
    for (int s = 0; s < n; s++) {
        int u = s, qi = 0, w;
        while (seen[u] < 0) {
            if (!heap[u]) return {-1, {}};
            Edge e = heap[u]->top();
            heap[u]->delta -= e.w, pop(heap[u]);
            Q[qi] = e, path[qi++] = u, seen[u] = s;
            res += e.w, u = uf.find(e.a);
            if (seen[u] == s) { /// found cycle, contract
                Node* cyc = 0;
                int end = qi, time = uf.time();
                do
                    cyc = merge(cyc, heap[w = path[--qi]]);
                while (uf.merge(u, w));
                u = uf.find(u), heap[u] = cyc, seen[u] = -1;
                cys.push_front({u, time, {&Q[qi], &Q[end
                    ]}});
            }
        }
        for (int i = 0; i < qi; i++)
            in[uf.find(Q[i].b)] = Q[i];
    }
}

```

```

for (auto& [u, t, comp] : cycs) { // restore sol (
optional)
uf.rollback(t);
Edge inEdge = in[u];
for (auto& e : comp)
in[uf.find(e.b)] = e;
in[uf.find(inEdge.b)] = inEdge;
}
for (int i = 0; i < n; i++)
par[i] = in[i].a;
return {res, par};
}

```

2.8.8 Árvore dominadora

179d68

```

/** Source:
 * https://github.com/kth-competitive-programming/kactl/blob
 * /master/content/graph/DirectedMST.h */

/** Finds a minimum spanning tree/arborescence rooted at
some node of a directed graph */

struct RollbackUF { // RollBack union find, O(logn)
vector<int> e;
vector<pii> st;
RollbackUF(int n) : e(n, -1) {}
int size(int x) { return -e[find(x)]; }
int find(int x) { return e[x] < 0 ? x : find(e[x]); }
int time() { return st.size(); }
void rollback(int t) {
for (int i = time(); i > t;)
e[st[i].first] = st[i].second;
st.resize(t);
}
bool merge(int a, int b) {
a = find(a), b = find(b);
if (a == b) return false;
if (e[a] > e[b]) swap(a, b);
st.push_back({a, e[a]});
}

```

```

st.push_back({b, e[b]});
e[a] += e[b];
e[b] = a;
return true;
}
};

struct Edge {
int a, b;
ll w;
};

struct Node { /// lazy skew heap node
Edge key;
Node *l, *r;
ll delta;
void prop() {
key.w += delta;
if (l) l->delta += delta;
if (r) r->delta += delta;
delta = 0;
}
Edge top() {
prop();
return key;
}
};

Node* merge(Node* a, Node* b) {
if (!a || !b) return a ?: b;
a->prop(), b->prop();
if (a->key.w > b->key.w) swap(a, b);
swap(a->l, (a->r = merge(b, a->r)));
return a;
}

void pop(Node*& a) {
a->prop();
a = merge(a->l, a->r);
}

pair<ll, vector<int>> dmst(int n, int r, vector<Edge> const&

```

```

g) { // O(E log V)
RollbackUF uf(n);
vector<Node*> heap(n);
for (Edge e : g)
    heap[e.b] = merge(heap[e.b], new Node{e});
ll res = 0;
vector<int> seen(n, -1), path(n), par(n);
seen[r] = r;
vector<Edge> Q(n), in(n, Edge{-1, -1, 0}), comp;
deque<tuple<int, int, vector<Edge>>> cys;
for (int s = 0; s < n; s++) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
        if (!heap[u]) return {-1, {}};
        Edge e = heap[u] -> top();
        heap[u] -> delta -= e.w, pop(heap[u]);
        Q[qi] = e, path[qi++] = u, seen[u] = s;
        res += e.w, u = uf.find(e.a);
        if (seen[u] == s) { /// found cycle, contract
            Node* cyc = 0;
            int end = qi, time = uf.time();
            do
                cyc = merge(cyc, heap[w = path[--qi]]);
            while (uf.merge(u, w));
            u = uf.find(u), heap[u] = cyc, seen[u] = -1;
            cys.push_front({u, time, {&Q[qi], &Q[end
                ]}});
        }
    }
    for (int i = 0; i < qi; i++)
        in[uf.find(Q[i].b)] = Q[i];
}

for (auto& [u, t, comp] : cys) { // restore sol (
optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp)
        in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
}

```

```

}
for (int i = 0; i < n; i++)
    par[i] = in[i].a;
return {res, par};
}

```

2.8.9 Conectividade dinâmica

```

2f6aaa
#include <bits/stdc++.h>
using namespace std;

using ii = pair<int, int>;

struct Dsu {
    int n;
    vector<int> pai, w;
    vector<ii> rb;

    Dsu(int n_) : n(n_), pai(n + 1), w(n + 1, 1) { iota(pai.begin(), pai.end(), 0); }

    int find(int x) {
        if (pai[x] == x) return x;
        return find(pai[x]);
    }

    void join(int a, int b) {
        a = find(a), b = find(b);

        if (a != b) {
            if (w[a] < w[b]) swap(a, b);
            rb.emplace_back(b, pai[b]);
            rb.emplace_back(a, w[a]);

            pai[b] = a;
            w[a] += w[b];
            n--;
        } else {
            rb.emplace_back(0, 0);
        }
    }
}

```

```

}

void rollback() {
    if (rb.back() == ii{0, 0}) {
        rb.pop_back();
        return;
    }

    auto f = [this](auto& v) {
        auto [pos, val] = this->rb.back();
        this->rb.pop_back();
        v[pos] = val;
    };

    f(w);
    f(pai);
    n++;
}

};

struct DynConnectivity {
    int n, k;
    vector<vector<ii>> tree;
    Dsu dsu;

    DynConnectivity(int n_, int k_) : n(n_), k(k_), tree((k
        + 1) * 4), dsu(n) {};

    void add(int no, int l, int r, int a, int b, ii const& u
        ) {
        if (a <= l and r <= b) {
            tree[no].push_back(u);
            return;
        }
        if (l == r) return;

        int m = (l + r) / 2;

        if (a <= m) add(no * 2, l, m, a, b, u);
        if (b > m) add(no * 2 + 1, m + 1, r, a, b, u);
    }
};

```

```

}

// marks the existence of the edge u between the times a
// and b
void add(int a, int b, ii const& u) { add(1, 0, k, a, b,
    u); }

void solve(vector<int>& res, int no, int l, int r) {
    for (auto& [a, b] : tree[no]) {
        dsu.join(a, b);
    }

    if (l == r) {
        res[l] = dsu.n;
    } else {
        int m = (l + r) / 2;
        solve(res, no * 2, l, m);
        solve(res, no * 2 + 1, m + 1, r);
    }

    for (int i = 0; i < (int)tree[no].size(); i++)
        dsu.rollback();
}

vector<int> solve() {
    vector<int> v(k + 1);
    solve(v, 1, 0, k);
    return v;
}

};

// example solution
// solution for the problem – Dynamic Connectivity
// https://cses.fi/problemset/task/2133/
// given a graph with n vertex and m edges originally and k
// toggle edge
// operations, find the number of components before the
// first and
// after each operation

```

```

int main() {
    ios_base::sync_with_stdio(false), cin.tie(nullptr);

    int n, m, k;
    cin >> n >> m >> k;

    DynConnectivity dyn(n, k);

    set<ii> g;
    map<ii, int> gt;

    while (m--) {
        int x, y;
        cin >> x >> y;

        if (x > y) swap(x, y);

        gt[{x, y}] = 0;
    }

    // for every edge finds the times of start and end of
    // the edge
    for (int i = 1; i <= k; i++) {
        int op, x, y;
        cin >> op >> x >> y;

        if (x > y) swap(x, y);

        if (op == 1) {
            gt[{x, y}] = i;
        } else {
            auto it = gt.find({x, y});
            int ini = it->second;
            gt.erase(it);

            dyn.add(ini, i - 1, {x, y});
        }
    }

    // the edges that exist after everything is done

```

```

    for (auto [p, key] : gt) {
        dyn.add(key, k, p);
    }

    auto res = dyn.solve();

    for (int i = 0; i <= k; i++) {
        cout << res[i] << " \n"[i == k];
    }
}

```

2.8.10 Centróide

9d2552

```

#include <algorithm>
#include <array>
#include <iostream>
#include <random>
#include <string>
#include <tuple>
#include <utility>
#include <vector>

#include <cstdio>

using namespace std;
using ll = long long;

int const maxn = 202020;

vector<vector<int>> g;
vector<char> cvis;
vector<int> sz;

namespace Centroid {

int dfs_sz(int x, int p = -1) {
    sz[x] = 1;
    for (int u : g[x]) {
        if (u == p or cvis[u]) continue;
        sz[x] += dfs_sz(u, x);
    }
}

```



```

    }
    return sz[x];
}

int dfs_cent(int tot, int x, int p = -1) {
    int cent = x;
    for (int u : g[x]) {
        if (u == p or cvis[u]) continue;
        if (sz[u] > tot / 2) {
            sz[x] -= sz[u];
            sz[u] += sz[x];
            cent = dfs_cent(tot, u, x);
            break;
        }
    }
    return cent;
}

int centroid(int x) {
    if (sz[x] == 0) dfs_sz(x);
    int cent = dfs_cent(sz[x], x);
    return cent;
}

} // end namespace Centroid

// EXAMPLE PROBLEM
// COUNT NUMBER OF PATHS WITH LENGHT IN [L, L+1, ..., R]
struct PartialSum {
    vector<ll> path_cnt_acc, path_cnt;
    int max_idx = 0;

    PartialSum(int n) : path_cnt_acc(n), path_cnt(n) {};

    void clear() {
        fill(begin(path_cnt), begin(path_cnt) + max_idx + 1,
            0);
        max_idx = 0;
    }
}

```

```

inline void add(int i, ll d) {
    max_idx = max(max_idx, i);
    path_cnt[i] += d;
}

void recompute() {
    for (int i = 0; i <= max_idx; i++) {
        path_cnt_acc[i] = path_cnt[i] + (i > 0 ?
            path_cnt_acc[i - 1] : 0);
    }
}

inline ll get_range(int l, int r) {
    ll ans = path_cnt_acc[min(max_idx, r)];
    if (l > 0) ans -= path_cnt_acc[min(max_idx, l - 1)];
    return ans;
}

};

PartialSum sum(maxn);

void dfs_add(int x, int p = -1, int d = 1, int to_add = 1) {
    sum.add(d, to_add);
    for (int u : g[x]) {
        if (u == p or cvis[u]) continue;
        dfs_add(u, x, d + 1, to_add);
    }
}

long long dfs_get(int l, int r, int x, int p = -1, int d =
1) {
    if (d > r) return 0;
    ll ans = sum.get_range(l - d, r - d);
    for (int u : g[x]) {
        if (u == p or cvis[u]) continue;
        ans += dfs_get(l, r, u, x, d + 1);
    }
    return ans;
}

```

```

long long solve(int l, int r, int x) {
    x = Centroid::centroid(x);
    cvis[x] = 1;

    long long ans = 0;

    if (g[x].empty()) return 0;
    sort(g[x].begin(), g[x].end(),
        [&](auto lhs, auto rhs) { return tie(cvis[lhs], sz[
            lhs]) < tie(cvis[rhs], sz[rhs]); });

    sum.add(0, 1);
    sum.recompute();
    int last = -1;
    for (int u : g[x]) {
        if (cvis[u]) break;
        if (last != -1) {
            dfs_add(last);
            sum.recompute();
        }
        ans += dfs_get(l, r, u);
        last = u;
    }

    sum.clear();

    for (int u : g[x]) {
        if (cvis[u]) break;
        ans += solve(l, r, u);
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false), cin.tie(nullptr);

    int n, l, r;
    cin >> n >> l >> r;

    g.resize(n + 1);

```

```

    cvis.resize(n + 1);
    sz.resize(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int a, b;
        cin >> a >> b;

        g[a].push_back(b);
        g[b].push_back(a);
    }

    cout << solve(l, r, 1) << "\n";
}

```

2.8.11 Isomorfismo de árvores

```

// grafo g, verts de 1 ate n
map<vi, int> mp;
int dfs_hash(int u, int p=0) {
    vi vec;

    for(int v : g[u]) {
        if(v != p) {
            int h = dfs_hash(v, u);
            vec.pb(h);
        }
    }

    sort(vec.begin(), vec.end());
    if(mp.count(vec)) return mp[vec];
    return mp[vec] = mp.size();
}

// DAQUI PRA BAIXO EH SE VOCE PRECISA COMPARAR UNROOTED
int lvl[maxn], pai[maxn];
void dfs(int u, int p=0, int l=0) {
    pai[u] = p;
    lvl[u] = l;
    for(int v : g[u])
        if(v != p) dfs(v, u, l + 1);
}

```

```

}

pii find_centers() {
    int rr;
    for(int k = 0; k < 2; ++k) {
        int r = k ? rr : 1;

        dfs(r);

        rr = 1;
        for(int i = 1; i <= n; ++i)
            if(lvl[i] > lvl[rr]) rr = i;

        if(!k) r = rr;
    }

    vector<int> aux;
    for(; rr; rr = pai[rr])
        aux.pb(rr);

    int sz = aux.size();
    return {aux[sz / 2], (sz%2) ? 0 : aux[sz / 2 - 1]};
}

pii hash_unrooted() {
    auto [c1, c2] = find_centers();
    int h1 = dfs_hash(c1);
    int h2 = c2 != 0 ? dfs_hash(c2) : -1;
    if(h1 > h2) swap(h1, h2);
    return {h1, h2};
}

```

2.8.12 Caminho euleriano

Problema 2.8.1: Dado um grafo não direcionado G , de N vértices e M arestas um **Caminho Euleriano** é uma sequência de vértices $(v_0, \dots, v_M)$ e uma sequência de arestas $(e_0, \dots, e_{M-1})$ que satisfaça as seguintes condições:

- $(e_0, \dots, e_{M-1})$ é uma permutação de $(0, \dots, M-1)$
- Para $0 \leq i < M-1$ a aresta e_i conecta v_i e v_{i+1}

Algoritmo:

402faa

```

// complexidade O(M), retorna vector dos vertices e das
// arestas onde cada elist[i] conecta vlist[i], vlist[i+1]
// elist são os indices das arestas (conforme a ordem de
// insercao, 0...M-1) do caminho euleriano
// se vlist retornar vazio não existe caminho euleriano
struct edge {
    int t, id;
};

pair<vector<int>, vector<int>>> eulerpath_undirected(int N,
    int M, const vector<pair<int, int>>& edges) {
    vector<vector<edge>> G(N);
    for (int i = 0; i < M; i++) {
        auto e = edges[i];
        G[e.first].push_back(edge{e.second, i});
        G[e.second].push_back(edge{e.first, i});
    }
    int start = -1, cnt = 0;
    for (int i = 0; i < N; i++) {
        if (G[i].size() % 2 == 1) {
            start = i;
            cnt++;
        }
    }
    if (cnt >= 4) return {vector<int>(), vector<int>()};
    if (cnt == 0) start = (M != 0 ? edges[0].s : 0);
    vector<bool> used(M, false);
    vector<int> vlist, elist;
    auto dfs = [&](auto& self, int v) -> void {
        while (!G[v].empty()) {
            edge e = G[v].back();
            G[v].pop_back();
            if (!used[e.id]) {
                used[e.id] = true;
                self(self, e.t);
                elist.push_back(e.id);
            }
        }
        vlist.push_back(v);
    }
}

```

```

};
dfs(dfs, start);
reverse(vlist.begin(), vlist.end());
reverse(elist.begin(), elist.end());
if (elist.size() != M) return {vector<int>(), vector<int>
    >()};

return {vlist, elist};
}

```

2.9 2-SAT

Problema 2.9.1: Desejamos selecionar valores booleanos para variáveis $x_1, x_2, x_3, \dots, x_n$ para satisfazer uma expressão booleana (SAT). A expressão deve estar em CNF (forma normal conjuntiva), que é a conjunção de várias cláusulas, onde cada cláusula é uma disjunção de literais (variáveis ou negação de variáveis). O 2-SAT é uma restrição do problema onde cada cláusula tem apenas 2 literais. Exemplo em 3 variáveis:

$$(a \vee \neg b) \wedge (\neg a \vee b) \wedge (\neg a \vee \neg b) \wedge (a \vee \neg c)$$

Algoritmo 2-SAT:

9ee997

```

// Complexidade O(N+M) n numero de variaveis, m numero de
// clausulas
struct TwoSatSolver {
    int n_vars;
    int n_vertices;
    vector<vector<int>> adj, adj_t;
    vector<bool> used;
    vector<int> order, comp;
    vector<bool> assignment;

    TwoSatSolver(int n_vars) : n_vars(n_vars), n_vertices
        (2 * n_vars), adj(n_vertices), adj_t(n_vertices),
        used(n_vertices), order(), comp(n_vertices, -1),
        assignment(n_vars) {
        order.reserve(n_vertices);
    }
}

```

```

void dfs1(int v) {
    used[v] = true;
    for (int u : adj[v]) {
        if (!used[u])
            dfs1(u);
    }
    order.push_back(v);
}

void dfs2(int v, int cl) {
    comp[v] = cl;
    for (int u : adj_t[v]) {
        if (comp[u] == -1)
            dfs2(u, cl);
    }
}

bool solve_2SAT() {
    order.clear();
    used.assign(n_vertices, false);
    for (int i = 0; i < n_vertices; ++i) {
        if (!used[i])
            dfs1(i);
    }

    comp.assign(n_vertices, -1);
    for (int i = 0, j = 0; i < n_vertices; ++i) {
        int v = order[n_vertices - i - 1];
        if (comp[v] == -1)
            dfs2(v, j++);
    }

    assignment.assign(n_vars, false);
    for (int i = 0; i < n_vertices; i += 2) {
        if (comp[i] == comp[i + 1])
            return false;
        assignment[i / 2] = comp[i] > comp[i + 1];
    }
    return true;
}

```

```

void add_disjunction(int a, bool na, int b, bool nb) {
    // na and nb signify whether a and b are to be
    // negated
    a = 2 * a ^ na;
    b = 2 * b ^ nb;
    int neg_a = a ^ 1;
    int neg_b = b ^ 1;
    adj[neg_a].push_back(b);
    adj[neg_b].push_back(a);
    adj_t[b].push_back(neg_a);
    adj_t[a].push_back(neg_b);
}

// static void example_usage() {
//     TwoSatSolver solver(3); // a, b, c
//     solver.add_disjunction(0, false, 1, true); //
//     a v not b
//     solver.add_disjunction(0, true, 1, true); // not
//     a v not b
//     solver.add_disjunction(1, false, 2, false); //
//     b v c
//     solver.add_disjunction(0, false, 0, false); //
//     a v a
// }
// if (solver.solve_2SAT() == false) {
//     cout << "s UNSATISFIABLE\n";
//     return 0;
// }

// cout << "s SATISFIABLE\nv ";
// for (int i = 0; i < n; i++) {
//     if (solver.assignment[i]) cout << i+1 << " "; //
//     para trues
//     else cout << -(i+1) << " "; // para falses
// }
// };

```

3 DP

3.1 Principais problemas

3.1.1 Knapsack 0-1

eaaf00

```
// 0 indexado
ll v[maxn], p[maxn], dp[MAXCAP];
ll n, C;

ll ks() {
    for(int i = 0; i < n; i++)
        for(ll c = C; c >= p[i]; c--)
            dp[c] = max(dp[c], dp[c-p[i]] + v[i]);
    return dp[C];
}
```

3.1.2 LIS

2b61ec

```
// 1 indexado
ll a[maxn], dp[maxn];
ll n;

int lis() {
    for(int i = 1; i <= n; i++) dp[i] = inf;
    for(int i = 1; i <= n; i++) {
        int x = lower_bound(dp+1, dp+1+n, a[i]) - dp-1;
        dp[x+1] = min(dp[x+1], a[i]);
    }

    for(int i = n; i > 0; i--)
        if(dp[i] != inf) return i;
    return 0;
}
```

3.1.3 Edit Distance

Problema 3.1.1: Dadas strings s e t de tamanho n e m (1-indexado), pode-se fazer as seguintes operações: adicionar, alterar ou remover um caractere de uma

string. Calcule a **edit distance**, que é o menor número de operações para deixar ambas strings iguais.

Solução: Note que remover um char de uma string equivale a adicionar na outra, então temos duas operações. Considere a dp de $d_{i,j}$ que é a edit distance considerando só até os caracteres i e j das strings. É fácil ver que:

$$d_{i,j} = \min\{d_{i-1,j-1} + e_{i,j}, d_{i-1,j} + 1, d_{i,j-1} + 1\}$$

Onde $e_{i,j} = 1$ se $s_i = t_j$, e é 0 caso contrário. Note que só precisa de $i-1$ e $j-1$ pois, para posições anteriores, terá que adicionar um caractere, e na posição a frente bastaria trocar essa adição por uma edição. A resposta é $d_{n,m}$ e a complexidade é $O(nm)$.

8cfc9f

```
string s, t;
int dp[maxn][2];

int ed_dist() {
    int c = 1;
    for(int i = 1; i <= sza(t); i++) dp[i][0] = i;

    for(int i = 1; i <= sza(s); i++, c^=1) {
        dp[0][c] = i;
        for(int j = 1; j <= sza(t); j++) {
            dp[j][c] = dp[j-1][c^1] + (s[i-1] != t[j-1]);
            dp[j][c] = min(dp[j][c], 1+min(dp[j][c^1], dp[j-1][c]));
        }
    }

    return dp[t.size()][c^1];
}
```

3.2 Otimização

3.3 Outras técnicas

3.3.1 Convex Hull Trick

Problema 3.3.1: Suponha que você tenha um conjunto de funções lineares, isto é, $S = \{kx + c | k, c \in \mathbb{Z}\}$. Você pode fazer duas coisas: adicionar funções ao conjunto e fazer queries que pedem, dado x , a função f que minimiza $f(x)$.

Solução: Interprete as funções como pontos (k, c) no plano. Dado uma query x , você quer encontrar o ponto (k, c) que minimiza $(x, 1) \cdot (k, c)$. É possível ver que os pontos que podem ser solução formam a parte de baixo de um fecho convexo. A intuição da resposta é pegar a reta normal a $(x, 1)$ (que tem inclinação $-x$), e ver o primeiro ponto que ela toca. As arestas do fecho estão ordenadas em relação à sua inclinação, então basta encontrar o ponto cuja inclinação das arestas passa $-x$ (isto é, a aresta anterior do ponto é menor que $-x$, e a próxima é $\geq -x$). Adicionar novos pontos deve ser feita usando uma estrutura de dados ordenada que permite inserção e remoção arbitrária (tipo set) para que as adições possam ser feitas em qualquer ordem.

422395

```
struct Line {
    mutable ll k, c, m;
    bool operator<(Line o) const { return k < o.k; }
    bool operator<(ll x) const { return m < x; }
};

struct LineCont : multiset<Line, less<>> {
    ll div(ll a, ll b) { // divisao inteira com negativo
        return a/b - ((a^b) < 0 && a%b);
    }
    bool ncon(iterator a, iterator b) { // not convex
        if(b == end()) { a->m = inf; return 0; }
        if(a->k == b->k) a->m = (a->c > b->c) ? -inf : inf;
        else a->m = div(b->c - a->c, b->k - a->k);
        return a->m >= b->m;
    }
    void add(ll k, ll c) {
        auto z = insert({k, c, 0}), y = z++, x = y;
        while(ncon(y, z)) z = erase(z);
        if(x != begin() && ncon(--x, y)) ncon(x, erase(y));
        while((y = x) != begin() && (--x)->m >= y->m)
            ncon(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(-x);
        return l.k * x + l.c;
    }
};
```

Problema 3.3.2: Uma versão mais simples do problema anterior, mas as adições são com k crescente e as consultas podem ser feitas num intervalo da convex hull.

Solução: Solução parecida com a anterior, mas dessa vez se utiliza um vetor para no lugar do set para permitir as queries em intervalo. Como as adições são ordenadas, apenas o final do vetor é modificado.

cffe1c

```
struct Line {
    ll k, c, m;
    bool operator<(Line o) const { return k < o.k; }
    bool operator<(ll x) const { return m < x; }
};

vector<Line> cht;

ll _div(ll a, ll b) { // divisao inteira com negativo
    return a/b - ((a^b) < 0 && a%b);
}

void add(ll k, ll c) {
    ll m;
    while(!cht.empty() && (m = _div(c-cht.back().c, k-cht.back().k)) >= cht.back().m)
        cht.pp();

    if(cht.empty()) m = inf;
    cht.pb({k, c, m});
}

ll query(ll x, int beg, int end) {
    assert(!cht.empty() && beg < end);
    auto l = *lower_bound(cht.begin()+beg, cht.begin()+end, -x);
    return l.k * x + l.c;
}
```

3.3.2 Divide and Conquer

Para recorrências da seguinte forma:

$$dp(i, j) = \min_{0 \leq k \leq j} dp(i-1, k-1) + C(k, j)$$

Para $0 \leq i < m$ e $0 \leq j < n$, onde $C(k, j)$ é uma função de custo e $dp(i, j) = 0$ para $j < 0$. Seja $opt(i, j)$ o k que minimiza. Para a trick funcionar, opt deve ser monótona, ou seja, $opt(i, j) \leq opt(i, j+1)$. Isso vale, por exemplo, se C satisfaz a desigualdade quadrangular.

Digamos que sabemos o valor de $opt(i, j)$. Então, para $j' \leq j$, $opt(i, j') \leq opt(i, j)$, então quando calculamos $opt(i, j')$ podemos ignorar alguns valores. Para minimizar o tempo de execução, calcula primeiro em $opt(i, n/2)$, depois $opt(i, n/4)$, $opt(i, 3n/4)$ etc. A complexidade final será $O(mn \log n)$, pois cada valor k é considerado só em $\log n$ níveis.

Muitos desses problemas podem ser resolvidos com CHT.

```

36fdf8
// N testado, mas copieie do cp algo
vi dp_before, dp_cur;

ll C(int i, int j);

// compute dp_cur[l], ... dp_cur[r] (inclusive)
void compute(int l, int r, int optl, int optr) {
    if (l > r)
        return;

    int mid = (l + r) >> 1;
    pii best = {LLONG_MAX, -1};

    for (int k = optl; k <= min(mid, optr); k++) {
        smin(best, pii{(k ? dp_before[k-1] : 0) + C(k, mid), k});
    }

    dp_cur[mid] = best.ff;
    int opt = best.ss;

    compute(l, mid-1, optl, opt);
    compute(mid+1, r, opt, optr);
}

ll solve() {

```

```

dp_before.assign(n, 0);
dp_cur.assign(n, 0);

F(n) dp_before[i] = C(0, i);

F1(m-1) {
    compute(0, n-1, 0, n-1);
    dp_before = dp_cur;
}

return dp_before[n-1];
}

```

3.3.3 Knuth

Para recorrências de intervalo da seguinte forma:

$$dp(i, j) = C(i, j) + \min_{i \leq k < j} dp(i, k) + dp(k+1, j)$$

Usando a mesma ideia de opt , ele tem que satisfazer $opt(i, j-1) \leq opt(i, j) \leq opt(i+1, j)$. Isso é verdade, por exemplo, se C satisfaz a desigualdade quadrangular ($C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$) e $C(b, c) \leq C(a, d)$, para $a \leq b \leq c \leq d$.

Vamos calcular os estados de modo que $dp(i, j-1)$ e $dp(i+1, j)$ sejam calculados antes de $dp(i, j)$. Para isso, é só fazer o loop do i ser decrescente e o do j crescente. A complexidade fica $O(n^2)$.

```

1d0bab
// N testado, mas copieie do cp algo

ll dp[maxn][maxn];
int opt[maxn][maxn];

ll C(int i, int j);

ll solve() {
    F(n) {
        opt[i][i] = i;
        dp[i][i] = 0 // depende do problema
    }

    for(int i = n-2; i >= 0; i--) {

```



```
    for (int j = i+1; j < n; j++) {  
        ll mn = inf;  
        ll cost = C(i, j);  
        for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]); k++) {  
            if (mn >= dp[i][k] + dp[k+1][j] + cost) {  
                opt[i][j] = k;  
                mn = dp[i][k] + dp[k+1][j] + cost;  
            }  
        }  
        dp[i][j] = mn;  
    }  
}  
  
return dp[0][n-1];  
}
```

4 Geometria

4.1 Struct

09f261

```
// const ld EPS = 1e-6;

typedef long long T;
struct pt {
    T x, y;
    pt(T x = 0, T y = 0) : x(x), y(y) {}
    pt operator+(pt o) { return pt(x + o.x, y + o.y); }
    pt operator-(pt o) { return pt(x - o.x, y - o.y); }
    // pt operator*(T a) { return pt(a*x, a*y); }
    T operator*(pt o) { return x * o.x + y * o.y; }
    T operator^(pt o) { return x * o.y - y * o.x; }
    // pt operator^(pt o) { return pt(y*o.z - z*o.y, z*o.x -
    // x*o.z, x*o.y - y*o.x); }
    // bool operator==(pt p) { return x == p.x && y == p.y; }
    // bool operator==(pt p) { return abs(x - p.x) < EPS &&
    // abs(y - p.y) < EPS; }
    T norm2() { return x * x + y * y; }
    ld norm() { return sqrtl(norm2()); }
    bool operator<(pt o) const {
        if (x != o.x) return x < o.x;
        return y < o.y;
    }
};

// Testes insuficientes (...pra mim deu bom?)
// angulo de -pi a pi. Representa quanto a
// percorre no sentido anti-horario pra chegar em b
// nao sei o que ocorre se um deles for zero... apenas evite
// ld ang(pt a, pt b) { return atan2l(a ^ b, a * b); }

// Determinante (vetores nas linhas). Volume paralelepípedo.
// Usar versao 3d do cross product (^)
// T triple(pt a, pt b, pt c) { return a * (b^c); }
```

4.2 Coisas gerais

4.2.1 Intersecção de reta com reta

Suponha que tenhamos uma reta r_1 (no plano) parametrizada por t (i.e. $r_1 = a_1 + d_1 t$) e que seja não paralela a uma reta r_2 . Note que deve ser verdade que $(r - a_2) \times d_2 = 0$. Com isso, é possível determinar o valor de t .

$$t = \frac{(a_2 - a_1) \times d_2}{d_1 \times d_2}$$

então o ponto de intersecção pode ser calculado assim:

e6a31e

```
// provavelmente so faz sentido se T=long double
// qualquer coisa e so reescrever metendo uns casting
// mas a formula eh isso. testado no cses
pt intersect(pt a1, pt d1, pt a2, pt d2) { return a1 + d1 *
    (((a2 - a1) ^ d2) / (d1 ^ d2)); }
```

4.2.2 Checando se segmentos intersectam

b1c3b3

```
// testado no cses
// dei override no == de pontos pra o de long double
bool opposite_vecs(pt v1, pt v2) { return abs(abs(ang(v1, v2)
    )) - PI) < EPS; }

bool checa_ponto_segmento(pt ext1, pt ext2, pt testa) {
    if (ext1 == testa || ext2 == testa) return true;
    if (opposite_vecs(testa - ext1, testa - ext2)) return
        true;
    return false;
}

bool checa_intersec_segmento(pt p1, pt p2, pt p3, pt p4) {
    pt d1 = p1 - p2;
    pt d2 = p3 - p4;
    if (abs(d1 ^ d2) < EPS) {
        if (checa_ponto_segmento(p1, p2, p3) ||
            checa_ponto_segmento(p1, p2, p4) ||
```

```

        checa_ponto_segimento(p3, p4, p1) ||
        checa_ponto_segimento(p3, p4, p2)) {
            return true;
        }
        return false;
    }
    pt inter = intersect(p1, d1, p3, d2);
    if (checa_ponto_segimento(p1, p2, inter) &&
        checa_ponto_segimento(p3, p4, inter)) {
        return true;
    }
    return false;
}

```

4.2.3 Equação da bissetriz

Dados pontos dois a dois distintos X , Y e Z , pra pegar a equação da reta bissetriz do ângulo YXZ , uma opção é normalizar os vetores YX e ZX e pegar o ponto médio deles.

4.2.4 Distância de ponto a reta

Se $P = (x_0, y_0)$ e a equação da reta for $Ax + Bx + C = 0$, a distância do ponto até a reta é

$$\frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$$

Se $P = (x_0, y_0, z_0)$ e a equação do plano for $Ax + Bx + Cx + D = 0$, a distância do ponto até o plano é

$$\frac{|Ax_0 + By_0 + Cz_0 + D|}{\sqrt{A^2 + B^2 + C^2}}$$

4.2.5 Equação da reta a partir de dois pontos

Se $P = (x_0, y_0)$ e $Q = (x_1, y_1)$, então podemos setar $A = y_0 - y_1$, $B = x_1 - x_0$ e $C = -Ax_0 - By_0$.

4.2.6 Projeção de ponto em reta

Sejam P e $r : Ax + By + C = 0$ os objetos em questão. É fácil tomar um Q qualquer na reta (seta ou $x = 0$ ou $y = 0$ e isola). Queremos achar t real tal que

$(p + t \cdot (A, B) - q) \cdot (A, B) = 0$. Isolando, achamos

$$t = \frac{(q - p) \cdot (A, B)}{A^2 + B^2}$$

e daí o que queremos é $p + t \cdot (A, B)$.

4.2.7 Distância de ponto a segmento

Se o ponto for X e o segmento PQ , basta você ir vendo em que "seção" do plano está o X . Por exemplo, se $(X - P) \cdot (Q - P) \geq 0$ e $(X - Q) \cdot (P - Q) \geq 0$, quer dizer que meio que está no meio e a distância será a menor distância até a reta. Caso contrário, será a distância ou até P ou até Q . E pra distância até a reta dá pra usar que ela é $\sqrt{PX^2 - h^2}$ onde $h = (X - P) \cdot (Q - P) / \|Q - P\|$.

4.2.8 Intersecção reta com círculo

O círculo está na origem, tem raio r e a reta tem equação $ax + by + c = 0$. Ideias principais: vetor (a, b) é perpendicular à reta, então encontrar o ponto da reta mais próximo da origem (tem $d_0 = \frac{|c|}{\sqrt{a^2 + b^2}}$) escalonando (a, b) para essa distância. Olhando d_0 se descobre se tem 0, 1 ou 2 intersecções. Se tem 2, os pontos são equidistantes aos encontrados, com distância $d = \sqrt{r^2 - \frac{c^2}{a^2 + b^2}}$ (por Pitágoras). Esse método é mais preciso do que resolver algebricamente. EPS = precisão.

```

3887e3
int circ_line_inter(ld r, ld a, ld b, ld c, pt& p1, pt& p2)
{
    // Retorna a quantidade de pontos. Salva em p1 e p2. Se só
    // tem 1, p2 não muda

    ld x0 = -a*c/(a*a+b*b), y0 = -b*c/(a*a+b*b);
    ld dif = c*c-r*r*(a*a+b*b);
    if(dif > EPS) return 0;
    if(dif > -EPS) { // |dif| ~ 0
        p1.x = x0, p1.y = y0;
        return 1;
    }
    ld d = r*r - c*c/(a*a+b*b);
    ld mult = sqrt(d/(a*a+b*b));
    p1.x = x0 + b*mult;
    p2.x = x0 - b*mult;
}

```

```

p1.y = y0 - a*mult;
p2.y = y0 + a*mult;
return 2;
}

```

4.2.9 Intersecção de círculo com círculo

Tem 2 dois círculos, um centrado na origem com raio r_1 , e outro no ponto x_2, y_2 com raio r_2 . Subtraindo as duas equações dos círculos, você vai reduzir ao problema de achar a intersecção da reta $(2x_2)x + (2y_2)y + (r_2^2 - r_1^2 - x_2^2 - y_2^2) = 0$ com o primeiro círculo. Só tomar cuidado caso ambos círculos estejam centrados no mesmo ponto.

4.2.10 Tangentes comuns a dois círculos

Se os dois forem idênticos, infinitas.

Se tiver um dentro do outro, sem se tocarem, não tem.

Se tiver um dentro do outro, com o de dentro barely touching o de fora (um ponto só), tem uma só reta tangente aos dois.

Se eles se tocarem em dois pontos, tem duas.

Se eles se tocarem em um único ponto, externamente, tem três

Se eles não se tocarem e tiver um fora do outro, tem quatro.

```

e9a8ba
struct circle : pt {
    ld r;
};

struct line {
    ld a, b, c;
};

// const ld EPS = 1E-9;

ld sqr(ld a) { return a * a; }

void tangents(pt c, ld r1, ld r2, vector<line>& ans) {
    ld r = r2 - r1;
    ld z = sqr(c.x) + sqr(c.y);
    ld d = z - sqr(r);
    if (d < -EPS) return;
    d = sqrtl(abs(d));

```

```

    line l;
    l.a = (c.x * r + c.y * d) / z;
    l.b = (c.y * r - c.x * d) / z;
    l.c = r1;
    ans.push_back(l);
}

vector<line> tangents(circle a, circle b) {
    vector<line> ans;
    for (int i = -1; i <= 1; i += 2)
        for (int j = -1; j <= 1; j += 2)
            tangents(b - a, a.r * i, b.r * j, ans);
    for (size_t i = 0; i < ans.size(); ++i)
        ans[i].c -= ans[i].a * a.x + ans[i].b * a.y;
    return ans;
}

```

4.2.11 Miscelânea

Fórmula de Heron numericamente estável

```

077ba4
ld heron2(ld a, ld b, ld c) {
    // Sort a, b, c into descending order
    if (a < b) swap(a, b);
    if (a < c) swap(a, c);
    if (b < c) swap(b, c);
    ld p = (a + (b + c)) * (c - (a - b)) * (c + (a - b)) * (
        a + (b - c));
    return 0.25 * sqrtl(p);
}

```

Cálculo de centros de triângulos

```

c1b2b2
// fórmulas "funcionam" em dimensões maiores
// tipo, se você executar a fórmula, o cara obtido tá no
// plano do
// triângulo e, realmente, vai ser o esperado

pt bary(pt A, pt B, pt C, ld a, ld b, ld c) { return (A * a
    + B * b + C * c) / (a + b + c); }

```

```

pt centroid(pt A, pt B, pt C) {
    // geometric center of mass
    return bary(A, B, C, 1, 1, 1);
}

pt circumcenter(pt A, pt B, pt C) {
    // intersection of perpendicular bisectors
    ld a = norm(B - C), b = norm(C - A), c = norm(A - B);
    return bary(A, B, C, a * (b + c - a), b * (c + a - b), c
        * (a + b - c));
}

pt incenter(pt A, pt B, pt C) {
    // intersection of internal angle bisectors
    return bary(A, B, C, abs(B - C), abs(A - C), abs(A - B))
        ;
}

pt orthocenter(pt A, pt B, pt C) {
    // intersection of altitudes
    ld a = norm(B - C), b = norm(C - A), c = norm(A - B);
    return bary(A, B, C, (a + b - c) * (c + a - b), (b + c -
        a) * (a + b - c),
        (c + a - b) * (b + c - a));
}

pt excenter(pt A, pt B, pt C) {
    // intersection of two external angle bisectors
    ld a = abs(B - C), b = abs(A - C), c = abs(A - B);
    return bary(A, B, C, -a, b, c);

    //// NOTE: there are three excenters
    // return bary(A, B, C, a, -b, c);
    // return bary(A, B, C, a, b, -c);
}

```

4.3 Polígonos

4.3.1 Paralelograma e triângulo

```

8c23af
// no cp algorithms esse T era int
T signed_area_parallelogram(pt p1, pt p2, pt p3) { return (
    p2 - p1) ^ (p3 - p2); }

double triangle_area(pt p1, pt p2, pt p3) {
    return abs(signed_area_parallelogram(p1, p2, p3)) / 2.0;
}

bool clockwise(pt p1, pt p2, pt p3) { return
    signed_area_parallelogram(p1, p2, p3) < 0; }

bool counter_clockwise(pt p1, pt p2, pt p3) { return
    signed_area_parallelogram(p1, p2, p3) > 0; }

```

4.3.2 Área de polígono

```

3f348f
// Tb da pra pegar um ponto O arbitrario e ir somando as
// areas
// (com sinal) dos triangulos formados por O e cada par de
// pontos
// consecutivos do poligono
ld area(const vector<pt>& fig) {
    ld res = 0;
    for (unsigned i = 0; i < fig.size(); i++) {
        pt p = i ? fig[i - 1] : fig.back();
        pt q = fig[i];
        res += (p.x - q.x) * (p.y + q.y);
    }
    return abs(res) / 2;
}

```

4.3.3 Checa se ponto está dentro de polígono

```

849835
// preparacao em O(n).
// cada query em O(log n)

```

```

// abaixo, detalhes do que precisa ser feito
// coloca essas duas na struct
/*
long long cross(pt p) { return x * p.y - y * p.x; }
long long cross(pt a, pt b) { return (a - *this).cross(b - *
    this); }
bool operator<(pt o) {
    if (x != o.x) return x < o.x;
    return y < o.y;
}
*/
// testado
// tem que tar counterclockise
// tem que rodar a funcao prepare no seu proprio
// vetor de pontos (sem ser o seq)
bool lexComp(const pt& l, const pt& r) { return l.x < r.x ||
    (l.x == r.x && l.y < r.y); }

int sgn(long long val) { return val > 0 ? 1 : (val == 0 ? 0
    : -1); }

vector<pt> seq;
pt translation;
int n;
bool pointInTriangle(pt a, pt b, pt c, pt point) {
    long long s1 = abs(a.cross(b, c));
    long long s2 = abs(point.cross(a, b)) + abs(point.cross(
        b, c)) + abs(point.cross(c, a));
    return s1 == s2;
}

void prepare(vector<pt>& points) {
    n = points.size();
    int pos = 0;
    for (int i = 1; i < n; i++) {
        if (lexComp(points[i], points[pos])) pos = i;
    }
    rotate(points.begin(), points.begin() + pos, points.end
        ());
}

```

```

n--;
seq.resize(n);
for (int i = 0; i < n; i++)
    seq[i] = points[i + 1] - points[0];
translation = points[0];
}

bool pointInConvexPolygon(pt point) {
    point = point - translation;
    if (seq[0].cross(point) != 0 && sgn(seq[0].cross(point))
        != sgn(seq[0].cross(seq[n - 1])))
        return false;
    if (seq[n - 1].cross(point) != 0 &&
        sgn(seq[n - 1].cross(point)) != sgn(seq[n - 1].cross
            (seq[0])))
        return false;

    if (seq[0].cross(point) == 0) return seq[0].norm2() >=
        point.norm2();

    int l = 0, r = n - 1;
    while (r - l > 1) {
        int mid = (l + r) / 2;
        int pos = mid;
        if (seq[pos].cross(point) >= 0)
            l = mid;
        else
            r = mid;
    }
    int pos = l;
    return pointInTriangle(seq[pos], seq[pos + 1], pt(0, 0),
        point);
}

```

4.3.4 Soma de Minkowski

Sejam A e B as regiões delimitadas pelos polígonos convexos P e Q . Seja $C = A + B = \{a + b | a \in A, b \in B\}$. Esse algoritmo obtém o polígono R associado à região C .

5cd65e

```
// Basta pegar os dois poligonos em ordem counter clockise
```

```

// e ja pode rodar
// Roda em O(|P| + |Q|)

// coloca essas duas na struct
/*
long long cross(pt p) { return x * p.y - y * p.x; }
long long cross(pt a, pt b) { return (a - *this).cross(b -
    this); }
*/
// testado

void reorder_polygon(vector<pt>& P) {
    size_t pos = 0;
    for (size_t i = 1; i < P.size(); i++) {
        if (P[i].y < P[pos].y || (P[i].y == P[pos].y && P[i]
            .x < P[pos].x)) pos = i;
    }
    rotate(P.begin(), P.begin() + pos, P.end());
}

vector<pt> minkowski(vector<pt> P, vector<pt> Q) {
    // the first vertex must be the lowest
    reorder_polygon(P);
    reorder_polygon(Q);
    // we must ensure cyclic indexing
    P.push_back(P[0]);
    P.push_back(P[1]);
    Q.push_back(Q[0]);
    Q.push_back(Q[1]);
    // main part
    vector<pt> result;
    size_t i = 0, j = 0;
    while (i < P.size() - 2 || j < Q.size() - 2) {
        result.push_back(P[i] + Q[j]);
        auto cross = (P[i + 1] - P[i]).cross(Q[j + 1] - Q[j]
            );
        if (cross >= 0 && i < P.size() - 2) ++i;
        if (cross <= 0 && j < Q.size() - 2) ++j;
    }
    return result;
}

```

```

}

```

4.3.5 Teorema de Pick

Considere um polígono (sem self-intersections) de área S com vértices no latice. Então, se I é o número de pontos do látice estritamente dentro e B o número na borda. Então

$$S = I + \frac{B}{2} - 1$$

```

e5dbf2
/** Count integer (lattice) points inside and on boundary of
    polygon (can be concave)
    * BE CAREFUL: T must be INTEGER!!! */
template<typename T> pair<T, T> count_lattice_points(vector
    <Point<T>> const& v) {
    /** Pick's theorem: A = i + b/2 - 1 (area =
        inner_lattice + boundary_lattice/2 - 1) */
    pair<T, T> ans = {0, 0}; // <inside, boundary>
    T area2 = 0;
    int n = v.size();
    for (int i = 0, j = n - 1; i < n; j = i, i++) {
        Point<T> dif = v[i] - v[j];
        dif.x = abs(dif.x), dif.y = abs(dif.y);
        ans.se += gcd(dif.x, dif.y);
        area2 += v[i].cross(v[j]);
    }
    area2 = abs(area2);
    ans.fi = (area2 - ans.se + 2) / 2;
    return ans;
}

```

4.3.6 Número de lattice points dentro de non-lattice polygon

Processamos as arestas individualmente e somamos a contribuição de cada uma. Se estamos passando pela aresta de endpoints $A = (x_1, y_1)$ e $B = (x_2, y_2)$, podemos ver ela como uma função linear

$$y = kx + b, k = \frac{y_2 - y_1}{x_2 - x_1}, b = \frac{y_1 x_2 - x_1 y_2}{x_2 - x_1}$$

Segue, então, uma função que calcula o número de lattice points (x, y) satisfazendo $0 \leq x < n$ e $0 \leq y \leq \lfloor kx + b \rfloor$. Você precisa criar uma struct pra lidar com fração.

ad90e3

```
// Na pratica, se todos os numeradores e denominadores forem
// no maximo C, então com as chamadas recursivas vai ser no
// max
// C^2 se voce ficar dividindo pelo gcd (simplificando as
// fracoes)
// isso significa que se epsilon é a precisao dos k e b
// dados, então
// a precisao das doubles usadas deve ser epsilon^2

int count_lattices(Fraction k, Fraction b, long long n) {
    auto fk = k.floor();
    auto fb = b.floor();
    auto cnt = 0LL;
    if (k >= 1 || b >= 1) {
        cnt += (fk * (n - 1) + 2 * fb) * n / 2;
        k -= fk;
        b -= fb;
    }
    auto t = k * n + b;
    auto ft = t.floor();
    if (ft >= 1) {
        cnt += count_lattices(1 / k, (t - t.floor()) / k, t.
            floor());
    }
    return cnt;
}
```

4.4 Convex Hull

4.4.1 Graham Scan

7d158c

```
#define CW -1
#define CCW 1
```

```
int orient(pt a, pt b, pt c) {
    T v = (b-a)^(c-a);
    if(v < 0) return CW;
    else if(v > 0) return CCW;
    return 0;
}

vector<pt> convex_hull(vector<pt>& a, bool usecol=false) {
    // Saida em sentido horario. usecol true = bota colineares
    // no resultado
    // ALERTA: garanta que nao tem pontos repetidos no vetor
    // ALETA: a eh modificado (apenas ordenado)
    pt p0 = *min_element(all(a), [](pt a, pt b){
        return pt(a.y, a.x) < pt(b.y, b.x);
    });

    sort(all(a), [&p0](pt a, pt b) {
        int o = orient(p0, a, b);
        if(o == 0) return (a - p0).norm2() < (b - p0).norm2();
        return o == CW;
    });

    if(usecol) {
        int i = sza(a) - 2;
        while(i >= 0 && orient(p0, a[i], a.back()) == 0) i--;
        reverse(a.begin()+i+1, a.end());
    }

    vector<pt> st;
    for(int i = 0; i < sza(a); i++) {
        if(usecol)
            while(sza(st) > 1 && orient(st[sza(st)-2], st.back(),
                a[i]) == CCW)
                st.pp();
        else
            while(sza(st) > 1 && orient(st[sza(st)-2], st.back(),
                a[i]) != CW)
                st.pp();
        st.pb(a[i]);
    }
}
```



```

    return st;
}

```

4.4.2 Monotone Chain

3ba8b1

```

#define CW -1
#define CCW 1

int orient(pt a, pt b, pt c) {
    T v = (b-a)^(c-a);
    if(v < 0) return CW;
    else if(v > 0) return CCW;
    return 0;
}

bool isor(int ot, pt a, pt b, pt c, bool usecol) {
    int o = orient(a, b, c);
    return o == ot || (usecol && o == 0);
}

vector<pt> convex_hull(vector<pt>& a, bool usecol=false) {
    // Saida em sentido horario. usecol true = bota colineares
    // no resultado
    // ALERTA: garanta que nao tem pontos repetidos no vetor
    // ALERTA: a eh modificado (ordenado)
    if(sza(a) == 1) return a;

    sort(all(a));
    pt p1 = a[0], p2 = a.back();
    vector<pt> up = {p1}, down = {p2};

    for(int i = 1; i < sza(a); i++) {
        int o = CW;

        vector<pt>& t = up;
        if(i == sza(a) - 1 || isor(o, p1, a[i], p2, usecol)) {
            while(sza(t) > 1 && !isor(o, t[sza(t)-2], t.back(), a[
                i], usecol))
                t.ppb();

```

```

        t.pb(a[i]);
    }

    { // NAO TIRAR ESSA CHAVE
        o = CCW;
        vector<pt>& t = down;
        if(i == sza(a) - 1 || isor(o, p1, a[i], p2, usecol)) {
            while(sza(t) > 1 && !isor(o, t[sza(t)-2], t.back(),
                a[i], usecol))
                t.ppb();
            t.pb(a[i]);
        }
    }
}

if(usecol && sza(up) == sza(a)) {
    reverse(all(a));
    return a;
}

for(int i = sza(down) - 2; i > 0; i--) up.pb(down[i]);
return up;
}

```

4.4.3 Dinâmico

16a320

```

// "segment tree"-style implementation of online decremental
// dynamic convex hull
// Based on paper "Maintenance of configurations in the
// plane" by Overmars and van Leeuwen, with
// some modifications. This implementation only supports
// efficient (O(log^2 n)) deletion of points,
// which is enough to compute nested convex hulls in O(n log
// ^2 n). It should be possible to modify
// this to support insertions as well.

// Assumes all points are distinct
// Assumes coordinates are at most 10^6
// Can handle 200000 points in a few seconds.

```

```

#include <algorithm>
#include <cassert>
#include <cstdio>
#include <map>
#include <set>
#include <stdint.h>
#include <vector>

struct Point {
    int64_t x, y;
    Point operator-(Point p) const { return {x - p.x, y - p.y}; }
    int64_t cross(Point p) const { return x * p.y - y * p.x; }
    int64_t dot(Point p) const { return x * p.x + y * p.y; }
    bool operator<(Point p) const {
        if (y != p.y) return y < p.y;
        return x < p.x;
    }
    bool operator==(Point p) const { return x == p.x && y == p.y; }
    Point operator-() const { return {-x, -y}; }
};

int64_t cross(Point a, Point b, Point c) { return (b - a).cross(c - a); }

class LeftHull {
    std::vector<Point> ps;
    struct Node {
        int bl, br;
        int L, R;
        int lchd, rchd;
    };
    std::vector<Node> nodes;
    int root;
    bool isleaf(int w) { return nodes[w].lchd == -1 && nodes[w].rchd == -1; }
    void pull(int w) {
        assert(!isleaf(w));

```

```

        int l = nodes[w].lchd, r = nodes[w].rchd;
        int64_t split_y = ps[nodes[r].L].y;
        while (!isleaf(l) || !isleaf(r)) {
            int a = nodes[l].bl, b = nodes[l].br, c = nodes[r].bl, d = nodes[r].br;
            if (a != b && cross(ps[a], ps[b], ps[c]) > 0) {
                l = nodes[l].lchd;
            } else if (c != d && cross(ps[b], ps[c], ps[d]) > 0) {
                r = nodes[r].rchd;
            } else if (a == b) {
                r = nodes[r].lchd;
            } else if (c == d) {
                l = nodes[l].rchd;
            } else {
                int64_t s1 = cross(ps[a], ps[b], ps[c]);
                int64_t s2 = cross(ps[b], ps[a], ps[d]);
                assert(s1 + s2 >= 0);
                if (s1 + s2 == 0 || s1 * ps[d].y + s2 * ps[c].y < split_y * (s1 + s2)) {
                    l = nodes[l].rchd;
                } else {
                    r = nodes[r].lchd;
                }
            }
        }
        nodes[w].bl = nodes[l].L;
        nodes[w].br = nodes[r].L;
    }

    void build(int w, int L, int R) {
        nodes[w].L = L;
        nodes[w].R = R;
        if (R - L == 1) {
            nodes[w].lchd = nodes[w].rchd = -1;
            nodes[w].bl = nodes[w].br = L;
        } else {
            int M = (L + R) / 2;
            nodes[w].lchd = w + 1;
            nodes[w].rchd = w + 2 * (M - L);
            build(nodes[w].lchd, L, M);

```

```

        build(nodes[w].rchd, M, R);
        pull(w);
    }
}

int erase(int w, int L, int R) {
    if (R <= nodes[w].L || L >= nodes[w].R) return w;
    if (L <= nodes[w].L && R >= nodes[w].R) return -1;
    nodes[w].lchd = erase(nodes[w].lchd, L, R);
    nodes[w].rchd = erase(nodes[w].rchd, L, R);
    if (nodes[w].lchd == -1) return nodes[w].rchd;
    if (nodes[w].rchd == -1) return nodes[w].lchd;
    pull(w);
    return w;
}

// only works for whole hull
void get_hull(int w, int l, int r, std::vector<int>& res) {
    if (isleaf(w)) {
        res.push_back(nodes[w].L);
    } else if (r <= nodes[w].bl) {
        get_hull(nodes[w].lchd, l, r, res);
    } else if (l >= nodes[w].br) {
        get_hull(nodes[w].rchd, l, r, res);
    } else {
        assert(l <= nodes[w].bl && nodes[w].br <= r);
        get_hull(nodes[w].lchd, l, nodes[w].bl, res);
        get_hull(nodes[w].rchd, nodes[w].br, r, res);
    }
}

public:
    LeftHull(const std::vector<Point>& ps) : ps(ps), nodes(
        ps.size() * 2), root(0) {
        build(0, 0, ps.size());
    }

    std::vector<int> get_hull() {
        if (root == -1) return {};
        std::vector<int> res;
        get_hull(root, 0, ps.size() - 1, res);
        return res;
}

```

```

    }
    void erase(int L) { root = erase(root, L, L + 1); }
};

std::vector<Point> ps;
std::map<Point, int> id;
int layer[1000005];
int ans[1000005];

int main() {
    int N;
    scanf("%d", &N);
    for (int i = 0; i < N; i++) {
        int X, Y;
        scanf("%d %d", &X, &Y);
        ps.push_back({X, Y});
        id[{X, Y}] = i;
    }

    std::sort(ps.begin(), ps.end());
    struct LeftHull left(ps);
    std::reverse(ps.begin(), ps.end());
    for (auto& p : ps) {
        p = -p;
    }
    struct LeftHull right(ps);
    for (auto& p : ps) {
        p = -p;
    }
    std::reverse(ps.begin(), ps.end());
    for (int l = 1, cnt = 0; cnt < N; l++) {
        std::set<int> hull;
        for (int i : left.get_hull()) {
            hull.insert(i);
        }
        for (int i : right.get_hull()) {
            hull.insert(N - 1 - i);
        }
        for (int i : hull) {
            assert(!layer[i]);
}

```

```

        cnt++;
        layer[i] = 1;
        left.erase(i);
        right.erase(N - 1 - i);
    }
}
for (int i = 0; i < N; i++) {
    ans[id[ps[i]]] = layer[i];
}
for (int i = 0; i < N; i++) {
    printf("%d\n", ans[i]);
}
}

```

4.5 Line Sweep

4.5.1 Detecta se par de segmentos se intersecta

ec84c3

```

// testado
// https://acm.timus.ru/problem.aspx?space=1&num=1469

#include <bits/stdc++.h>

using namespace std;

const double EPS = 1E-9;

struct pt {
    double x, y;
};

struct seg {
    pt p, q;
    int id;

    seg(pt p, pt q, int id) : p(p), q(q), id(id) {}

    double get_y(double x) const {
        if (abs(p.x - q.x) < EPS) return p.y;
        return p.y + (q.y - p.y) * (x - p.x) / (q.x - p.x);
    }
};

```

```

    }
};

bool intersect1d(double l1, double r1, double l2, double r2)
{
    if (l1 > r1) swap(l1, r1);
    if (l2 > r2) swap(l2, r2);
    return max(l1, l2) <= min(r1, r2) + EPS;
}

int vec(const pt& a, const pt& b, const pt& c) {
    double s = (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
    return abs(s) < EPS ? 0 : s > 0 ? +1 : -1;
}

bool intersect(const seg& a, const seg& b) {
    return intersect1d(a.p.x, a.q.x, b.p.x, b.q.x) &&
        intersect1d(a.p.y, a.q.y, b.p.y, b.q.y) &&
        vec(a.p, a.q, b.p) * vec(a.p, a.q, b.q) <= 0 &&
        vec(b.p, b.q, a.p) * vec(b.p, b.q, a.q) <= 0;
}

bool operator<(const seg& a, const seg& b) {
    double x = max(min(a.p.x, a.q.x), min(b.p.x, b.q.x));
    return a.get_y(x) < b.get_y(x) - EPS;
}

struct event {
    double x;
    int tp, id;

    event() {}
    event(double x, int tp, int id) : x(x), tp(tp), id(id) {}

    bool operator<(const event& e) const {
        if (abs(x - e.x) > EPS) return x < e.x;
        return tp > e.tp;
    }
};

```

```

};

set<seg> s;
vector<set<seg>::iterator> where;

set<seg>::iterator prev(set<seg>::iterator it) { return it
    == s.begin() ? s.end() : --it; }

set<seg>::iterator next(set<seg>::iterator it) { return ++it
    ; }

pair<int, int> solve(const vector<seg>& a) {
    int n = (int)a.size();
    vector<event> e;
    for (int i = 0; i < n; ++i) {
        e.push_back(event(min(a[i].p.x, a[i].q.x), +1, i));
        e.push_back(event(max(a[i].p.x, a[i].q.x), -1, i));
    }
    sort(e.begin(), e.end());

    s.clear();
    where.resize(a.size());
    for (size_t i = 0; i < e.size(); ++i) {
        int id = e[i].id;
        if (e[i].tp == +1) {
            set<seg>::iterator nxt = s.lower_bound(a[id]),
                prv = prev(nxt);
            if (nxt != s.end() && intersect(*nxt, a[id]))
                return make_pair(nxt->id, id);
            if (prv != s.end() && intersect(*prv, a[id]))
                return make_pair(prv->id, id);
            where[id] = s.insert(nxt, a[id]);
        } else {
            set<seg>::iterator nxt = next(where[id]), prv =
                prev(where[id]);
            if (nxt != s.end() && prv != s.end() &&
                intersect(*nxt, *prv))
                return make_pair(prv->id, nxt->id);
            s.erase(where[id]);
        }
    }
}

```

```

}

return make_pair(-1, -1);
}

int main() {
    int n;
    cin >> n;
    vector<seg> edges;
    pt a, b;
    for (int i = 0; i < n; i++) {
        cin >> a.x >> a.y >> b.x >> b.y;
        edges.push_back(seg(a, b, i));
    }
    auto p = solve(edges);
    if (p.first == -1) {
        cout << "NO\n";
    } else {
        cout << "YES\n";
        cout << p.first + 1 << " " << p.second + 1 << "\n";
    }
}

```

4.6 Grafos planares

Um grafo planar com k componentes conexas satisfaz

$$V - A + F = 1 + k$$

Se $V \geq 3$, o número máximo de arestas de um grafo planar é $3V - 6$. Dá pra conseguir isso em qualquer grafo planar em que as faces são limitadas por triângulos. Ou seja, $A = O(V)$ em grafos planares. O número máximo de faces é $2V - 4$. Todo grafo planar tem um vértice de grau 5 ou menos.

4.6.1 Encontrar faces

$O(n \log n)$

Retorna um vetor de vértices para cada face. Primeiro é a face externa.

Faces internas são retornadas counterclockwise e a externa é clockwise.

a5d30c

```
struct Point {
```

```

int64_t x, y;

Point(int64_t x_, int64_t y_) : x(x_), y(y_) {}

Point operator-(const Point& p) const { return Point(x -
    p.x, y - p.y); }

int64_t cross(const Point& p) const { return x * p.y - y
    * p.x; }

int64_t cross(const Point& p, const Point& q) const {
    return (p - *this).cross(q - *this); }

int half() const { return int(y < 0 || (y == 0 && x < 0)
    ); }
};

std::vector<std::vector<size_t>> find_faces(std::vector<
    Point> vertices,

                                std::vector<std
                                ::vector<
                                size_t>> adj
                                ) {

    size_t n = vertices.size();
    std::vector<std::vector<char>> used(n);
    for (size_t i = 0; i < n; i++) {
        used[i].resize(adj[i].size());
        used[i].assign(adj[i].size(), 0);
        auto compare = [&](size_t l, size_t r) {
            Point pl = vertices[l] - vertices[i];
            Point pr = vertices[r] - vertices[i];
            if (pl.half() != pr.half()) return pl.half() <
                pr.half();
            return pl.cross(pr) > 0;
        };
        std::sort(adj[i].begin(), adj[i].end(), compare);
    }
    std::vector<std::vector<size_t>> faces;
    for (size_t i = 0; i < n; i++) {
        for (size_t edge_id = 0; edge_id < adj[i].size();

```

```

        edge_id++) {
            if (used[i][edge_id]) {
                continue;
            }
            std::vector<size_t> face;
            size_t v = i;
            size_t e = edge_id;
            while (!used[v][e]) {
                used[v][e] = true;
                face.push_back(v);
                size_t u = adj[v][e];
                size_t e1 = std::lower_bound(adj[u].begin(),
                    adj[u].end(), v,
                    [&](size_t l,
                        size_t r) {
                            Point pl =
                                vertices
                                [l] -
                                vertices
                                [u];
                            Point pr =
                                vertices
                                [r] -
                                vertices
                                [u];
                            if (pl.half
                                () !=
                                pr.half
                                ())
                                return
                                    pl.
                                    half
                                    ()
                                    <
                                    pr.
                                    half
                                    ();
                            return pl.
                                cross(
                                    pr) >

```

```

                                0;
                                }) -
                                adj[u].begin() + 1;
    if (e1 == adj[u].size()) {
        e1 = 0;
    }
    v = u;
    e = e1;
}
std::reverse(face.begin(), face.end());
int sign = 0;
for (size_t j = 0; j < face.size(); j++) {
    size_t j1 = (j + 1) % face.size();
    size_t j2 = (j + 2) % face.size();
    int64_t val = vertices[face[j]].cross(
        vertices[face[j1]], vertices[face[j2]]);
    if (val > 0) {
        sign = 1;
        break;
    } else if (val < 0) {
        sign = -1;
        break;
    }
}
if (sign <= 0) {
    faces.insert(faces.begin(), face);
} else {
    faces.emplace_back(face);
}
}
return faces;
}

```

4.6.2 Construir grafo planar a partir de segmentos

$O(n^2 \log n)$

aa3568

```
using dbl = long double;
```

```

const dbl eps = 1e-9;

struct Point {
    dbl x, y;

    Point() {}
    Point(dbl x_, dbl y_) : x(x_), y(y_) {}

    Point operator*(dbl d) const { return Point(x * d, y * d); }

    Point operator+(const Point& p) const { return Point(x + p.x, y + p.y); }

    Point operator-(const Point& p) const { return Point(x - p.x, y - p.y); }

    dbl cross(const Point& p) const { return x * p.y - y * p.x; }

    dbl cross(const Point& p, const Point& q) const { return (p - *this).cross(q - *this); }

    dbl dot(const Point& p) const { return x * p.x + y * p.y; }

    dbl dot(const Point& p, const Point& q) const { return (p - *this).dot(q - *this); }

    bool operator<(const Point& p) const {
        if (fabs(x - p.x) < eps) {
            if (fabs(y - p.y) < eps) {
                return false;
            } else {
                return y < p.y;
            }
        } else {
            return x < p.x;
        }
    }
}

```

```

    bool operator==(const Point& p) const { return fabs(x -
        p.x) < eps && fabs(y - p.y) < eps; }

    bool operator>=(const Point& p) const { return !(*this <
        p); }
};

struct Line {
    Point p[2];

    Line(Point l, Point r) {
        p[0] = l;
        p[1] = r;
    }
    Point& operator[](const int& i) { return p[i]; }
    const Point& operator[](const int& i) const { return p[i]
    }; }
    Line(const Line& l) {
        p[0] = l.p[0];
        p[1] = l.p[1];
    }
    Point getOrth() const { return Point(p[1].y - p[0].y, p
        [0].x - p[1].x); }
    bool hasPointLine(const Point& t) const { return std::
        fabs(p[0].cross(p[1], t)) < eps; }
    bool hasPointSeg(const Point& t) const { return
        hasPointLine(t) && t.dot(p[0], p[1]) < eps; }
};

std::vector<Point> interLineLine(Line l1, Line l2) {
    if (std::fabs(l1.getOrth().cross(l2.getOrth())) < eps) {
        if (l1.hasPointLine(l2[0]))
            return {l1[0], l1[1]};
        else
            return {};
    }
    Point u = l2[1] - l2[0];
    Point v = l1[1] - l1[0];
    dbl s = u.cross(l2[0] - l1[0]) / u.cross(v);

```

```

    return {Point(l1[0] + v * s)};
}

std::vector<Point> interSegSeg(Line l1, Line l2) {
    if (l1[0] == l1[1]) {
        if (l2[0] == l2[1]) {
            if (l1[0] == l2[0])
                return {l1[0]};
            else
                return {};
        } else {
            if (l2.hasPointSeg(l1[0]))
                return {l1[0]};
            else
                return {};
        }
    }
    if (l2[0] == l2[1]) {
        if (l1.hasPointSeg(l2[0]))
            return {l2[0]};
        else
            return {};
    }
    auto li = interLineLine(l1, l2);
    if (li.empty()) return li;
    if (li.size() == 2) {
        if (l1[0] >= l1[1]) std::swap(l1[0], l1[1]);
        if (l2[0] >= l2[1]) std::swap(l2[0], l2[1]);
        std::vector<Point> res(2);
        if (l1[0] < l2[0])
            res[0] = l2[0];
        else
            res[0] = l1[0];
        if (l1[1] < l2[1])
            res[1] = l1[1];
        else
            res[1] = l2[1];
        if (res[0] == res[1]) res.pop_back();
        if (res.size() == 2u && res[1] < res[0])
            return {};
    }
}

```



```

        else
            return res;
    }
    Point cand = li[0];
    if (l1.hasPointSeg(cand) && l2.hasPointSeg(cand))
        return {cand};
    else
        return {};
}

std::pair<std::vector<Point>, std::vector<std::vector<size_t>>>
>>>
build_graph(std::vector<Line> segments) {
    std::vector<Point> p;
    std::vector<std::vector<size_t>> adj;
    std::map<std::pair<int64_t, int64_t>, size_t> point_id;
    auto get_point_id = [&](Point pt) {
        auto repr = std::make_pair(int64_t(std::round(pt.x *
            1000000000) + 1e-6),
                                   int64_t(std::round(pt.y *
            1000000000) + 1e-6))
        ;
        if (!point_id.count(repr)) {
            adj.emplace_back();
            size_t id = point_id.size();
            point_id[repr] = id;
            p.push_back(pt);
            return id;
        } else {
            return point_id[repr];
        }
    };
    for (size_t i = 0; i < segments.size(); i++) {
        std::vector<size_t> curr = {get_point_id(segments[i]
            [0]), get_point_id(segments[i][1])};
        for (size_t j = 0; j < segments.size(); j++) {
            if (i == j) continue;
            auto inter = interSegSeg(segments[i], segments[j]
                );
            for (auto pt : inter) {

```

```

                curr.push_back(get_point_id(pt));
            }
        }
        std::sort(curr.begin(), curr.end(), [&](size_t l,
            size_t r) { return p[l] < p[r]; });
        curr.erase(std::unique(curr.begin(), curr.end()),
            curr.end());
        for (size_t j = 0; j + 1 < curr.size(); j++) {
            adj[curr[j]].push_back(curr[j + 1]);
            adj[curr[j + 1]].push_back(curr[j]);
        }
    }
    for (size_t i = 0; i < adj.size(); i++) {
        std::sort(adj[i].begin(), adj[i].end());
        // removing edges that were added multiple times
        adj[i].erase(std::unique(adj[i].begin(), adj[i].end()
            ()), adj[i].end());
    }
    return {p, adj};
}

```

4.6.3 Point location

Dada uma subdivisão do plano, determinar que face da subdivisão o ponto pertence em $O(\log n)$.

Implementado pra int. Mas dá pra mudar pra double mudando as funções de comparação. Assume que a subdivisão está armazenada corretamente numa DCEL e a face externa está numerada com -1 .

Para cada query, é retornado um par $(1, i)$ se o ponto está na face de número i e um par $(0, i)$ é retornado caso esteja em cima de uma edge de número i .

```

285e8c
// https://acm.timus.ru/problem.aspx?space=1&num=1848&locale=en
// deu AC
// muito cuidado com memoria
// usar -1 pra face externa
// resposta eh (1, i) se ta estritamente na face 1
// resposta eh (0, i) se ta na aresta i
// Nesse problema em particular era pra dizer quantos
// pontos tavam em cada poligono (disjuntos).

```

```

#include <bits/stdc++.h>

using namespace std;

typedef long long ll;

bool ge(const ll& a, const ll& b) { return a >= b; }
bool le(const ll& a, const ll& b) { return a <= b; }
bool eq(const ll& a, const ll& b) { return a == b; }
bool gt(const ll& a, const ll& b) { return a > b; }
bool lt(const ll& a, const ll& b) { return a < b; }
int sgn(const ll& x) { return le(x, 0) ? eq(x, 0) ? 0 : -1 : 1; }

struct pt {
    ll x, y;
    pt() {}
    pt(ll _x, ll _y) : x(_x), y(_y) {}
    pt operator-(const pt& a) const { return pt(x - a.x, y - a.y); }
    ll dot(const pt& a) const { return x * a.x + y * a.y; }
    ll dot(const pt& a, const pt& b) const { return (a - *this).dot(b - *this); }
    ll cross(const pt& a) const { return x * a.y - y * a.x; }
    ll cross(const pt& a, const pt& b) const { return (a - *this).cross(b - *this); }
    bool operator==(const pt& a) const { return a.x == x && a.y == y; }
};

struct Edge {
    pt l, r;
};

bool edge_cmp(Edge* edge1, Edge* edge2) {
    const pt a = edge1->l, b = edge1->r;
    const pt c = edge2->l, d = edge2->r;
    int val = sgn(a.cross(b, c)) + sgn(a.cross(b, d));

```

```

    if (val != 0) return val > 0;
    val = sgn(c.cross(d, a)) + sgn(c.cross(d, b));
    return val < 0;
}

enum EventType { DEL = 2, ADD = 3, GET = 1, VERT = 0 };

struct Event {
    EventType type;
    int pos;
    bool operator<(const Event& event) const { return type < event.type; }
};

vector<Edge*> sweepline(vector<Edge*> planar, vector<pt> queries) {
    using pt_type = decltype(pt::x);

    // collect all x-coordinates
    auto s = set<pt_type, std::function<bool(const pt_type&, const pt_type&>>>(lt);
    for (pt p : queries)
        s.insert(p.x);
    for (Edge* e : planar) {
        s.insert(e->l.x);
        s.insert(e->r.x);
    }

    // map all x-coordinates to ids
    int cid = 0;
    auto id = map<pt_type, int, std::function<bool(const pt_type&, const pt_type&>>>(lt);
    for (auto x : s)
        id[x] = cid++;

    // create events
    auto t = set<Edge*, decltype(*edge_cmp)>(edge_cmp);
    auto vert_cmp = [] (const pair<pt_type, int>& l, const pair<pt_type, int>& r) {
        if (!eq(l.first, r.first)) return lt(l.first, r.

```

```

        first);
        return l.second < r.second;
    };
    auto vert = set<pair<pt_type, int>, decltype(vert_cmp)>(
        vert_cmp);
    vector<vector<Event>> events(cid);
    for (int i = 0; i < (int)queries.size(); i++) {
        int x = id[queries[i].x];
        events[x].push_back(Event{GET, i});
    }
    for (int i = 0; i < (int)planar.size(); i++) {
        int lx = id[planar[i]->l.x], rx = id[planar[i]->r.x];
        if (lx > rx) {
            swap(lx, rx);
            swap(planar[i]->l, planar[i]->r);
        }
        if (lx == rx) {
            events[lx].push_back(Event{VERT, i});
        } else {
            events[lx].push_back(Event{ADD, i});
            events[rx].push_back(Event{DEL, i});
        }
    }

    // perform sweep line algorithm
    vector<Edge*> ans(queries.size(), nullptr);
    for (int x = 0; x < cid; x++) {
        sort(events[x].begin(), events[x].end());
        vert.clear();
        for (Event event : events[x]) {
            if (event.type == DEL) {
                t.erase(planar[event.pos]);
            }
            if (event.type == VERT) {
                vert.insert(
                    make_pair(min(planar[event.pos]->l.y,
                                planar[event.pos]->r.y), event.pos));
            }
        }
    }

```

```

        if (event.type == ADD) {
            t.insert(planar[event.pos]);
        }
        if (event.type == GET) {
            auto jt = vert.upper_bound(make_pair(queries
                [event.pos].y, planar.size()));
            if (jt != vert.begin()) {
                --jt;
                int i = jt->second;
                if (ge(max(planar[i]->l.y, planar[i]->r.
                    y), queries[event.pos].y)) {
                    ans[event.pos] = planar[i];
                    continue;
                }
            }
            Edge* e = new Edge;
            e->l = e->r = queries[event.pos];
            auto it = t.upper_bound(e);
            if (it != t.begin()) ans[event.pos] = *(--it);
            delete e;
        }
    }

    for (Event event : events[x]) {
        if (event.type != GET) continue;
        if (ans[event.pos] != nullptr && eq(ans[event.
            pos]->l.x, ans[event.pos]->r.x)) continue;

        Edge* e = new Edge;
        e->l = e->r = queries[event.pos];
        auto it = t.upper_bound(e);
        delete e;
        if (it == t.begin())
            e = nullptr;
        else
            e = *(--it);
        if (ans[event.pos] == nullptr) {
            ans[event.pos] = e;
            continue;
        }
    }

```

```

    }
    if (e == nullptr) continue;
    if (e == ans[event.pos]) continue;
    if (id[ans[event.pos]->r.x] == x) {
        if (id[e->l.x] == x) {
            if (gt(e->l.y, ans[event.pos]->r.y)) ans
                [event.pos] = e;
        }
    } else {
        ans[event.pos] = e;
    }
}
return ans;
}

struct DCEL {
    struct Edge {
        pt origin;
        Edge* nxt = nullptr;
        Edge* twin = nullptr;
        int face;
        Edge(const pt& p, int f) : origin(p), face(f) {}
    };
    vector<Edge*> body;
    void add_edge(pt origin, int face) {
        Edge* e = new Edge(origin, face);
        body.push_back(e);
    }
};

vector<pair<int, int>> point_location(DCEL& planar, vector<
    pt> queries) {
    vector<pair<int, int>> ans(queries.size());
    vector<Edge*> planar2;
    map<intptr_t, int> pos;
    map<intptr_t, int> added_on;
    int n = planar.body.size();
    for (int i = 0; i < n; i++) {
        if (planar.body[i]->face > planar.body[i]->twin->

```

```

        face) continue;
        Edge* e = new Edge;
        e->l = planar.body[i]->origin;
        e->r = planar.body[i]->twin->origin;
        added_on[(intptr_t)e] = i;
        pos[(intptr_t)e] = lt(planar.body[i]->origin.x,
            planar.body[i]->twin->origin.x)
            ? planar.body[i]->face
            : planar.body[i]->twin->face;

        if (i & 1) {
            delete planar.body[i];
            delete planar.body[i]->twin;
        }
        // cerr << i << "aa\n";
        planar2.push_back(e);
    }
    // delete &planar;
    auto res = sweepline(planar2, queries);
    for (int i = 0; i < (int)queries.size(); i++) {
        if (res[i] == nullptr) {
            ans[i] = make_pair(1, -1);
            continue;
        }
        pt p = queries[i];
        pt l = res[i]->l, r = res[i]->r;
        if (eq(p.cross(l, r), 0) && le(p.dot(l, r), 0)) {
            ans[i] = make_pair(0, added_on[(intptr_t)res[i]
                ]]);
            continue;
        }
        ans[i] = make_pair(1, pos[(intptr_t)res[i]]);
    }
    for (auto e : planar2)
        delete e;
    return ans;
}

int main() {
    int n;
    vector<pt> queries;

```

```

cin >> n;
for (int i = 0; i < n; i++) {
    pt p;
    cin >> p.x >> p.y;
    queries.push_back(p);
}
int num_pols;
cin >> num_pols;
DCEL planar;
vector<int> ans_pol, num_face;
ans_pol.push_back(0);
for (int i = 0; i < num_pols; i++) {
    int verts_pol;
    ans_pol.push_back(0);
    cin >> verts_pol;
    vector<pt> lista_pts;
    for (int j = 0; j < verts_pol; j++) {
        pt atual;
        cin >> atual.x >> atual.y;
        lista_pts.push_back(atual);
    }
    int ind_first_edge = planar.body.size();
    for (int j = 0; j < verts_pol; j++) {
        int prox = (j + 1) % verts_pol;
        planar.add_edge(lista_pts[j], i + 1);
        planar.add_edge(lista_pts[prox], -1);
        num_face.push_back(i + 1);
        num_face.push_back(i + 1);
        planar.body[planar.body.size() - 2]->twin =
            planar.body[planar.body.size() - 1];
        planar.body[planar.body.size() - 1]->twin =
            planar.body[planar.body.size() - 2];
    }
    for (int j = ind_first_edge; j < planar.body.size();
        j++) {
        int nxt_id =
            j + 2 < planar.body.size() ? j + 2 : (
                ind_first_edge + j + 2 - planar.body.
                    size());
        if ((j - ind_first_edge) % 2 == 0) {

```

```

            planar.body[j]->nxt = planar.body[nxt_id];
        } else {
            planar.body[nxt_id]->nxt = planar.body[j];
        }
    }

    for (auto ans : point_location(planar, queries)) {
        // cout << ans.first << " " << ans.second << "\n";
        if (ans.first) {
            if (ans.second != -1) ans_pol[ans.second]++;
        } else {
            ans_pol[num_face[ans.second]]++;
        }
    }

    for (int i = 1; i < ans_pol.size(); i++) {
        cout << ans_pol[i] << "\n";
    }
}

```

4.7 Miscelânea

4.7.1 Par mais próximo de pontos

Dados n pontos, resolve em $O(n \log n)$.

404cdf

```

struct pt {
    int x, y, id;
};

struct cmp_x {
    bool operator()(const pt& a, const pt& b) const {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    }
};

struct cmp_y {
    bool operator()(const pt& a, const pt& b) const { return
        a.y < b.y; }
}

```

```

};

int n;
vector<pt> a;

double mindist;
pair<int, int> best_pair;

void upd_ans(const pt& a, const pt& b) {
    double dist = sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
    if (dist < mindist) {
        mindist = dist;
        best_pair = {a.id, b.id};
    }
}

vector<pt> t;

void rec(int l, int r) {
    if (r - l <= 3) {
        for (int i = l; i < r; ++i) {
            for (int j = i + 1; j < r; ++j) {
                upd_ans(a[i], a[j]);
            }
        }
        sort(a.begin() + l, a.begin() + r, cmp_y());
        return;
    }

    int m = (l + r) >> 1;
    int midx = a[m].x;
    rec(l, m);
    rec(m, r);

    merge(a.begin() + l, a.begin() + m, a.begin() + m, a.begin() + r, t.begin(), cmp_y());
    copy(t.begin(), t.begin() + r - l, a.begin() + l);

    int tsz = 0;

```

```

        for (int i = l; i < r; ++i) {
            if (abs(a[i].x - midx) < mindist) {
                for (int j = tsz - 1; j >= 0 && a[i].y - t[j].y < mindist; --j)
                    upd_ans(a[i], t[j]);
                t[tsz++] = a[i];
            }
        }
    }

    // na main
    t.resize(n);
    sort(a.begin(), a.end(), cmp_x());
    mindist = 1E20;
    rec(0, n);

```

Dá pra generalizar isso pra encontrar o triângulo de menor perímetro: divide o plano em duas metades, acha o menor perímetro de cada metade aí constrói uma fita de largura $\text{minper}/2$ e itera pelos triângulos que podem melhorar a resposta.

4.7.2 Triangulação de Delaunay e Diagrama de Voronoi

Seja $S = \{p_i\}$ um conjunto de pontos no plano. Um diagrama de Voronoi $V(S)$ de S é uma partição do plano em n regiões V_i dadas por $V_i = \{p \in \mathbb{R}^2 : \rho(p, p_i) = \min \rho(p, p_k)\}$. As células do diagrama de Voronoi são polígonos (possivelmente infinitos). Uma triangulação de Delaunay $D(S)$ de S é uma triangulação em que cada ponto p_i ou fica na ou dentro da borda do circuncírculo de cada triângulo $T \in D(S)$.

Tem um caso chato em que todos os pontos são colineares.

Uma propriedade da triangulação de Delaunay é que ela maximiza o ângulo mínimo dentre todas as possíveis triangulações.

A Minimum Euclidean Spanning Tree de um conjunto de pontos é um subconjunto das arestas da sua triangulação de Delaunay.

Dualidade: Se S é não colinear e não existe um circuncírculo que passa por 4 pontos, então V e D são duais e é possível obter um a partir do outro em $O(n)$. Se for colinear, dá pra fazer de boa também: Se V e D' são duais, é possível obter D' a partir de D removendo todas as arestas tais que dois triângulos com essa aresta em comum tenham o mesmo circuncírculo.

Por conta da dualidade, é só construir um deles e gg. Pega a triangulação Delaunay, aí os triângulos que tiverem aresta em comum você conecta os circuncentros deles.

Isso é feito em $O(n \log n)$ (PS: se der tudo colinear tem que tratar diferente porque da erro)

053448

```

/**
 * Author: Philippe Legault
 * Date: 2016
 * License: MIT
 * Source: https://github.com/Bathlamos/delaunay-triangulation/
 * Description: Fast Delaunay triangulation.
 * Each circumcircle contains none of the input points.
 * There must be no duplicate points.
 * If all points are on a line, no triangles will be returned.
 * Should work for doubles as well, though there may be precision issues in 'circ'.
 * Returns triangles in order \{t[0][0], t[0][1], t[0][2], t[1][0], \dots\}, all counter-clockwise.
 * Time:  $O(n \log n)$ 
 * Status: stress-tested
 */
#pragma once

// #include "Point.h"

/**
 * Author: Ulf Lundstrom
 * Date: 2009-02-26
 * License: CC0
 * Source: My head with inspiration from tinyKACTL
 * Description: Class to handle points in the plane.
 * T can be e.g. double or long long. (Avoid int.)
 * Status: Works fine, used a lot
 */
#pragma once

template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template <class T> struct Point {
    typedef Point P;

```

```

    T x, y;
    explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x, y) < tie(p.x, p.y); }
    bool operator==(P p) const { return tie(x, y) == tie(p.x, p.y); }
    P operator+(P p) const { return P(x + p.x, y + p.y); }
    P operator-(P p) const { return P(x - p.x, y - p.y); }
    P operator*(T d) const { return P(x * d, y * d); }
    P operator/(T d) const { return P(x / d, y / d); }
    T dot(P p) const { return x * p.x + y * p.y; }
    T cross(P p) const { return x * p.y - y * p.x; }
    T cross(P a, P b) const { return (a - *this).cross(b - *this); }
    T dist2() const { return x * x + y * y; }
    double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this / dist(); } // makes dist()=1
    P perp() const { return P(-y, x); } // rotates +90 degrees
    P normal() const { return perp().unit(); }
    // returns point rotated 'a' radians ccw around the origin
    P rotate(double a) const { return P(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a)); }
    friend ostream& operator<<(ostream& os, P p) { return os << "(" << p.x << ", " << p.y << ")"; }
};

typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t lll; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX, LLONG_MAX); // not equal to any other point

struct Quad {
    Q rot, o;
    P p = arb;

```

```

    bool mark;
    P& F() { return r()->p; }
    Q& r() { return rot->rot; }
    Q prev() { return rot->o->rot; }
    Q next() { return r()->prev(); }
}* H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
    ll p2 = p.dist2(), A = a.dist2() - p2, B = b.dist2() -
        p2, C = c.dist2() - p2;
    return p.cross(a, b) * C + p.cross(b, c) * A + p.cross(c
        , a) * B > 0;
}
Q makeEdge(P orig, P dest) {
    Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
    H = r->o;
    r->r()->r() = r;
    rep(i, 0, 4) r = r->rot, r->p = arb, r->o = i & 1 ? r :
        r->r();
    r->p = orig;
    r->F() = dest;
    return r;
}
void splice(Q a, Q b) {
    swap(a->o->rot->o, b->o->rot->o);
    swap(a->o, b->o);
}
Q connect(Q a, Q b) {
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}

pair<Q, Q> rec(const vector<P>& s) {
    if (sz(s) <= 3) {
        Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.
            back());
        if (sz(s) == 2) return {a, a->r()};
        splice(a->r(), b);
    }
}

```

```

    auto side = s[0].cross(s[1], s[2]);
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r()
        };
}

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
Q A, B, ra, rb;
int half = sz(s) / 2;
tie(ra, A) = rec({all(s) - half});
tie(B, rb) = rec({sz(s) - half + all(s)});
while ((B->p.cross(H(A)) < 0 && (A = A->next())) || (A->
    p.cross(H(B)) > 0 && (B = B->r()->o)))
    ;
Q base = connect(B->r(), A);
if (A->p == ra->p) ra = base->r();
if (B->p == rb->p) rb = base;

#define DEL(e, init, dir)

\
Q e = init->dir;

\
if (valid(e))

\
while (circ(e->dir->F(), H(base), e->F())) {

\
    Q t = e->dir;

\
    splice(e, e->prev());

\
    splice(e->r(), e->r()->prev());

\
    e->o = H;
}

```



```

        \
        H = e;

        \
        e = t;

        \
    }
    for (;;) {
        DEL(LC, base->r(), o);
        DEL(RC, base, prev());
        if (!valid(LC) && !valid(RC)) break;
        if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
            base = connect(RC, base->r());
        else
            base = connect(base->r(), LC->r());
    }
    return {ra, rb};
}

vector<P> triangulate(vector<P> pts) {
    sort(all(pts));
    assert(unique(all(pts)) == pts.end());
    if (sz(pts) < 2) return {};
    Q e = rec(pts).first;
    vector<Q> q = {e};
    int qi = 0;
    while (e->o->F().cross(e->F(), e->p) < 0)
        e = e->o;
#define ADD

    \
    {

        \
        Q c = e;

        \
        do {

```

```

        \
        c->mark = 1;

        \
        pts.push_back(c->p);

        \
        q.push_back(c->r());

        \
        c = c->next();

        \
    } while (c != e);

    \
}
ADD;
pts.clear();
while (qi < sz(q))
    if (!(e = q[qi++])->mark) ADD;
return pts;
}

```

4.7.3 Área da união de polígonos

```

e45530

/**
 * Author: black_horse2014, chilli
 * Date: 2019-10-29
 * License: Unknown
 * Source: https://codeforces.com/gym/101673/submission/50481926
 * Description: Calculates the area of the union of $n$
 *               polygons (not necessarily
 *               convex). The points within each polygon must be given in
 *               CCW order.
 * (Epsilon checks may optionally be added to sideOf/sgn,
 * but shouldn't be needed.)

```

```

* Time:  $O(N^2)$ , where  $N$  is the total number of points
* Status: stress-tested, Submitted on ECNA 2017 Problem A
*/

// #include "Point.h" botei no código da triangulação
// #include "sideOf.h" tá aqui:
/**
 * Author: Ulf Lundstrom
 * Date: 2009-03-21
 * License: CC0
 * Source:
 * Description: Returns where  $p$  is as seen from  $s$ 
 *   towards  $e$ .  $1/0/-1$   $\rightarrow$  left/on
 * line/right. If the optional argument  $\epsilon$  is given 0 is
 * returned if  $p$  is within distance  $\epsilon$ 
 * from the line.  $p$  is supposed to be Point<T> where  $T$  is e.
 * g. double or long long. It uses products
 * in intermediate steps so watch out for overflow if using
 * int or long long. Usage: bool left =
 * sideOf(p1,p2,q)==1; Status: tested
 */

template <class P> int sideOf(P s, P e, P p) { return sgn(s.
    cross(e, p)); }

template <class P> int sideOf(const P& s, const P& e, const
    P& p, double eps) {
    auto a = (e - s).cross(p - s);
    double l = (e - s).dist() * eps;
    return (a > l) - (a < -l);
}

typedef Point<double> P;
double rat(P a, P b) { return sgn(b.x) ? a.x / b.x : a.y / b
    .y; }
double polyUnion(vector<vector<P>>& poly) {
    double ret = 0;
    rep(i, 0, sz(poly)) rep(v, 0, sz(poly[i])) {
        P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])
            ];

```

```

        vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
        rep(j, 0, sz(poly)) if (i != j) {
            rep(u, 0, sz(poly[j])) {
                P C = poly[j][u], D = poly[j][(u + 1) % sz(
                    poly[j])];
                int sc = sideOf(A, B, C), sd = sideOf(A, B,
                    D);
                if (sc != sd) {
                    double sa = C.cross(D, A), sb = C.cross(
                        D, B);
                    if (min(sc, sd) < 0) segs.emplace_back(
                        sa / (sa - sb), sgn(sc - sd));
                } else if (!sc && !sd && j < i && sgn((B - A)
                    .dot(D - C)) > 0) {
                    segs.emplace_back(rat(C - A, B - A), 1);
                    segs.emplace_back(rat(D - A, B - A), -1)
                        ;
                }
            }
        }
        sort(all(segs));
        for (auto& s : segs)
            s.first = min(max(s.first, 0.0), 1.0);
        double sum = 0;
        int cnt = segs[0].second;
        rep(j, 1, sz(segs)) {
            if (!cnt) sum += segs[j].first - segs[j - 1].
                first;
            cnt += segs[j].second;
        }
        ret += A.cross(B) * sum;
    }
    return ret / 2;
}

```

4.7.4 Intersecção de half-plane

$O(n \log n)$

2e310c

```

// Redefine epsilon and infinity as necessary. Be mindful of
precision errors.

```

```

const long double eps = 1e-9, inf = 1e9;

// Basic point/vector struct.
struct Point {

    long double x, y;
    explicit Point(long double x = 0, long double y = 0) : x
        (x), y(y) {}

    // Addition, subtraction, multiply by constant, dot
    // product, cross product.

    friend Point operator+(const Point& p, const Point& q) {
        return Point(p.x + q.x, p.y + q.y); }

    friend Point operator-(const Point& p, const Point& q) {
        return Point(p.x - q.x, p.y - q.y); }

    friend Point operator*(const Point& p, const long double
        & k) { return Point(p.x * k, p.y * k); }

    friend long double dot(const Point& p, const Point& q) {
        return p.x * q.x + p.y * q.y; }

    friend long double cross(const Point& p, const Point& q)
        { return p.x * q.y - p.y * q.x; }
};

// Basic half-plane struct.
struct Halfplane {

    // 'p' is a passing point of the line and 'pq' is the
    // direction vector of the line.
    Point p, pq;
    long double angle;

    Halfplane() {}
    Halfplane(const Point& a, const Point& b) : p(a), pq(b -
        a) { angle = atan2l(pq.y, pq.x); }

```

```

    // Check if point 'r' is outside this half-plane.
    // Every half-plane allows the region to the LEFT of its
    // line.
    bool out(const Point& r) { return cross(pq, r - p) < -
        eps; }

    // Comparator for sorting.
    bool operator<(const Halfplane& e) const { return angle
        < e.angle; }

    // Intersection point of the lines of two half-planes.
    // It is assumed they're never parallel.
    friend Point inter(const Halfplane& s, const Halfplane&
        t) {
        long double alpha = cross((t.p - s.p), t.pq) / cross
            (s.pq, t.pq);
        return s.p + (s.pq * alpha);
    }
};

// Actual algorithm
vector<Point> hp_intersect(vector<Halfplane>& H) {

    Point box[4] = { // Bounding box in CCW order
        Point(inf, inf), Point(-inf, inf), Point
            (-inf, -inf), Point(inf, -inf) };

    for (int i = 0; i < 4; i++) { // Add bounding box half-
        planes.
        Halfplane aux(box[i], box[(i + 1) % 4]);
        H.push_back(aux);
    }

    // Sort by angle and start algorithm
    sort(H.begin(), H.end());
    deque<Halfplane> dq;
    int len = 0;
    for (int i = 0; i < int(H.size()); i++) {

```

```

// Remove from the back of the deque while last half
// -plane is redundant
while (len > 1 && H[i].out(inter(dq[len - 1], dq[len
- 2]))) {
    dq.pop_back();
    --len;
}

// Remove from the front of the deque while first
// half-plane is redundant
while (len > 1 && H[i].out(inter(dq[0], dq[1]))) {
    dq.pop_front();
    --len;
}

// Special case check: Parallel half-planes
if (len > 0 && fabs1(cross(H[i].pq, dq[len - 1].pq))
    < eps) {
    // Opposite parallel half-planes that ended up
    // checked against each other.
    if (dot(H[i].pq, dq[len - 1].pq) < 0.0) return
        vector<Point>();

    // Same direction half-plane: keep only the
    // leftmost half-plane.
    if (H[i].out(dq[len - 1].p)) {
        dq.pop_back();
        --len;
    } else
        continue;
}

// Add new half-plane
dq.push_back(H[i]);
++len;
}

// Final cleanup: Check half-planes at the front against
// the back and vice-versa

```

```

while (len > 2 && dq[0].out(inter(dq[len - 1], dq[len -
2]))) {
    dq.pop_back();
    --len;
}

while (len > 2 && dq[len - 1].out(inter(dq[0], dq[1])))
{
    dq.pop_front();
    --len;
}

// Report empty intersection if necessary
if (len < 3) return vector<Point>();

// Reconstruct the convex polygon from the remaining
// half-planes.
vector<Point> ret(len);
for (int i = 0; i + 1 < len; i++) {
    ret[i] = inter(dq[i], dq[i + 1]);
}
ret.back() = inter(dq[len - 1], dq[0]);
return ret;
}

```

4.7.5 Distância de Manhattan máxima

Distância de Manhattan: soma dos módulos das diferenças de cada coordenada.

Distância de Chebyshev: máximo dos módulos das diferenças de cada coordenada.

Note que se $\alpha : (x, y) \rightarrow (x + y, x - y)$, teremos que a distância de Manhattan entre os pontos p e q é igual à de Chebyshev entre os pontos $\alpha(p)$ e $\alpha(q)$.

Código pra distância de Manhattan máxima em $O(n \cdot 2^d \cdot d)$, onde d é o número de dimensões

```

0cef8e
long long ans = 0;
for (int msk = 0; msk < (1 << d); msk++) {
    long long mx = LLONG_MIN, mn = LLONG_MAX;
    for (int i = 0; i < n; i++) {
        long long cur = 0;

```

```

    for (int j = 0; j < d; j++) {
        if (msk & (1 << j))
            cur += p[i][j];
        else
            cur -= p[i][j];
    }
    mx = max(mx, cur);
    mn = min(mn, cur);
}
ans = max(ans, mx - mn);
}

```

4.7.6 Manhattan Minimum Spanning Tree

Dados pontos no plano, encontrar arestas que os conectam e têm uma soma total mínima de pesos (manhattan é o peso).

83f913

```

struct point {
    long long x, y;
};

// Returns a list of edges in the format (weight, u, v).
// Passing this list to Kruskal algorithm will give the
// Manhattan MST.
vector<tuple<long long, int, int>> manhattan_mst_edges(
    vector<point> ps) {
    vector<int> ids(ps.size());
    iota(ids.begin(), ids.end(), 0);
    vector<tuple<long long, int, int>> edges;
    for (int rot = 0; rot < 4; rot++) { // for every
        rotation
        sort(ids.begin(), ids.end(),
            [&](int i, int j) { return (ps[i].x + ps[i].y)
                < (ps[j].x + ps[j].y); });
        map<int, int, greater<int>> active; // (xs, id)
        for (auto i : ids) {
            for (auto it = active.lower_bound(ps[i].x); it
                != active.end(); active.erase(it++)) {
                int j = it->second;
            }
        }
    }
    return edges;
}

```

```

        if (ps[i].x - ps[i].y > ps[j].x - ps[j].y)
            break;
        assert(ps[i].x >= ps[j].x && ps[i].y >= ps[j].y);
        edges.push_back({(ps[i].x - ps[j].x) + (ps[i].y - ps[j].y), i, j});
    }
    active[ps[i].x] = i;
}
for (auto& p : ps) { // rotate
    if (rot & 1)
        p.x *= -1;
    else
        swap(p.x, p.y);
}
return edges;
}

```

5 Estruturas de dados

5.1 Minimum Stack e Queue

Problema 5.1.1: Quero uma stack que também guarde o menor elemento.

Solução: Faz uma stack de `pair<int, int>` em que a primeira componente é o elemento em si e a segunda componente é o menor elemento dele pra baixo. Então se eu quero botar um elemento fica trivial. Se eu remover um elemento fica trivial também.

Problema 5.1.2: Quero a mesma coisa só que pra uma queue (removendo eles da frente da fila).

Solução 1: Faz uma deque de `int` que guarda os elementos necessários. Na cabeça da fila tem o menor cara e ela tá ordenada. Quando bota um cara qualquer daí vc vai tirando os caras do final da fila maiores que esse cara e aí dps disso bota ele. Mas na hora de tirar o cara da fila, precisamos saber quem é esse cara porque pode ser que ele tenha sido removido da estrutura de dados em si porque ele era maior que alguém.

```

3c87dc
deque<int> q;

int minimum = q.front();

// Add
while (!q.empty() && q.back() > new_element)
    q.pop_back();
q.push_back(new_element);

// Remove
if (!q.empty() && q.front() == remove_element) q.pop_front();

```

Solução 2: Nessa modificação não precisamos saber que elemento estamos querendo remover. Para isso, armazena tb o índice do cara na fila, quantos já foram add e quantos já foram removidos.

```

a601fc
deque<pair<int, int>> q;
int cnt_added = 0;
int cnt_removed = 0;

int minimum = q.front().first;

```

```

while (!q.empty() && q.back().first > new_element)
    q.pop_back();
q.push_back({new_element, cnt_added});
cnt_added++;

if (!q.empty() && q.front().second == cnt_removed) q.
    pop_front();
cnt_removed++;

```

Solução 3: Nessa modificação, de fato guardamos todos os elementos. Também dá pra remover um elemento da frente sem saber o valor dele. A ideia é que dá pra simular uma fila usando duas stacks s_1 e s_2 (cada uma delas modificada de modo que achemos o mínimo em $O(1)$). Adicionamos novos elementos a s_1 e removemos elementos de s_2 . Se s_2 fica vazia movemos todos os elementos de s_1 para s_2 . Daí basta achar o mínimo nas duas stacks.

```

d7fd03
if (s1.empty() || s2.empty())
    minimum = s1.empty() ? s2.top().second : s1.top().second
    ;
else
    minimum = min(s1.top().second, s2.top().second);

int minimum = s1.empty() ? new_element : min(new_element, s1
    .top().second);
s1.push({new_element, minimum});

if (s2.empty()) {
    while (!s1.empty()) {
        int element = s1.top().first;
        s1.pop();
        int minimum = s2.empty() ? element : min(element, s2
            .top().second);
        s2.push({element, minimum});
    }
}
int remove_element = s2.top().first;
s2.pop();

```

Problema 5.1.3: Queremos achar o mínimo de cada subarray de tamanho M i.e. todos os valores

$\min_{j \leq i \leq M-1+j} A[j]$
para cada $j = 0 \dots N - M$.

Solução: Trivial a partir do que foi apresentado.

5.2 Disjoint Set Union

Esse é o Union Find. Com essa estrutura, temos as seguintes operações. Cada uma delas roda em $O(1)$ em média.

1. **make_set(v)** cria um conjunto $\{v\}$
2. **union_sets(a, b)** une o conjunto que contém o elemento a com o que contém o b
3. **find_set(v)** retorna o "representante" do conjunto que contém o elemento v

Para o seguinte código, suponha que as arrays são de tamanho MAXN.

```

3d58ed
int find_set(int v) {
    // Path compression. Sem isso, fica O(log n)
    // ao invés de O(1)
    if (v == parent[v]) return v;
    return parent[v] = find_set(parent[v]);
}

void make_set(int v) {
    parent[v] = v;
    rank[v] = 0;
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (rank[a] < rank[b]) swap(a, b);
        parent[b] = a;
        if (rank[a] == rank[b]) rank[a]++;
    }
}

```

Problema 5.2.1: Inicialmente temos um grafo vazio. Em updates, conectamos dois vértices a e b . Em queries, vemos se dois vértices estão na mesma componente.

Solução: Trivial.

Problema 5.2.2: Uma imagem é inicialmente toda branca. Numa query, o pixel (x, y) fica preto. Determinar o tamanho da maior componente branca.

Solução: Vai iterando pelos pixels brancos e fazendo o óbvio. A vantagem sobre DFS/BFS é que processa os elementos da matriz linha a linha. Isso usa $O(\min(m, n))$ de memória. Sendo mais claro, quando você processa uma linha, basta usar a linha anterior.

Problema 5.2.3: Você vai receber apenas queries (l, r, c) onde vc colore os índices de l a r (inicialmente não coloridos i.e. com a cor 0) com a cor c .

Solução: Aqui será usado DSU sem size/rank, aumentando a complexidade de cada operação para $O(\log n)$. Isso será feito porque o representante é importante aqui.

Processamos as queries em reverso. Trabalhamos com um grafo funcional. Inicialmente, todo vértice aponta para si mesmo. Para cada query, começo em l , aí vou pro eventual sucessor de l , digamos v . Daí seto o sucessor de v pra $v + 1$ e repito enquanto eu tiver que $v \leq r$.

Repare que, ao fazer isso, cada vértice aponta (eventualmente) para o próximo não colorido.

```

9f4c29
for (int i = 0; i <= L; i++) {
    make_set(i);
}

for (int i = m - 1; i >= 0; i--) {
    int l = query[i].l;
    int r = query[i].r;
    int c = query[i].c;
    for (int v = find_set(l); v <= r; v = find_set(v)) {
        answer[v] = c;
        parent[v] = v + 1;
    }
}

```

Tem como fazer uma otimização de **union by rank** se guardarmos a próxima não pintada num array end. Daí, dá bom na hora de dar merge em dois sets.

```

14b04b
void dsumerge(int x, int y) {

```

```

    x = find(x);
    y = find(y);
    parents[x] = parents[y] = max(x, y);
}

// no for de dentro do int v = find_set(1), coloca
// dsumerge(v, v+1) ao inves de parent[v] = v+1
// otimizacao de 1.32s pra 1.12s no SPOJ/CFLARR

```

Problema 5.2.4: Você quer manter, pra cada vértice, a distância até o representante.

Solução: Só fazer um pair.

```

                                045864
void make_set(int v) {
    parent[v] = make_pair(v, 0);
    rank[v] = 0;
}

pair<int, int> find_set(int v) {
    if (v != parent[v].first) {
        int len = parent[v].second;
        parent[v] = find_set(parent[v].first);
        parent[v].second += len;
    }
    return parent[v];
}

void union_sets(int a, int b) {
    a = find_set(a).first;
    b = find_set(b).first;
    if (a != b) {
        if (rank[a] < rank[b]) swap(a, b);
        parent[b] = make_pair(a, 1);
        if (rank[a] == rank[b]) rank[a]++;
    }
}

```

Problema 5.2.5: Você quer adicionar arestas entre vértices e dizer de maneira online se uma componente é um grafo bipartido.

Solução: Mantém um pair com a paridade. Atualiza a paridade toda vez que der find. E na hora do union, a paridade do ex-líder de uma componente é atual-

izada de acordo com a paridade do lugar que juntou as duas componentes.

```

                                118296
void make_set(int v) {
    parent[v] = make_pair(v, 0);
    rank[v] = 0;
    bipartite[v] = true;
}

pair<int, int> find_set(int v) {
    if (v != parent[v].first) {
        int parity = parent[v].second;
        parent[v] = find_set(parent[v].first);
        parent[v].second ^= parity;
    }
    return parent[v];
}

void add_edge(int a, int b) {
    pair<int, int> pa = find_set(a);
    a = pa.first;
    int x = pa.second;

    pair<int, int> pb = find_set(b);
    b = pb.first;
    int y = pb.second;

    if (a == b) {
        if (x == y) bipartite[a] = false;
    } else {
        if (rank[a] < rank[b]) swap(a, b);
        parent[b] = make_pair(a, x ^ y ^ 1);
        bipartite[a] ^= bipartite[b];
        if (rank[a] == rank[b]) ++rank[a];
    }
}

bool is_bipartite(int v) { return bipartite[find_set(v).first]; }

```

Problema 5.2.6: Range minimum query (offline) em $O(\alpha(n))$.

Solução: Pra cada $i = 0 \dots n-1$, vamos processar todas as queries com $R = i$. Enquanto processamos, mantemos um DSU tal que o i aponta pro primeiro $j > i$ com $a[j] < a[i]$. Então a resposta pra query é simplesmente $a[\text{find}(L)]$. Dá pra usar path compression sem nenhum ajuste. Mas, pra usar union by rank, precisamos guardar o líder de verdade num array separado.

eb8519

```
struct Query {
    int L, R, idx;
};

vector<int> answer;
// container[i] tem tds as queries com R=i
vector<vector<Query>> container;

// ...

stack<int> s;
for (int i = 0; i < n; i++) {
    while (!s.empty() && a[s.top()] > a[i]) {
        parent[s.top()] = i;
        s.pop();
    }
    s.push(i);
    for (Query q : container[i]) {
        answer[q.idx] = a[find_set(q.L)];
    }
}
```

Problema 5.2.7: Resolver LCA com DSU

Solução: Veja o problema correspondente na parte de LCA.

Problema 5.2.8: Fazer um DSU explícito i.e. existe um array de vector tal que pra todo representante de componente, é mantido um vector dos caras da componente atual. Fazer isso do jeito intuitivo leva apenas $O(m + n \log n)$, onde m é o número de queries nos n elementos.

1ce8f1

```
vector<int> lst [MAXN];
int parent [MAXN];

void make_set(int v) {
    lst[v] = vector<int>(1, v);
}
```

```
parent[v] = v;
}

int find_set(int v) { return parent[v]; }

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (lst[a].size() < lst[b].size()) swap(a, b);
        while (!lst[b].empty()) {
            int v = lst[b].back();
            lst[b].pop_back();
            parent[v] = a;
            lst[a].push_back(v);
        }
    }
}
```

P.S.: Dá pra generalizar essa ideia de adicionar a menor estrutura de dados na maior. Acaba sendo log em média. Por exemplo, considere o seguinte problema. Tenho uma árvore e cada folha tem um número. Pra cada nó, quero o número de números distintos nas folhas da subárvore. Basta ir dando merge nos sets de acordo com essa regrinha de ir botando os menores nos maiores.

Problema 5.2.9: Manter estrutura limpa de árvore i.e. saber o pai real

Solução: Basta manter um array real_parent. Você pode pensar que na hora de unir, por ter que mudar o pai de todo mundo, isso vai demorar mais. Só que acontece que pelo argumento de 5.2.8, isso ocorre em log na média.

5.3 Fenwick tree

Dada uma array $A[1 \dots n]$, obtenho um array $T[1 \dots n]$ tal que

$$T_i = \sum_{j=g(i)}^i A_j$$

em que $g(i) = i - (i \& (-i)) + 1$. Com isso, pra obter a soma de 1 até k , basta ir somando $T_k + T_{g(k)-1} + \dots$. Pra dar update em A_i , você precisa iterar pelos j tais que $g(j) \leq i \leq j$.

Essa escolha pra g faz com que isso possa ser feito range query e update pontual (online) em $O(\log n)$ ambas. Também fácil ver que dá pra trocar isso por qualquer

operação em grupo $(G, *)$. Aqui a versão clássica:

```

2b1375
// voce tem um array [1..n] de input. faz um vetor bit de
// maxn zerado
// pra inicializar, faz update(i, arr[i]) pra cada i
// (isso adiciona arr[i] no indice i em log)
// o resto e obvio
ll sum(int idx) { // soma ate idx
    ll ret = 0;
    for (; idx > 0; idx -= idx & -idx)
        ret += bit[idx];
    return ret;
}

ll sum_interval(int l, int r) { return sum(r) - sum(l - 1);
}

void add(int idx, ll delta) {
    for (; idx <= n; idx += idx & -idx)
        bit[idx] += delta;
}

```

Também dá pra fazer uma BIT 2D

```

a5bdd9
// temos um retangulo n x m e sum(x, y) retorna a soma em
// [1, x] cartesiano [1, y]
ll sum(int x, int y) {
    ll ret = 0;
    for (int i = x; i > 0; i -= i & (-i))
        for (int j = y; j > 0; j -= j & (-j))
            ret += bit[i][j];
    return ret;
}

// assumindo: x1 <= x2 e y1 <= y2
// soma do retangulo que vai de (x1, y1) ate (x2, y2)
ll sum_rect(int x1, int y1, int x2, int y2) {
    return sum(x2, y2) + sum(x1 - 1, y1 - 1) - sum(x2, y1 - 1) - sum(x1 - 1, y2);
}

```

```

void add(int x, int y, ll delta) {
    for (int i = x; i <= n; i += i & (-i))
        for (int j = y; j <= m; j += j & (-j))
            bit[i][j] += delta;
}

```

Em particular, se forem poucos pontos dá pra usar um map se não tiver memória suficiente. Mas aí entra um log na complexidade, tem que ficar esperto.

A grande vantagem de BIT é que o código é muito mais curto que seg, por exemplo.

Problema 5.3.1: Range update e point query.

Solução: Ao invés de fazer uma BIT baseada na array $A[1..n]$ de interesse, faça na array $B[1..n+1]$ definida por $B[i] = A[i] - A[i-1]$ ($A[0] = 0$). Daí, pra adicionar x em $A[l..r]$, basta adicionar x em $B[l]$ e remover x em $B[r+1]$. Pra obter $A[k]$, basta calcular $B[1] + \dots + B[k]$.

Problema 5.3.2: Range update e range query.

Solução: Faremos duas bits $T_1[]$ e $T_2[]$ (inicialmente zeradas) de modo que a resposta da nossa query de soma $[1..k]$ seja $\text{soma}(T_1, k) \cdot k - \text{soma}(T_2, k)$. A ideia é que quando eu quero adicionar x no intervalo $[l, r]$ da minha array, eu faço $\text{add}(T_1, l, x)$, $\text{add}(T_1, r+1, -x)$, $\text{add}(T_2, l, x \cdot (l-1))$ e $\text{add}(T_2, r+1, -x \cdot r)$.

Se fugir muito disso, talvez compense fazer uma seg mesmo.

5.4 Segmentation Tree

5.4.1 Seg Normal

```

d55e4c
// 1 indexado

#define EL 2*i, l, (l+r)/2
#define ER 2*i+1, (l+r)/2+1, r
#define INI 1, 1, n

struct Node {
    ll v;

    Node(ll v=0): v(v) {}
    Node operator+(Node ot) { return Node(v+ot.v); }
    void update(ll _v) { v = _v; }
};

```

```

struct Seg {
    Node dt[4*maxn];

    Node build(int i, int l, int r) {
        if(l == r) return dt[i] = Node(a[r]);
        return dt[i] = build(EL) + build(ER);
    }

    Node query(int ql, int qr, int i, int l, int r) {
        if(l > qr || r < ql) return Node();
        if(ql <= l && r <= qr) return dt[i];
        return query(ql, qr, EL) + query(ql, qr, ER);
    }

    Node update(int pos, ll v, int i, int l, int r) {
        if(r < pos || l > pos) return dt[i];
        if(l == r) {
            dt[i].update(v);
            return dt[i];
        }

        return dt[i] = update(pos, v, EL) + update(pos, v,
            ER);
    }
};

```

5.4.2 Memória eficiente

Para fazer a seg usar $2n$ de memória em vez de $4n$, basta mudar como se calcula os índices:

```

d41d8c
#define EL i+1, l, (l+r)/2
#define ER i + 2*((l+r)/2 - l + 1), (l+r)/2+1, r

// Usa 2 * n memoria
// So muda indices

```

Não notei nenhuma diferença na performance nos testes feitos.

5.4.3 Lazy

893436

```

// 1 indexado

#define LEFT 2*i
#define RIGHT 2*i+1

#define EL LEFT, l, (l+r)/2
#define ER RIGHT, (l+r)/2+1, r
#define INI 1, 1, n

struct Node {
    ll v, acc=0; // Adiciona acumulador

    Node(ll v=0): v(v) {}
    Node operator+(Node ot) { return Node(v+ot.v); }

    // Update apenas marca
    void update(ll _v) {acc = _v;}
    // Realmente aplica update. CUIDADO SE NAO VERIFICOU SE
    // TA MARCADO
    void apply(int l, int r) { v = acc * (r-l+1); }
};

struct Seg {
    Node dt[4*maxn];
    bool mark[4*maxn]; // Se tem update

    Node build(int i, int l, int r) {
        mark[i] = false; // Inicializa mark
        if(l == r) return dt[i] = Node(a[r]);
        return dt[i] = build(EL) + build(ER);
    }

    void flush(int i, int l, int r) {
        if(l != r) {
            dt[LEFT].update(dt[i].acc);
            dt[RIGHT].update(dt[i].acc);
            mark[LEFT] = mark[RIGHT] = true;
        }
        dt[i].apply(l, r);
    }
};

```

```

    mark[i] = false;
}

Node query(int ql, int qr, int i, int l, int r) {
    if(mark[i]) flush(i, l, r);

    if(l > qr || r < ql) return Node();
    if(ql <= l && r <= qr) return dt[i];
    return query(ql, qr, EL) + query(ql, qr, ER);
}

Node update(int ql, int qr, ll v, int i, int l, int r) {
    if(mark[i]) flush(i, l, r);

    if(l > qr || r < ql) return dt[i];
    if(ql <= l && r <= qr) {
        dt[i].update(v);
        flush(i, l, r);
        return dt[i];
    }

    return dt[i] = update(ql, qr, v, EL) + update(ql, qr,
        , v, ER);
}
};

```

5.4.4 Persistent

Vértices apontam uns aos outros. Cada update aloca um vértice para cada um modificado, e cria uma nova raiz.

2d7467

```

// 1 indexado
// Note que EL, ER mudaram um pouco

#define EL l, (l + r) / 2
#define ER (l + r) / 2 + 1, r
#define INI l, n

struct Node {
    ll v;
    Node *l, *r;

```

```

Node(ll v = 0) : v(v), l(0), r(0) {}
Node(Node* l, Node* r) : l(l), r(r), v(0) {
    // Merge de nos eh aqui
    if (l) v += l->v;
    if (r) v += r->v;
}

};

Node* roots[maxop];

Node* build(int l, int r) {
    if (l == r) return new Node(a[r]);
    return new Node(build(EL), build(ER));
}

// Atencao que aqui não retorna ponteiro
Node query(int ql, int qr, Node* v, int l, int r) {
    if (l > qr || r < ql) return Node();
    if (ql <= l && r <= qr) return *v;
    Node q1 = query(ql, qr, v->l, EL), q2 = query(ql, qr, v
        ->r, ER);
    return Node(&q1, &q2);
}

Node* update(int pos, ll upd, Node* v, int l, int r) {
    if (r < pos || l > pos) return v;
    if (l == r) {
        return new Node(v->v + upd);
    }

    return new Node(update(pos, upd, v->l, EL), update(pos,
        upd, v->r, ER));
}

```

5.4.5 Esparsa

Cria os vértices sob demanda. Cuidado com uso de memória (talvez tenha que por os nós em um vetor, e salvar o índice em vez de ponteiros. Isso economiza 12 bytes por nó).

908cba

```
// 1 indexado
// Note que EL, ER mudaram um pouco

#define EL 1, (1 + r) / 2
#define ER (1 + r) / 2 + 1, r
#define INI &seg.root, 1, n

struct Node {
    ll v;
    Node *l, *r;

    Node(ll v = 0) : v(v), l(0), r(0) {}
    void merge(Node l, Node r) {
        v = l.v + r.v;
    }
    void merge() { merge(*l, *r); }
    void update(ll _v) { v = _v; }
};

struct Seg {
    Node root;

    // Cuidado se for fazer lazy, tem que verificar se nao eh
    // folha antes
    inline void extend(Node *v) {
        if(v->l == 0) {
            v->l = new Node();
            v->r = new Node();
        }
    }

    void update(ll pos, ll x, Node *v, ll l, ll r) {
        if(r < pos || l > pos) return;
        if(l == r) {
            v->update(x);
            return;
        }
        extend(v);
    }
};
```

```
update(pos, x, v->l, EL); update(pos, x, v->r, ER);
v->merge();
}

Node query(int ql, int qr, Node *v, int l, int r) {
    if(l > qr || r < ql) return Node();
    if(ql <= l && r <= qr) return *v;

    extend(v);

    Node res;
    res.merge(query(ql, qr, v->l, EL), query(ql, qr, v->r,
        ER));
    return res;
}
};
```

5.5 Sqrt decomposition

Ideia principal: dividir em buckets de tamanho raiz.

Bota tamanho const pro tamanho de bloco.

5.5.1 Algoritmo de Mo

Responder Q queries de maneira offline em $O((N + Q)\sqrt{N})$

Parece pior, mas a questão é que se fosse fazer do jeito normal, o passo de merge teria complicações.

Aqui o código pro DQUERY do SPOJ. Cada query pede o número de valores distintos num dado intervalo.

a3ce2f

```
#include <bits/stdc++.h>

#define MAX 30000
#define MVAL 1000100
#define MRESPS 200100

using namespace std;

int n, q;
int arr[MAX];
```

```

int freqs[MVAL];
int respostas[MRESPS];
int ans = 0;
int bl;

struct Query {
    int l, r, id;
    bool operator<(Query other) const {
        return make_pair(l / bl, r) < make_pair(other.l / bl, other.r);
    }
};

void add(int x) {
    x = arr[x];
    freqs[x]++;
    if (freqs[x] == 1) {
        ans++;
    }
}

void remove(int x) {
    x = arr[x];
    freqs[x]--;
    if (freqs[x] == 0) {
        ans--;
    }
}

int main() {
    memset(freqs, 0, sizeof(freqs));

    cin >> n;
    bl = sqrt(n + 0.0) + 1;

    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cin >> q;

```

```

int l, r;
vector<Query> queries;

for (int i = 0; i < q; i++) {
    Query qq;
    cin >> qq.l >> qq.r;
    qq.l--;
    qq.r--;
    qq.id = i;
    queries.push_back(qq);
}

sort(queries.begin(), queries.end());

int cur_l = 0;
int cur_r = -1;

for (int i = 0; i < q; i++) {
    while (cur_l > queries[i].l) {
        cur_l--;
        add(cur_l);
    }
    while (cur_r < queries[i].r) {
        cur_r++;
        add(cur_r);
    }
    while (cur_l < queries[i].l) {
        remove(cur_l);
        cur_l++;
    }
    while (cur_r > queries[i].r) {
        remove(cur_r);
        cur_r--;
    }
    respostas[queries[i].id] = ans;
}

for (int i = 0; i < q; i++) {
    cout << respostas[i] << "\n";
}

```

```
    return 0;
}
```

5.5.2 Mo com update

Cada query além do l, r e id, tem o t, que indica quantos updates tem antes dela

Ordenam-se as queries por bloco do l. Empates são resolvidos pelo bloco do r. Empates são resolvidos pelo t

Blocos de tamanho $(2n^2)^{1/3}$

Depois disso faz:

```
979973
int L = 0, R = -1, t = 0;

for (auto q : qs) {
    while (L > q.l)
        add(--L);
    while (R < q.r)
        add(++R);
    while (L < q.l)
        remove(L++);
    while (R > q.r)
        remove(R--);
    viagem_no_tempo(q.t, t, q.l, q.r);
    t = q.t;
    anses[q.id] = curr_ans;
}
```

em que o viagem no tempo percorre as updates no intervalo de tempo de q.t a t, oq pode cancelar algumas updates ou ativar algumas updates

Segue o código de uma questão que pedia, num intervalo, a soma dos caras distintos

```
f79749
#include <bits/stdc++.h>

const int maxn = 5e4 + 100;
const int maxq = 1e6 + 100;
const int B = 1700;
```

```
using namespace std;

int n, m;
int arr[maxn], cp_arr[maxn], freqs[maxn + maxq];
long long int anses[maxq];
int is_query[maxq];

unordered_map<int, int> ids;
int idsa[maxn];

long long int curr_ans = 0;

struct Q {
    int l, r, id, t, ce;
    Q(int l, int r, int id, int t) : l(l), r(r), id(id), t(t) {}
    bool operator<(const Q& q2) const {
        if (l / B > q2.l / B) return false;
        if (l / B < q2.l / B) return true;
        if (r / B > q2.r / B) return false;
        if (r / B < q2.r / B) return true;
        return t < q2.t;
    }
};

vector<Q> qs;
vector<Q> ups;

void viagem_no_tempo(int t_novo, int t_ant, int l, int r)
{
    int i;
    int idupsir, idupsil, idupsirid;
    for (i = t_ant; i <= t_novo - 1; i++) {

        if (l <= ups[i].l && ups[i].l <= r) {
            idupsir = ups[i].t;
            idupsil = idsa[ups[i].l];
            freqs[idupsir]++;
            if (freqs[idupsir] == 1) curr_ans += ups[i].r;
            freqs[idupsil]--;
        }
    }
}
```

```

        if (freqs[idupsil] == 0) curr_ans -= arr[ups[i].l];
    }
    arr[ups[i].l] = ups[i].r;
    idsa[ups[i].l] = ups[i].t;
}
for (i = t_antes - 1; i > t_novo - 1; i--) {

    idupsirid = ups[i].ce;
    idupsil = idsa[ups[i].l];
    // Quero remover a update i
    if (l <= ups[i].l && ups[i].l <= r) {
        freqs[idupsirid]++;
        if (freqs[idupsirid] == 1) curr_ans += ups[i].id;
    };
    freqs[idupsil]--;
    if (freqs[idupsil] == 0) curr_ans -= arr[ups[i].l];
}
arr[ups[i].l] = ups[i].id; // id da update = valor
                           antes dela
idsa[ups[i].l] = ups[i].ce;
}

void add(int x) {
    freqs[idsa[x]]++;
    if (freqs[idsa[x]] == 1) curr_ans += arr[x];
}

void remove(int x) {
    freqs[idsa[x]]--;
    if (freqs[idsa[x]] == 0) curr_ans -= arr[x];
}

int main() {

    memset(freqs, 0, sizeof(freqs));

    cin >> n;

```

```

int curr_id = 0;

for (int i = 0; i < n; i++) {
    cin >> arr[i];
    cp_arr[i] = arr[i];
    if (ids.find(arr[i]) == ids.end()) ids[arr[i]] =
        curr_id++;
}

cin >> m;

char op;
int tempo = 0;
int l, r;

for (int i = 0; i < m; i++) {
    cin >> op >> l >> r;
    anses[i] = 0;
    if (op == 'Q') {
        is_query[i] = 1;
        l--;
        r--;
        qs.push_back(Q(l, r, i, tempo));
    } else if (op == 'U') {
        is_query[i] = 0;
        l--;
        ups.push_back(Q(l, r, cp_arr[l], ++tempo));
        if (ids.find(r) == ids.end()) ids[r] = curr_id
            ++;
        ups[(int)ups.size() - 1].t = ids[r];
        ups[(int)ups.size() - 1].ce = ids[cp_arr[l]];
        cp_arr[l] = r;
    }
}

for (int i = 0; i < n; i++)
    idsa[i] = ids[arr[i]];

sort(qs.begin(), qs.end());

```



```

int L = 0, R = -1, t = 0;

for (auto q : qs) {
    while (L > q.l)
        add(--L);
    while (R < q.r)
        add(++R);
    while (L < q.l)
        remove(L++);
    while (R > q.r)
        remove(R--);
    viagem_no_tempo(q.t, t, q.l, q.r);
    t = q.t;
    anses[q.id] = curr_ans;
}

for (int i = 0; i < m; i++)
    if (is_query[i]) cout << anses[i] << "\n";
}

```

5.6 Wavelet tree

Imagina que você tem uma array de inteiros. Essa estrutura permite você:

1. computar o k -ésimo menor valor de um segmento
2. Dado um índice i , dar swap nos índices i e $i + 1$
3. Colocar um cara no final da array
4. Tirar um cara do final da array
5. Encontrar a quantidade de vezes que números de x até y aparecem em um determinado intervalo.

0ec832

```

#include <bits/stdc++.h>
using namespace std;

/** Wavelet Tree data structure. Input array is 0 based */
struct waveletTree {

```

```

int lo, hi, mi; // minimum, maximum and (lo+hi)/2
                // element on array
waveletTree *L, *R; // children
vector<int> mp; // map how many elements went left

#define mapLeft(i) (mp[i]) // original mapLeft */
#define mapRight(i) (i - mapLeft(i)) // original mapRight */

/** *beg points to the first element, *end points to
    after the last element (just like stl
    * default functions). Elements are in range [lo..hi],
    inclusive */
waveletTree(int* beg, int* end, int lo, int hi) { // O(
nlogA)
    L = R = NULL;
    this->lo = lo;
    this->hi = hi;
    this->mi = (lo + hi) / 2;

    mp.reserve(end - beg + 1);
    mp.push_back(0);
    for (auto it = beg; it != end; it++) {
        mp.push_back(mp.back() + ((*it) <= mi));
    }

    if (lo != hi) {
        auto pivot =
            stable_partition(beg, end, [&](int x) {
                return x <= mi; }); // split the vector
        L = new waveletTree(beg, pivot, lo, mi);
        R = new waveletTree(pivot, end, mi + 1, hi);
    }
}

/** K-th smallest element on range[l..r] */
int kthSmallest(int k, int l, int r) {
    if (l > r) return -1; // out of bounds
    if (lo == hi) return lo; // leaf node

    int inLeft = mapLeft(r) - mapLeft(l - 1);

```

```

    if (k <= inLeft)
        return L->kthSmallest(k, mapLeft(l - 1) + 1,
                               mapLeft(r));
    else
        return R->kthSmallest(k - inLeft, mapRight(l - 1) + 1,
                               mapRight(r));
}

/** Frequency of elements between [x..y] in array[l..r]
 */
int rangeCount(int x, int y, int l, int r) {
    if (l > r || lo > y || hi < x) return 0; // out of
    bounds
    if (lo >= x && hi <= y) return r - l + 1; // total
    fit

    if (mi >= y)
        return L->rangeCount(x, y, mapLeft(l - 1) + 1,
                              mapLeft(r));
    else if (mi <= x)
        return R->rangeCount(x, y, mapRight(l - 1) + 1,
                              mapRight(r));
    else
        return L->rangeCount(x, mi, mapLeft(l - 1) + 1,
                              mapLeft(r)) +
            R->rangeCount(mi + 1, y, mapRight(l - 1)
                          + 1, mapRight(r));
}

/** Swap elements a[i] and a[i+1] */
void swapContiguous(int i) {
    if (lo == hi) return; // leaf node, no need to swap

    bool iLeft = mapLeft(i) == mapLeft(i - 1) + 1;
    // if a[i] <= mi
    bool i1Left = (mapLeft(i + 1)) == (mapLeft(i) + 1);
    // if a[i+1] <= mi

    if (iLeft && !i1Left)
        mapLeft(i)--;

```

```

    else if (!iLeft && i1Left)
        mapLeft(i)++;
    else if (iLeft && i1Left)
        L->swapContiguous(mapLeft(i));
    else
        R->swapContiguous(mapRight(i));
}

/** Push element k to end of array */
void push_back(int k) {
    mp.push_back(mp.back() + (k <= mi));
    if (lo != hi) {
        if (k <= mi)
            L->push_back(k);
        else
            R->push_back(k);
    }
}

/** Pop element k from the end of array */
void pop_back(int k) {
    mp.pop_back();
    if (lo != hi) {
        if (k <= mi)
            L->pop_back(k);
        else
            R->pop_back(k);
    }
}

~waveletTree() {
    if (L) delete L;
    if (R) delete R;
}

#undef mapLeft
#undef mapRight
};

const int MAXN = 1e5 + 5;

```

```

int a[MAXN];

// https://www.spoj.com/problems/ILKQUERY/
int main() {
    int n, q;
    scanf("%d%d", &n, &q);

    vector<int> all;
    for (int i = 1; i <= n; i++) {
        scanf("%d", a + i);
        all.push_back(a[i]);
    }
    sort(all.begin(), all.end());
    all.erase(unique(all.begin(), all.end()), all.end());
    for (int i = 1; i <= n; i++)
        a[i] = lower_bound(all.begin(), all.end(), a[i]) -
            all.begin();

    int N = all.size();
    vector<vector<int>> pos(N);
    for (int i = 1; i <= n; i++)
        pos[a[i]].push_back(i - 1);

    waveletTree T(a + 1, a + n + 1, 0, all.size());
    while (q--) {
        int k, i, l;
        scanf("%d%d%d", &k, &i, &l);
        int ans = T.kthSmallest(k, l, i + 1);
        printf("%d\n", l <= (int)pos[ans].size() ? pos[ans][
            l - 1] : -1);
    }

    return 0;
}

```

Com toggle:

37765d

```

#include <bits/stdc++.h>
using namespace std;

```

```

/** Wavelet Tree data structure. Input array is 0 based */
struct waveletTree {
    int lo, hi, mi; // minimum, maximum and (lo+hi)/2
                    // element on array
    waveletTree *L, *R; // children

    vector<int> mp; // map how many elements went left
    struct Bit { // count how many indices are active
        vector<int> bt;
        Bit() {}

        inline void init(int n) {
            bt.assign(n, 0);
            for (int i = 1; i < n; i++)
                update(i, 1);
        }

        inline void update(int i, int x) {
            for (; i < (int)bt.size(); i += i & -i)
                bt[i] += x;
        }

        inline int query(int i) {
            int r = 0;
            for (; i > 0; i -= i & -i)
                r += bt[i];
            return r;
        }

        inline int query(int i, int j) { return query(j) -
            query(i - 1); }
    } bt;

#define mapLeft(i) (mp[i]) // original mapLeft */
#define mapRight(i) (i - mapLeft(i)) // original mapRight */

    /** *beg points to the first element, *end points to
        after the last element (just like stl
        * default functions). Elements are in range [lo..hi],
        inclusive */
    waveletTree(int* beg, int* end, int lo, int hi) { // O(

```

```

nlogA)
L = R = NULL;
this->lo = lo;
this->hi = hi;
this->mi = (lo + hi) / 2;

bt.init(end - beg + 1);

mp.reserve(end - beg + 1);
mp.push_back(0);
for (auto it = beg; it != end; it++)
    mp.push_back(mp.back() + ((*it) <= mi));

if (lo != hi) {
    auto pivot =
        stable_partition(beg, end, [&](int x) {
            return x <= mi; }); // split the vector
    L = new waveletTree(beg, pivot, lo, mi);
    R = new waveletTree(pivot, end, mi + 1, hi);
}
}

/** K-th smallest element on range[l..r] */
int kthSmallest(int k, int l, int r) {
    if (l > r) return -1; // out of bounds
    if (lo == hi) return lo; // leaf node

    int inLeft = mapLeft(r) - mapLeft(l - 1);
    if (k <= inLeft)
        return L->kthSmallest(k, mapLeft(l - 1) + 1,
            mapLeft(r));
    else
        return R->kthSmallest(k - inLeft, mapRight(l - 1) + 1, mapRight(r));
}

/** Frequency of elements between [x..y] in array[l..r] */
int rangeCount(int x, int y, int l, int r) {
    if (l > r || lo > y || hi < x) return 0; // out of

```

```

    bounds
    if (lo >= x && hi <= y) return bt.query(l, r);
    if (mi >= y)
        return L->rangeCount(x, y, mapLeft(l - 1) + 1,
            mapLeft(r));
    else if (mi <= x)
        return R->rangeCount(x, y, mapRight(l - 1) + 1,
            mapRight(r));
    else
        return L->rangeCount(x, mi, mapLeft(l - 1) + 1,
            mapLeft(r)) +
            R->rangeCount(mi + 1, y, mapRight(l - 1) + 1,
                mapRight(r));
}

/** Toggle i-th active state (switch ON/OFF) */
void toggle(int i) {
    if (bt.query(i, i))
        bt.update(i, -1);
    else
        bt.update(i, 1);
    if (lo != hi) {
        if (mapLeft(i) == mapLeft(i - 1) + 1)
            L->toggle(mapLeft(i));
        else
            R->toggle(mapRight(i));
    }
}

~waveletTree() {
    if (L) delete L;
    if (R) delete R;
}

#undef mapLeft
#undef mapRight
};

const int MAXN = 1e5 + 5;
int a[MAXN];

```

```
// https://www.spoj.com/problems/ILKQUERY2/
int main() {
    int n, q;
    scanf("%d%d", &n, &q);

    vector<int> all;
    for (int i = 1; i <= n; i++) {
        scanf("%d", &a[i]);
        all.push_back(a[i]);
    }
    sort(all.begin(), all.end());
    all.erase(unique(all.begin(), all.end()), all.end());
    for (int i = 1; i <= n; i++)
        a[i] = lower_bound(all.begin(), all.end(), a[i]) -
            all.begin();

    waveletTree T(a + 1, a + n + 1, 0, all.size());
    while (q--) {
        int op;
        scanf("%d", &op);
        if (op == 0) {
            int i, l, k;
            scanf("%d%d%d", &i, &l, &k);
            if (binary_search(all.begin(), all.end(), k)) {
                k = lower_bound(all.begin(), all.end(), k) -
                    all.begin();
                printf("%d\n", T.rangeCount(k, k, i + 1, l +
                    1));
            } else
                puts("0");
        } else {
            int i;
            scanf("%d", &i);
            T.toggle(i + 1);
        }
    }

    return 0;
}
```

5.7 Policy Based Set

Veja std::nth_element para um algo similar. (Não é estrut de dados)

```
4c5d7c
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

// Antes era pb_ds;
using namespace __gnu_pbds;

typedef tree<
    int, // Key type
    null_type, // Mapped type.
    // null_mapped_type for older versions
    less<int>, // Key comp function
    rb_tree_tag,
    tree_order_statistics_node_update
> ordered_set;

// iterator find_by_order(int pos)
// iterator order_of_key(Key key)
```

6 Strings

6.1 Hashing

Permite comparar strings em $O(1)$. Nunca é 100% deterministicamente correto. Sempre há risco de colisão.

$$\text{hash}(s) = \sum_{i=0}^{n-1} s[i] \cdot p^i \mod m$$

onde p, m são escolhidos. Essa é uma polynomial rolling hashing function. Geralmente escolhe-se p como um número perto do número de letras do alfabeto por ex $p = 31$. Se também tem uppercase usa $p = 53$. Pra minimizar colisão m deve ser muito pequeno. Na prática $m = 2^{64}$ não é recomendado. Uma boa escolha é um número primo grande por ex $m = 10^9 + 9$. Use a conversão $a \rightarrow 1, b \rightarrow 2, \dots, z \rightarrow 26$.

```

c2d743
long long compute_hash(string const& s) {
    const int p = 31;
    const int m = 1e9 + 9;
    long long hash_value = 0;
    long long p_pow = 1;
    for (char c : s) {
        hash_value = (hash_value + (c - 'a' + 1) * p_pow) %
            m;
        p_pow = (p_pow * p) % m;
    }
    return hash_value;
}

```

Precomputar potências de p pode ser uma boa ideia.

Problema 6.1.1: Encontre strings duplicadas em array de strings e divida-as em grupos.

Solução: Com sort normal seria $O(nm \log n)$ mas com hash fica sendo $O(nm + n \log n)$.

```

db6916
vector<vector<int>> group_identical_strings(vector<string>
    const& s) {
    int n = s.size();
    vector<pair<long long, int>> hashes(n);

```

```

    for (int i = 0; i < n; i++)
        hashes[i] = {compute_hash(s[i]), i};

    sort(hashes.begin(), hashes.end());

    vector<vector<int>> groups;
    for (int i = 0; i < n; i++) {
        if (i == 0 || hashes[i].first != hashes[i - 1].first)
            groups.emplace_back();
        groups.back().push_back(hashes[i].second);
    }
    return groups;
}

```

Problema 6.1.2: Calcule o hash de $s[i..j]$

Solução:

$$\text{hash}(s[i..j]) = (\text{hash}(s[0..j]) - \text{hash}(s[0..i-1])) \cdot p^{-i} \mod m$$

podemos precomputar os inversos pra ficar em $O(1)$. No entanto, ter os hashes multiplicados por p^i já é suficiente pra compará-los porque é só multiplicar o de menor potência por p^{j-i} .

Problema 6.1.3: Determine o número de substrings diferentes numa string

Solução: Iteramos por todas as substrings. PS: isso demora $O(n^2)$ então use outra coisa se quiser mesmo fazer isso!

```

f91b53
int count_unique_substrings(string const& s) {
    int n = s.size();

    const int p = 31;
    const int m = 1e9 + 9;
    vector<long long> p_pow(n);
    p_pow[0] = 1;
    for (int i = 1; i < n; i++)
        p_pow[i] = (p_pow[i - 1] * p) % m;

    vector<long long> h(n + 1, 0);
    for (int i = 0; i < n; i++)
        h[i + 1] = (h[i] + (s[i] - 'a' + 1) * p_pow[i]) % m;

    int cnt = 0;

```

```

for (int l = 1; l <= n; l++) {
    unordered_set<long long> hs;
    for (int i = 0; i <= n - l; i++) {
        long long cur_h = (h[i + l] + m - h[i]) % m;
        cur_h = (cur_h * p_pow[n - i - l]) % m;
        hs.insert(cur_h);
    }
    cnt += hs.size();
}
return cnt;
}

```

Para diminuir a probabilidade de colisão podemos utilizar dois hashings com módulos diferentes (por meio de um par). Por exemplo $m = 10^9 + 9$ e $m = 10^9 + 7$

6.1.1 Rabin-Karp para matching

Dadas duas strings: um texto t e um padrão s , determinar se o padrão aparece no texto e se aparecer, enumere todas as ocorrências em $O(|s| + |t|)$.

```

5e2378
vector<int> rabin_karp(string const& s, string const& t) {
    const int p = 31;
    const int m = 1e9 + 9;
    int S = s.size(), T = t.size();

    vector<long long> p_pow(max(S, T));
    p_pow[0] = 1;
    for (int i = 1; i < (int)p_pow.size(); i++)
        p_pow[i] = (p_pow[i - 1] * p) % m;

    vector<long long> h(T + 1, 0);
    for (int i = 0; i < T; i++)
        h[i + 1] = (h[i] + (t[i] - 'a' + 1) * p_pow[i]) % m;
    long long h_s = 0;
    for (int i = 0; i < S; i++)
        h_s = (h_s + (s[i] - 'a' + 1) * p_pow[i]) % m;

    vector<int> occurrences;
    for (int i = 0; i + S - 1 < T; i++) {
        long long cur_h = (h[i + S] + m - h[i]) % m;

```

```

        if (cur_h == h_s * p_pow[i] % m) occurrences.
            push_back(i);
    }
    return occurrences;
}

```

6.2 Função prefixo e KMP

Dada uma string s de tamanho n , a função prefixo π diz para cada i qual o maior prefixo próprio (diferente da própria substring) de $s[0..i]$ que também é um sufixo dessa mesma string. Mais formalmente,

$$\pi[i] = \max_{k=0 \dots i} \{k : s[0 \dots k-1] = s[i-(k-1) \dots i]\}$$

Utilizamos $\pi(0) = 0$ por convenção. É possível calculá-la em $O(n)$:

```

f6d312
vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

```

Algoritmo de KMP: Para procurar uma substring s num texto t , considere a string $s + \# + t$ onde $\#$ não aparece nem em s nem em t . Note que s aparece a partir de $t[i]$ se, e somente se, $\pi[2n+i] = n$, onde $n = \text{len}(s)$. Mas como o valor de π é limitado, podemos ir calculando on the fly e gastar apenas $O(n)$ de memória. Ou seja, temos $O(n+m)$ tempo e $O(n)$ memória.

Problema 6.2.1: Calcule o número de aparências de cada prefixo $s[0 \dots i]$ na string s .

Solução:

```

8d8e79
vector<int> ans(n + 1);

```

```

for (int i = 0; i < n; i++)
    ans[pi[i]]++;
for (int i = n - 1; i > 0; i--)
    ans[pi[i - 1]] += ans[i];
for (int i = 0; i <= n; i++)
    ans[i]++;
// Nossa resposta eh ans[i+1]

```

Problema 6.2.2: Calcule o número de ocorrências de cada prefixo $s[0 \dots i]$ em t .

Solução: Faça literalmente a mesma coisa para $s + \# + t$. Note que só estaremos interessados em $\pi[i]$ para $i \geq n + 1$.

```

bb0552
vector<int> pi = prefix_function(s + "#" + t);
int n = s.size();
int m = t.size();
vector<int> ans(n + m + 1);
for (int i = n + 1; i < n + m + 1; i++)
    ans[pi[i]]++;
for (int i = n; i > 0; i--)
    ans[pi[i - 1]] += ans[i];
// Nossa resposta eh ans[i+1]

```

Problema 6.2.3: Número de substrings diferentes numa string

Solução: Novamente, esse não é melhor jeito, levando $O(n^2)$. Seja k a quantidade atual de substrings distintas. Colocamos um caractere c no final de s . Pra atualizar k , toma $t = s + c$, reverte t , computa o valor máximo $\pi_{\max}$ de t revertida. Atualiza $k += |s| + 1 - \pi_{\max}$.

Problema 6.2.4: Dado s de tamanho n queremos a string t de menor tamanho tal que s é uma concatenação de cópias de t

Solução: Seja $k = n - \pi[n - 1]$. Se $k|n$, então k é o tamanho da resposta, finalizando o problema. Caso contrário, não há resposta.

Note que no KMP nós concatenamos duas strings e sempre dava pra saber o π em algum índice correspondente a t on the fly. Ou seja, é possível construir um automaton i.e. uma máquina de estados finitos i.e. uma tabela de transições (old_π, c) $\rightarrow$ new_π . É possível fazer isso em $O(26n)$

```

23475a
void compute_automaton(string s, vector<vector<int>>& aut) {
    s += '#';
    int n = s.size();

```

```

vector<int> pi = prefix_function(s);
aut.assign(n, vector<int>(26));
for (int i = 0; i < n; i++) {
    for (int c = 0; c < 26; c++) {
        if (i > 0 && 'a' + c != s[i])
            aut[i][c] = aut[pi[i - 1]][c];
        else
            aut[i][c] = i + ('a' + c == s[i]);
    }
}

```

Motivos para fazer isso: acelerar o tempo para calcular a prefix function da string $s + \# + t$; t é uma string gigante construída usando umas regras.

Problema 6.2.5: Dado $k \leq 10^5$ e uma string s com tamanho $\leq 10^5$, precisamos calcular o número de ocorrências de s na k -ésima string de Gray. Lembrando que $g_1 = a$, $g_2 = aba$, $g_3 = abacaba$, etc..

Solução: Construir t é impossível. Mas só precisamos saber o valor da função de prefixos no final e, pra isso, só precisamos saber o valor dela no começo (porque daí é só ir iterando). Além da automação em si, também computamos os valores $G[i][j]$ - o valor da automação depois de processar a string g_i começando com o estado j . Também computamos os valores $K[i][j]$ - o número de ocorrências de s em g_i . Na verdade $K[i][j]$ é o número de vezes que a função prefixo tomou o valor $|s|$ enquanto as operações estavam sendo feitas. Então a resposta do problema seria $K[k][0]$.

Temos $G[0][j] = j$, $K[0][j] = 0$. Lembrando que $g_i = g_{i-1} + i + g_{i-1}$, temos

$$\text{mid} = \text{aut}[G[i-1][j]][i]$$

$$G[i][j] = G[i-1][\text{mid}]$$

$$K[i][j] = K[i-1][j] + (\text{mid} == |s|) + K[i-1][\text{mid}]$$

Problema 6.2.6: É dada s e alguns padrões t_i , sendo algum especificado assim: é uma string de caracteres normais e aí pode ter umas inserções recursivas em strings anteriores da forma t_k^{cnt} o que significa que nesse lugar devemos inserir a string t_k uma quantidade cnt de vezes. Por exemplo $t_1 = \text{"abdeca"}$, $t_2 = \text{"abc"} + t_1^{30} + \text{"abd"}$, $t_3 = t_2^{50} + t_1^{100}$ e $t_4 = t_2^{10} + t_3^{100}$.

Solução: É só fazer que nem eu disse no problema anterior.

6.3 Função Z

Seja s uma string de tamanho n . $z[i]$ é o tamanho da maior string que começa em $s[i]$ e é um prefixo de s . Escolhemos $z[0] = 0$.

9a2512

```
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

Seja t uma string de texto e p um padrão;

Problema 6.3.1: Ache todas as ocorrências de p no texto t

Solução: Criamos a nova string $s = p + \# + t$. Computamos a função Z para s . Para cada i de 0 ate $\text{length}(t) - 1$, consideramos o valor de $k = z[i + \text{length}(p) + 1]$. Se $k == \text{length}(p)$, então sabemos que tem uma ocorrência de p na posição i de t . Com isso, o tempo e o consumo de memória são ambos $O(\text{length}(t) + \text{length}(p))$.

Problema 6.3.2: Conte o número de substrings distintas de s .

Solução: Seja k o número atual. Adicionamos c a s . Tome a string $t = s + c$ e reverta. Computa a função Z de t e acha o maior valor $z_{\max}$. O prefixo de t de tamanho $z_{\max}$ ocorre em algum lugar no meio de t também. Prefixos menores também ocorrem.

Então o número de novas substrings é $\text{length}(t) - z_{\max}$, de modo que o tempo seja $O(n^2)$.

Note que também podemos recalculer o número de substrings quando colocamos um caractere no início da string em $O(n)$.

Problema 6.3.3: Dada uma string s de tamanho n , encontre a string t de menor comprimento tal que s possa ser representada de uma ou mais cópias de t .

Solução: Computa a função Z de s , loopa pelos i tais que $i|n$ e para no primeiro i tal que $i + z[i] = n$. Então, a string s pode ser comprimida para o tamanho i .

6.4 Suffix Array

Seja s uma string de tamanho n . O i -ésimo sufixo de s é a substring $s[i \dots n-1]$. Uma array de sufixos contém inteiros que representam os índices iniciais de todos os sufixos de uma determinada string após os sufixos serem ordenados. Por exemplo, a suffix array de $s = abaab$ é $(2, 3, 0, 4, 1)$.

Vamos primeiro fazer um algoritmo para ordenar os shifts cíclicos de uma string. A partir dele, se ordenarmos os shifts cíclicos de uma string s concatenada a um caractere estranho $\$,$ teremos a nossa suffix array em $O(n \log n)$.

fd3b6c

```
vector<int> sort_cyclic_shifts(string const& s) {
    int n = s.size();
    const int alphabet = 256;
    vector<int> p(n), c(n), cnt(max(alphabet, n), 0);
    for (int i = 0; i < n; i++)
        cnt[s[i]]++;
    for (int i = 1; i < alphabet; i++)
        cnt[i] += cnt[i - 1];
    for (int i = 0; i < n; i++)
        p[--cnt[s[i]]] = i;
    c[p[0]] = 0;
    int classes = 1;
    for (int i = 1; i < n; i++) {
        if (s[p[i]] != s[p[i - 1]]) classes++;
        c[p[i]] = classes - 1;
    }
    vector<int> pn(n), cn(n);
    for (int h = 0; (1 << h) < n; ++h) {
        for (int i = 0; i < n; i++) {
            pn[i] = p[i] - (1 << h);
            if (pn[i] < 0) pn[i] += n;
        }
        fill(cnt.begin(), cnt.begin() + classes, 0);
        for (int i = 0; i < n; i++)
            cnt[c[pn[i]]]++;
        for (int i = 1; i < classes; i++)
            cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; i--)
            p[--cnt[c[pn[i]]]] = pn[i];
        cn[p[0]] = 0;
    }
}
```

```

    classes = 1;
    for (int i = 1; i < n; i++) {
        pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
        pair<int, int> prev = {c[p[i - 1]], c[(p[i - 1] + (1 << h)) % n]};
        if (cur != prev) ++classes;
        cn[p[i]] = classes - 1;
    }
    c.swap(cn);
}
return p;
}

```

Isso requer tempo $O(n \log n)$ e $O(n)$ de memória.

A partir dele:

```

3412a8
vector<int> suffix_array_construction(string s) {
    s += "$";
    vector<int> sorted_shifts = sort_cyclic_shifts(s);
    sorted_shifts.erase(sorted_shifts.begin());
    return sorted_shifts;
}

```

Problema 6.4.1: Encontre o menor cyclic shift.

Solução: O algoritmo colocado ordena todos os shifts cíclicos e então $p[0]$ dá a posição do menor shift cíclico.

Problema 6.4.2: Ache uma string s dentro de um texto t online. Sabemos o texto t de antemão mas não a string s .

Solução: Fazemos a suffix array para o texto t em $O(|t| \log |t|)$. Certamente s deve ser um prefixo de um sufixo de t . Então basta fazer uma busca binária por s em p . Com isso, demoramos $O(|s| \log |t|)$. Se ocorrer mais vezes, então vão tar próximas, então faz uma segunda busca binária e acha o número de ocorrências dela.

Problema 6.4.3: Queremos saber se uma substring de s é menor que outra em $O(1)$

Solução: Depois de construir em $O(|s| \log |s|)$ a suffix array, podemos fazer em $O(1)$:

```

bc2e11
int compare(int i, int j, int l, int k) {

```

```

    pair<int, int> a = {c[k][i], c[k][(i + l - (1 << k)) % n]};
    pair<int, int> b = {c[k][j], c[k][(j + l - (1 << k)) % n]};
    return a == b ? 0 : a < b ? -1 : 1;
}

```

Problema 6.4.4: Dada uma string s , queremos o longest common prefix de dois sufixos que iniciam nas posições i e j . (aqui fazemos com $O(|s| \log |s|)$ de memória inicial, mas no problema seguinte fazemos sem).

Solução: Temos que nos lembrar das arrays $c[]$ de cada iteração e daí vem a memória adicional.

```

d6414a
// log_n e o log(n) na base 2 arredondado pra baixo
int lcp(int i, int j) {
    int ans = 0;
    for (int k = log_n; k >= 0; k--) {
        if (c[k][i % n] == c[k][j % n]) {
            ans += 1 << k;
            i += 1 << k;
            j += 1 << k;
        }
    }
    return ans;
}

```

Problema 6.4.5: Mesma coisa só que sem memória adicional.

Solução: Construa a array $lcp[0 \dots n - 2]$ onde $lcp[i]$ é igual ao tamanho do longest common prefix dos sufixos que iniciam em $p[i]$ e $p[i + 1]$. Essa array dirá para nós uma resposta para quaisquer dois sufixos adjacentes dessa string. Suponha que seja o pedido de computar o LCP dos sufixos $p[i]$ e $p[j]$. A resposta será $\min(lcp[i], \dots, lcp[j - 1])$. Ou seja será uma range minimum query que tem uma variedade de soluções. Para construir a lcp podemos usar o algoritmo de Kasai

```

334b33
vector<int> lcp_construction(string const& s, vector<int>
const& p) {
    int n = s.size();
    vector<int> rank(n, 0);
    for (int i = 0; i < n; i++)
        rank[p[i]] = i;

```

```

int k = 0;
vector<int> lcp(n - 1, 0);
for (int i = 0; i < n; i++) {
    if (rank[i] == n - 1) {
        k = 0;
        continue;
    }
    int j = p[rank[i] + 1];
    while (i + k < n && j + k < n && s[i + k] == s[j + k])
        k++;
    lcp[rank[i]] = k;
    if (k) k--;
}
return lcp;
}

```

Problema 6.4.6: Número de substrings distintas

Solução: Pensamos em quais novas substrings começam na posição $p[0]$ depois em $p[1]$ etc.. Tomamos os sufixos ordenadamente e vemos quais prefixos dão novas substrings. Para cada sufixo atual $p[i]$ teremos novas substrings para todos seus prefixos, exceto para os prefixos que coincidirem com o sufixo $p[i - 1]$. Então pegamos todos os prefixos exceto os $lcp[i - 1]$ primeiros. Então a resposta final é:

$$\sum_{i=0}^{n-1} (n - p[i]) - \sum_{i=0}^{n-2} lcp[i] = \frac{n^2 + n}{2} - \sum_{i=0}^{n-2} lcp[i]$$

6.5 Algoritmo de Aho-Corasick e Tries

O algoritmo de Aho-Corasick permite uma pesquisa rápida por múltiplos padrões num texto. O conjunto de strings que são padrões é chamado de dicionário. Denota-se o tamanho total das strings constituintes por m e o tamanho do alfabeto por k . O algoritmo constrói um autômato de estados finitos baseado em um trie em tempo $O(mk)$ e usa ele para processar o texto.

Um trie é uma árvore enraizada em que cada aresta é rotulada com alguma letra.

```
const int K = 26;
```

fe8a0c

```

struct Vertex {
    int next[K];
    bool output = false;

    Vertex() { fill(begin(next), end(next), -1); }
};

vector<Vertex> trie(1);

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (trie[v].next[c] == -1) {
            trie[v].next[c] = trie.size();
            trie.emplace_back();
        }
        v = trie[v].next[c];
    }
    trie[v].output = true;
}

```

É possível trocar de array para mapa para reduzir o consumo de memória para $O(m)$, mas isso aumenta a complexidade para $O(m \log k)$.

Construímos o autômato de estados finitos da seguinte forma: Se estamos no vértice correspondente a uma string t e queremos ir para um estado diferente usando o caractere c , então veja se tem uma aresta com esse caractere. Se tiver, vá para o vértice apontado. Se não tiver, achamos a maior string do trie que é um sufixo próprio da string t e a partir daí tenta fazer a transição. Com isso queremos manter a invariante de que o estado atual é o maior match parcial na string processada.

Um suffix link para um vértice p é uma aresta que aponta para o maior sufixo próprio da string correspondendo ao vértice p . O único caso especial é a raiz da árvore, cujo suffix link apontará para ela mesma. Agora reformulamos o statement sobre as transições na automação assim: enquanto não tiver transição do vértice atual da trie usando a letra atual, seguimos o suffix link.

a41621

```

const int K = 26;

struct Vertex {
    int next[K];

```

```

bool output = false;
int p = -1;
char pch;
int link = -1;
int go[K];

Vertex(int p = -1, char ch = '$') : p(p), pch(ch) {
    fill(begin(next), end(next), -1);
    fill(begin(go), end(go), -1);
}

};

vector<Vertex> t(1);

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch);
        }
        v = t[v].next[c];
    }
    t[v].output = true;
}

int go(int v, char ch);

int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0)
            t[v].link = 0;
        else
            t[v].link = go(get_link(t[v].p), t[v].pch);
    }
    return t[v].link;
}

int go(int v, char ch) {

```

```

    int c = ch - 'a';
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1)
            t[v].go[c] = t[v].next[c];
        else
            t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch);
    }
    return t[v].go[c];
}

```

O tempo total de achar todos os suffix links e transições será linear.

Também é possível fazer uma construção baseada em BFS (bottom-up a partir da raiz) ao invés de ficar computando transições e suffix links com chamadas recursivas. Isso tem algumas vantagens: ao invés de tamanho total m , o running time depende apenas no número de vértices n na trie. Além disso, é possível adaptar para alfabetos grandes usando uma estrutura de dados de array persistente, tornando o tempo de construção $O(n \log k)$ ao invés de $O(mk)$ (lembrando que m pode ir até n^2).

Podemos pensar indutivamente usando o fato de que a BFS a partir da raiz atravessa vértices na ordem de increasing length. Podemos assumir que quando estamos num vértice v , seu suffix link $u = \text{link}[v]$ já foi computado com sucesso e para todos os vértices com transições de length menores a partir deles também já foram computados.

Assuma que no momento que estamos em um vértice v e consideramos um caractere c , essencialmente temos dois casos

1. $go[v][c] = -1$. Nesse caso, designamos $go[v][c] = go[u][c]$, que já é conhecido pela hipótese de indução
2. $go[v][c] = w \neq -1$. Nesse caso, designamos $link[w] = go[u][c]$

Dessa maneira, gastamos $O(1)$ de tempo para cada par vértice/caractere, tornando o running time $O(mk)$. Mas nós copiamos aqui várias transições a partir de u no primeiro caso, enquanto que transições do segundo caso formam a trie e somam up to m over all vertices. Para evitar a cópia de $go[u][c]$, podemos usar uma estrutura de dados persistente, com a qual copiamos $go[u]$ para $go[v]$ e aí atualizamos os valores para caracteres em que a transição difere. Isso leva a um algoritmo $O(m \log k)$.

Problema 6.5.1: Encontre (printe) todas as strings de um conjunto em um texto em $O(\text{len} + \text{ans})$ onde len é o tamanho do texto e ans é o tamanho das strings printadas.

Solução: Construímos um autômato para esse conjunto de strings. Agora processamos cada letra do texto usando o autômato, começando pela raiz. Se estamos num estado v e vamos processar agora um caractere c fazemos $go(v, c)$. Como encontrar para um estado v se tem algum match com strings do conjunto?

Guardamos cada output vertex no índice da string correspondente a ele (ou então listas de índices caso strings duplicatas apareçam no conjunto). Daí encontramos em $O(n)$ os índices de todas as strings que dão match no estado atual simplesmente seguindo os suffix links a partir do vértice atual na raiz. Isso resulta em $O(nlen)$, o que não é muito bom. Dá pra otimizar computando e guardando o vértice output mais próximo que é alcançável utilizando suffix links. Isso pode ser feito linearmente de forma preguiçosa. Então pra cada vértice podemos avançar em $O(1)$ tempo para o próximo vértice marcado no caminho do suffix link i.e. pro próximo match. Então pra cada match gastamos $O(1)$ de maneira que fiquemos com $O(len + ans)$.

Problema 6.5.2: Encontrando a menor string (lexicograficamente) de um dado tamanho que não dá match em nenhuma das strings dadas.

Solução: Construa um autômato para o conjunto de strings. Como os vértices de output são os estados em que temos um match com uma string do conjunto, temos que evitar esses estados já que temos que evitar matches. Então deletamos todos os vértices ruins das máquinas e no grafo remanescente do autômato encontramos o menor caminho lexicograficamente de tamanho L . Fazemos isso em $O(L)$ com uma dfs.

Problema 6.5.3: Encontrando a menor string contendo todas as strings dadas.

Solução: Usa a mesma ideia. Para cada vértice, guarda uma máscara que denota as strings que dão match nesse estado. Então o problema pode ser reformulado assim: inicialmente, estando no estado ($v = root, mask = 0$), queremos chegar no estado ($v, mask = 2^n - 1$), em que n é o número de strings do conjunto. Vamos de um estado para outro usando uma letra e damos update na máscara de acordo. Usando uma BFS podemos encontrar qualquer caminho para esse estado com o menor tamanho.

Problema 6.5.4: Encontrando a menor string lexicograficamente de tamanho L contendo k strings.

Solução: Parecido com o outro problema. Mas agora o estado atual é determinado por uma tripla (v, len, cnt) e queremos ir do estado ($root, 0, 0$) para o estado (v, L, k), onde v pode ser qualquer vértice. Então podemos encontrar esse caminho usando dfs.

6.6 Algoritmo de Ukkonen

Constrói uma árvore de sufixos para uma string s de tamanho n em $O(n \log k)$ em que k é o tamanho do alfabeto.

A função `build_tree` constrói a árvore de sufixos. Ela é guardada num array de estruturas `node` onde `node[0]` é a raiz.

bb84ae

```
string s;
int n;

struct node {
    int l, r, par, link;
    map<char, int> next;

    node(int l = 0, int r = 0, int par = -1) : l(l), r(r),
        par(par), link(-1) {}
    int len() { return r - l; }
    int& get(char c) {
        if (!next.count(c)) next[c] = -1;
        return next[c];
    }
};
node t[MAXN];
int sz;

struct state {
    int v, pos;
    state(int v, int pos) : v(v), pos(pos) {}
};
state ptr(0, 0);

state go(state st, int l, int r) {
    while (l < r)
        if (st.pos == t[st.v].len()) {
            st = state(t[st.v].get(s[l]), 0);
            if (st.v == -1) return st;
        } else {
            if (s[t[st.v].l + st.pos] != s[l]) return state
                (-1, -1);
            if (r - l < t[st.v].len() - st.pos) return state
```

```

        (st.v, st.pos + r - 1);
        l += t[st.v].len() - st.pos;
        st.pos = t[st.v].len();
    }
    return st;
}

int split(state st) {
    if (st.pos == t[st.v].len()) return st.v;
    if (st.pos == 0) return t[st.v].par;
    node v = t[st.v];
    int id = sz++;
    t[id] = node(v.l, v.l + st.pos, v.par);
    t[v.par].get(s[v.l]) = id;
    t[id].get(s[v.l + st.pos]) = st.v;
    t[st.v].par = id;
    t[st.v].l += st.pos;
    return id;
}

int get_link(int v) {
    if (t[v].link != -1) return t[v].link;
    if (t[v].par == -1) return 0;
    int to = get_link(t[v].par);
    return t[v].link = split(go(state(to, t[to].len()), t[v].l + (t[v].par == 0), t[v].r));
}

void tree_extend(int pos) {
    for (;;) {
        state nptr = go(ptr, pos, pos + 1);
        if (nptr.v != -1) {
            ptr = nptr;
            return;
        }

        int mid = split(ptr);
        int leaf = sz++;
        t[leaf] = node(pos, n, mid);
        t[mid].get(s[pos]) = leaf;
    }
}

```

```

        ptr.v = get_link(mid);
        ptr.pos = t[ptr.v].len();
        if (!mid) break;
    }
}

void build_tree() {
    sz = 1;
    for (int i = 0; i < n; ++i)
        tree_extend(i);
}

```

Tudo comprimido:

```

b43022
const int N = 1000000, // maximum possible number of nodes
in suffix tree
    INF = 1000000000; // infinity constant
string a; // input string for which the suffix
tree is being built
int t[N][26], // array of transitions (state,
letter)
    l[N], // left...
    r[N], // ...and right boundaries of the substring of a
which correspond to incoming edge
    p[N], // parent of the node
    s[N], // suffix link
    tv, // the node of the current suffix (if we're mid-
edge, the lower node of the edge)
    tp, // position in the string which corresponds to the
position on the edge (between l[tv] and
// r[tv], inclusive)
    ts, // the number of nodes
    la; // the current character in the string

void ukkadd(int c) { // add character s to the tree
suff:: // we'll return here after each transition to the
suffix (and will add character again)
    if (r[tv] < tp) { // check whether we're still within
the boundaries of the current edge

```

```

    // if we're not, find the next edge. If it doesn't
    // exist, create a leaf and add it to the
    // tree
    if (t[tv][c] == -1) {
        t[tv][c] = ts;
        l[ts] = la;
        p[ts++] = tv;
        tv = s[tv];
        tp = r[tv] + 1;
        goto suff;
    }
    tv = t[tv][c];
    tp = l[tv];
} // otherwise just proceed to the next edge
if (tp == -1 || c == a[tp] - 'a')
    tp++; // if the letter on the edge equal c, go down
        that edge
else {
    // otherwise split the edge in two with middle in
    // node ts
    l[ts] = l[tv];
    r[ts] = tp - 1;
    p[ts] = p[tv];
    t[ts][a[tp] - 'a'] = tv;
    // add leaf ts+1. It corresponds to transition
    // through c.
    t[ts][c] = ts + 1;
    l[ts + 1] = la;
    p[ts + 1] = ts;
    // update info for the current node - remember to
    // mark ts as parent of tv
    l[tv] = tp;
    p[tv] = ts;
    t[p[ts]][a[l[ts]] - 'a'] = ts;
    ts += 2;
    // prepare for descent
    // tp will mark where are we in the current suffix
    tv = s[p[ts - 2]];
    tp = l[ts - 2];
    // while the current suffix is not over, descend

```

```

        while (tp <= r[ts - 2]) {
            tv = t[tv][a[tp] - 'a'];
            tp += r[tv] - l[tv] + 1;
        }
    // if we're in a node, add a suffix link to it,
    // otherwise add the link to ts
    // (we'll create ts on next iteration).
    if (tp == r[ts - 2] + 1)
        s[ts - 2] = tv;
    else
        s[ts - 2] = ts;
    // add tp to the new edge and return to add letter
    // to suffix
    tp = r[tv] - (tp - r[ts - 2]) + 2;
    goto suff;
}

void build() {
    ts = 2;
    tv = 0;
    tp = 0;
    fill(r, r + N, (int)a.size() - 1);
    // initialize data for the root of the tree
    s[0] = 1;
    l[0] = -1;
    r[0] = -1;
    l[1] = -1;
    r[1] = -1;
    memset(t, -1, sizeof t);
    fill(t[1], t[1] + 26, 0);
    // add the text to the tree, letter by letter
    for (la = 0; la < (int)a.size(); ++la)
        ukkadd(a[la] - 'a');
}

```

6.7 Autômato de sufixos

6.8 Fatoração de Lyndon

Uma string é dita simples (ou uma palavra de Lyndon) se for estritamente menor que qualquer um de seus sufixos não triviais. Pode ser demonstrado que uma string é simples se e só se for estritamente menor que todos seus shifts cíclicos não triviais.

Seja dada uma string s . A fatoração de Lyndon de s é uma fatoração $s = w_1 w_2 \dots w_k$ em que todas as strings w_i são simples e $w_1 \geq w_2 \geq \dots \geq w_k$. Pode ser demonstrado que para toda string uma fatoração existe e é única.

O algoritmo de Duval constrói a fatoração de Lyndon em tempo $O(n)$ usando $O(1)$ de memória adicional.

```

0e6ce6
vector<string> duval(string const& s) {
    int n = s.size();
    int i = 0;
    vector<string> factorization;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                k++;
            j++;
        }
        while (i <= k) {
            factorization.push_back(s.substr(i, j - k));
            i += j - k;
        }
    }
    return factorization;
}

```

Problema 6.8.1: Encontrando o menor shift cíclico.

Solução: Seja s a string. Construímos a fatoração de Lyndon da string s em tempo $O(n)$. Então basta pegar uma string simples da fatoração que começa numa posição menor que n e termina numa posição $\geq n$. A posição inicial dessa string simples vai ser o início do menor shift cíclico.

```

2b037c
string min_cyclic_string(string s) {

```

```

s += s;
int n = s.size();
int i = 0, ans = 0;
while (i < n / 2) {
    ans = i;
    int j = i + 1, k = i;
    while (j < n && s[k] <= s[j]) {
        if (s[k] < s[j])
            k = i;
        else
            k++;
        j++;
    }
    while (i <= k)
        i += j - k;
}
return s.substr(ans, n / 2);
}

```

6.9 Extras

6.9.1 Parsing

Nesse primeiro código, assumimos que os operadores são binários e left-associative. Parêntesis são permitidos. Temos uma stack para números e outra para operadores e parêntesis.

```

d0a353
bool delim(char c) { return c == ' '; }

bool is_op(char c) { return c == '+' || c == '-' || c == '*' ||
    || c == '/'; }

int priority(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return -1;
}

void process_op(stack<int>& st, char op) {
    int r = st.top();

```



```

    st.pop();
    int l = st.top();
    st.pop();
    switch (op) {
    case '+':
        st.push(l + r);
        break;
    case '-':
        st.push(l - r);
        break;
    case '*':
        st.push(l * r);
        break;
    case '/':
        st.push(l / r);
        break;
    }
}

int evaluate(string& s) {
    stack<int> st;
    stack<char> op;
    for (int i = 0; i < (int)s.size(); i++) {
        if (delim(s[i])) continue;

        if (s[i] == '(') {
            op.push('(');
        } else if (s[i] == ')') {
            while (op.top() != '(') {
                process_op(st, op.top());
                op.pop();
            }
            op.pop();
        } else if (is_op(s[i])) {
            char cur_op = s[i];
            while (!op.empty() && priority(op.top()) >=
                priority(cur_op)) {
                process_op(st, op.top());
                op.pop();
            }
        }
    }
}

```

```

        op.push(cur_op);
    } else {
        int number = 0;
        while (i < (int)s.size() && isalnum(s[i]))
            number = number * 10 + s[i++] - '0';
        --i;
        st.push(number);
    }
}

while (!op.empty()) {
    process_op(st, op.top());
    op.pop();
}
return st.top();
}

```

Caso tenhamos associatividade pela direita por ex $a**b**c$ dever ser percebido como $a**(b**c)$ então precisamos substituir a linha

e3bb46

```
while (!op.empty() && priority(op.top()) >= priority(cur_op))
    )
```

por

3666e5

```
while (!op.empty() && ((left_assoc(cur_op) && priority(op.
    top()) >= priority(cur_op)) ||
    (!left_assoc(cur_op) && priority(op.
    top()) > priority(cur_op))))
```

Também pode ser que existam unary operators. Por exemplo o unary plus e unary minus (nesse caso precisamos decidir se é binário ou unário). Note também que alguns unary operators são right associative. Aqui uma implementação para os 4 binary operators e unary operators + e -:

73764e

```
bool delim(char c) { return c == ' '; }

bool is_op(char c) { return c == '+' || c == '-' || c == '*'
    || c == '/'; }

```

```

bool is_unary(char c) { return c == '+' || c == '-'; }

int priority(char op) {
    if (op < 0) // unary operator
        return 3;
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return -1;
}

void process_op(stack<int>& st, char op) {
    if (op < 0) {
        int l = st.top();
        st.pop();
        switch (-op) {
            case '+':
                st.push(l);
                break;
            case '-':
                st.push(-l);
                break;
        }
    } else {
        int r = st.top();
        st.pop();
        int l = st.top();
        st.pop();
        switch (op) {
            case '+':
                st.push(l + r);
                break;
            case '-':
                st.push(l - r);
                break;
            case '*':
                st.push(l * r);
                break;
            case '/':
                st.push(l / r);

```

```

                break;
            }
        }
    }

int evaluate(string& s) {
    stack<int> st;
    stack<char> op;
    bool may_be_unary = true;
    for (int i = 0; i < (int)s.size(); i++) {
        if (delim(s[i])) continue;

        if (s[i] == '(') {
            op.push('(');
            may_be_unary = true;
        } else if (s[i] == ')') {
            while (op.top() != '(') {
                process_op(st, op.top());
                op.pop();
            }
            op.pop();
            may_be_unary = false;
        } else if (is_op(s[i])) {
            char cur_op = s[i];
            if (may_be_unary && is_unary(cur_op)) cur_op = -cur_op;
            while (!op.empty() && ((cur_op >= 0 && priority(
                op.top()) >= priority(cur_op)) ||
                (cur_op < 0 && priority(
                    op.top()) > priority(
                        cur_op)))) {
                process_op(st, op.top());
                op.pop();
            }
            op.push(cur_op);
            may_be_unary = true;
        } else {
            int number = 0;
            while (i < (int)s.size() && isalnum(s[i]))
                number = number * 10 + s[i++] - '0';

```

```

        —i;
        st.push(number);
        may_be_unary = false;
    }
}

while (!op.empty()) {
    process_op(st, op.top());
    op.pop();
}
return st.top();
}

```

6.9.2 Algoritmo de Manacher

Dada uma string s de tamanho n , queremos achar todos os pares (i, j) tais que $s[i \dots j]$ é um palíndromo. A princípio isso pode parecer impossível, já que é possível que existam $O(n^2)$ substrings palindrômicas. Entretanto, se nós mantivermos eles de maneira compacta, isso é resolvido: para cada posição i , encontramos o número de palíndromos não vazios centrados nessa posição. Isso se aproveita do fato de que se temos um palíndromo de tamanho l centrado em i também temos de tamanhos $l-2$, $l-4$, etc.. Para palíndromos pares assumimos que estão centrados em $s[i]$ e $s[i-1]$. Trabalharemos, então, com as arrays $d_{odd}[i]$ e $d_{even}[i]$. Por exemplo, se $s = abababc$, $d_{odd}[3] = 3$. Se $s = cbaabd$, $d_{even}[3] = 2$.

Dá pra resolver com string hashing em $O(n \log n)$ e com árvores de sufixos e fast LCA em $O(n)$, mas esse método daqui é mais simples e tem menos constante escondida.

Aqui a implementação para palíndromos de tamanho ímpar:

```

2b3cab
vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "^";
    vector<int> p(n + 2);
    int l = 1, r = 1;
    for (int i = 1; i <= n; i++) {
        p[i] = max(0, min(r - i, p[l + (r - i)]));
        while (s[i - p[i]] == s[i + p[i]]) {
            p[i]++;
        }
        if (i + p[i] > r) {

```

```

        l = i - p[i], r = i + p[i];
    }
}
return vector<int>(begin(p) + 1, end(p) - 1);
}

```

A implementação para o caso par é mais difícil e é bem fácil errar alguma coisa. O que podemos fazer é reduzir o problema todo pra o caso em que temos que lidar com palíndromos de tamanho ímpar. Para tanto, adicionamos símbolos # entre as letras e no final e no início da string. Com isso, teremos um único array d . Em particular, se $s = abcbcb$, estaremos lidando com a string $\#a\#b\#c\#b\#c\#a\#$ e a partir dela obteremos uma array $d = [1, 2, 1, 2, 1, 4, 1, 8, 1, 4, 1, 2, 1, 2, 1]$. A partir dela, teremos que $d[2i] = 2d_{even}[i] + 1$ e $d[2i + 1] = 2d_{odd}[i]$. Sendo mais preciso, podemos nos aproveitar da implementação anterior e utilizar a seguinte:

```

4b2c82
vector<int> manacher(string s) {
    string t;
    for (auto c : s) {
        t += string("#") + c;
    }
    auto res = manacher_odd(t + "#");
    return vector<int>(begin(res) + 1, end(res) - 1);
}

```

6.9.3 Encontrando repetições

Uma repetição é duas ocorrências de uma string consecutivas, isto é, é um par (i, j) tal que $s[i \dots j]$ consiste de duas strings concatenadas. O desafio é encontrar todas as repetições de uma dada string. Também podemos nos interessar por encontrar qualquer repetição ou encontrar a repetição mais longa.

Podem existir $O(n^2)$ repetições, mas isso não impede de calcular o número delas em $O(n \log n)$, pois o algoritmo dá às repetições uma forma comprimida em grupos.

Agluns resultados interessantes acerca do número de repetições:

1. O número de repetições primitivas (aquelas nas quais as metades não são repetições) é, no máximo, $O(n \log n)$
2. Se encodarmos repetições como tuplas de números (i, p, r) , em que i é a posição de início, p é o tamanho da string que se repete e r o número de repetições, então todas as repetições podem ser descritas com uma quantidade $O(n \log n)$ dessas triplas.

3. Strings de fibonacci definidas por $t_0 = a$, $t_1 = b$, $t_i = t_{i-1} + t_{i-2}$ são fortemente periódicas. O número de repetições na string de fibonacci f_i é $O(f_n \log f_n)$.

Algoritmo de Main-Lorentz: Encontra todas as tuplas de tamanho quatro $(cntr, l, k_1, k_2)$, em que $cntr$ é o último índice da primeira string da repetição em questão, l é o tamanho da repetição e k_1 e k_2 são relacionadas ao funcionamento interno do algoritmo.

Se você quiser só encontrar o número de repetições na string ou só quer achar a maior repetição da string, essa informação sobre as tuplas vai ser suficiente e o runtime ainda será $O(n \log n)$.

Note que se você expandir as tuplas para pegar as posições inicial e final de cada repetição, então o runtime será $O(n^2)$ (afinal, podem existir $O(n^2)$ repetições). Nessa implementação, guardamos todas as repetições encontradas num vector de pair e pegamos os pairs de início e fim.

```

2a846d
vector<int> z_function(string const& s) {
    int n = s.size();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}

int get_z(vector<int> const& z, int i) {
    if (0 <= i && i < (int)z.size())
        return z[i];
    else
        return 0;
}

vector<pair<int, int>> repetitions;

void convert_to_repetitions(int shift, bool left, int cntr,

```

```

int l, int k1, int k2) {
    for (int l1 = max(1, l - k2); l1 <= min(l, k1); l1++) {
        if (left && l1 == 1) break;
        int l2 = l - l1;
        int pos = shift + (left ? cntr - l1 : cntr - l - l1 + 1);
        repetitions.emplace_back(pos, pos + 2 * l - 1);
    }
}

void find_repetitions(string s, int shift = 0) {
    int n = s.size();
    if (n == 1) return;

    int nu = n / 2;
    int nv = n - nu;
    string u = s.substr(0, nu);
    string v = s.substr(nu);
    string ru(u.rbegin(), u.rend());
    string rv(v.rbegin(), v.rend());

    find_repetitions(u, shift);
    find_repetitions(v, shift + nu);

    vector<int> z1 = z_function(ru);
    vector<int> z2 = z_function(v + '#' + u);
    vector<int> z3 = z_function(ru + '#' + rv);
    vector<int> z4 = z_function(v);

    for (int cntr = 0; cntr < n; cntr++) {
        int l, k1, k2;
        if (cntr < nu) {
            l = nu - cntr;
            k1 = get_z(z1, nu - cntr);
            k2 = get_z(z2, nv + 1 + cntr);
        } else {
            l = cntr - nu + 1;
            k1 = get_z(z3, nu + 1 + nv - 1 - (cntr - nu));
            k2 = get_z(z4, (cntr - nu) + 1);
        }
    }
}

```

```
    if (k1 + k2 >= 1) convert_to_repetitions(shift, cntr  
        < nu, cntr, l, k1, k2);  
    }  
}
```

7 Álgebra e Teoria dos Números

7.1 Exponenciação binária

```
e06839
long long binpow(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1) res = res * a;
        a = a * a;
        b >>= 1;
    }
    return res;
}
```

Aplicações: Calcular $x^n \bmod m$, usar exponenciação de matrizes para calcular F_n , aplicar uma permutação k vezes, aplicar uma transformação linear num conjunto de pontos, tamanho de caminhos de tamanho k num grafo (elevando a matriz de adjacências), etc..

7.2 MDC e MMC

Ambos em $O(\log(\min(a, b)))$

```
e34a97
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int lcm(int a, int b) { return a / gcd(a, b) * b; }

int iterative_gcd(int a, int b) {
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}

// Nao diminui complexidade mas acelera msm assim
// Nunca vi um problema que precisasse disso
```

```
int fast_gcd(int a, int b) {
    if (!a || !b) return a | b;
    unsigned shift = __builtin_ctz(a | b);
    a >>= __builtin_ctz(a);
    do {
        b >>= __builtin_ctz(b);
        if (a > b) swap(a, b);
        b -= a;
    } while (b);
    return a << shift;
}
```

7.3 Algoritmo de Euclides Estendido

Dados $a, b \in \mathbb{Z}$, acha $x, y \in \mathbb{Z}$ tais que

$$ax + by = \gcd(a, b)$$

em $O(\log(\min(a, b)))$. Também produz resultados corretos para negativos.

Os x e y achados por esse algoritmo são ambos os menores possíveis (em módulo). Em particular, você tem a garantia de que $|x| \leq b/2$ e $|y| \leq a/2$. Além disso, não tem risco de overflow no meio dele.

Esse algoritmo deu AC num problema que pedia x e y com $|x| + |y|$ mínimo e $x \leq y$ (se tivesse empate) com $a, b \leq 10^9 + 1$.

```
e91de3
int iterative_gcd(int a, int b, int& x, int& y) {
    x = 1, y = 0;
    int x1 = 0, y1 = 1, a1 = a, b1 = b;
    while (b1) {
        int q = a1 / b1;
        tie(x, x1) = make_tuple(x1, x - q * x1);
        tie(y, y1) = make_tuple(y1, y - q * y1);
        tie(a1, b1) = make_tuple(b1, a1 - q * b1);
    }
    return a1;
}

int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
    }
    return gcd(b, a % b, x, y);
}
```

```

    y = 0;
    return a;
}
int x1, y1;
int d = gcd(b, a % b, x1, y1);
x = y1;
y = x1 - y1 * (a / b);
return d;
}

```

Problema 7.3.1: Dados $a, m \in \mathbb{Z}$ com $\gcd(a, m) = 1$, achar $b \in \mathbb{Z}_m$ tal que $a \cdot b \equiv 1 \pmod{m}$ em $O(\log(\min(a, m)))$

Solução: Aplicar o algoritmo estendido de euclides em a e m de modo a achar x, y tais que $ax + my = 1$. O que você procura é $(x \% m + m) \% m$.

7.4 Equações Diofantinas Lineares

Sejam $a, b, c \in \mathbb{Z}$. Queremos investigar soluções com $x, y \in \mathbb{Z}$ de

$$ax + by = c$$

o caso $a = 0$ ou $b = 0$ é trivial. **Assumiremos**, então, $a, b \neq 0$.

Seja x_0, y_0 uma solução qualquer. As soluções gerais são dadas por $x = x_0 + k \cdot \frac{b}{g}$ e $y = y_0 - k \cdot \frac{a}{g}$.

Problema 7.4.1: Ache qualquer solução

Solução:

```

24ec15
bool find_any_solution(int a, int b, int c, int& x0, int& y0,
    int& g) {
    g = gcd(abs(a), abs(b), x0, y0); // euclides ext
    if (c % g) {
        return false;
    }
    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}

```

Problema 7.4.2: Ache todas as soluções para $\min_x \leq x \leq \max_x$ e $\min_y \leq y \leq \max_y$.

Solução:

```

526c6d
// passei num problema em que x, y, a, b, c tinham modulo <=
// 10^8
// precisava mudar de int pra lli os x, os y, o gcd e fazer
// que nem botei no comentario pro shift
void shift_solution(int& x, int& y, int a, int b, int cnt) {
    x += cnt * b; // lll * cnt * b...
    y -= cnt * a;
}

int find_all_solutions(int a, int b, int c, int minx, int
    maxx, int miny, int maxy) {
    int x, y, g;
    if (!find_any_solution(a, b, c, x, y, g)) return 0;
    a /= g;
    b /= g;

    int sign_a = a > 0 ? +1 : -1;
    int sign_b = b > 0 ? +1 : -1;

    shift_solution(x, y, a, b, (minx - x) / b);
    if (x < minx) shift_solution(x, y, a, b, sign_b);
    if (x > maxx) return 0;
    int lx1 = x;

    shift_solution(x, y, a, b, (maxx - x) / b);
    if (x > maxx) shift_solution(x, y, a, b, -sign_b);
    int rx1 = x;

    shift_solution(x, y, a, b, -(miny - y) / a);
    if (y < miny) shift_solution(x, y, a, b, -sign_a);
    if (y > maxy) return 0;
    int lx2 = x;

    shift_solution(x, y, a, b, -(maxy - y) / a);
    if (y > maxy) shift_solution(x, y, a, b, sign_a);
    int rx2 = x;
}

```

```

if (lx2 > rx2) swap(lx2, rx2);
int lx = max(lx1, lx2);
int rx = min(rx1, rx2);

if (lx > rx) return 0;
return (rx - lx) / abs(b) + 1;
}

```

Problema 7.4.3: Quantas soluções não negativas?

Solução:

```

f3640a
long long count_nonnegative_solution(int a, int b, int c) {
    return find_all_solutions(ll(a), ll(b), ll(c), 0LL, ll(c
        / a + 1), 0LL, ll(c / b + 1));
}

```

Problema 7.4.4: Ache a solução com menor valor de $x + y$ (geralmente é dada uma restrição a mais para que a resposta não seja $-\infty$).

Solução: Basta usar a ideia de que $x + y = x_0 + y_0 + k \cdot \frac{b-a}{g}$.

Problema 7.4.5: Sejam M, N inteiros positivos ($1 \leq M \leq 500, 1 \leq N \leq 100$) e uma N -tupla de inteiros positivos $(a_1, \dots, a_N)$ ($1 \leq a_i \leq 10, 1 \leq i \leq N$). Encontre o número de N -tuplas de inteiros positivos $(k_1, \dots, k_N)$ tais que $a_1 k_1 + \dots + a_N k_N = M$.

Solução: TODO

Problema 7.4.6: Qual o menor valor de n tal que $ax + by = N$ admite solução com $x, y \geq 0$ para todo $N \geq n$?

Solução: $n = (a-1)(b-1)$. Para o caso de três variáveis já fica extremamente complicado abordando apenas matematicamente pelo menos.

7.5 Números de fibonacci

$$F_0 = 0, F_1 = 1, F_{n+2} = F_{n+1} + F_n$$

7.5.1 Propriedades interessantes

1. Identidade de Cassini

$$F_{n-1} \cdot F_{n+1} - F_n^2 = (-1)^n$$

2. Regra da adição

$$F_{n+k} = F_k F_{n+1} + F_{k-1} F_n$$

3. Aplicando $k = n$:

$$F_{2n} = F_n (F_{n+1} + F_{n-1})$$

4. Disso, prova-se que $F_n | F_{nk}$

5. Se $F_n | F_m$, então $n | m$

6. Identidade do GCD

$$\gcd(F_m, F_n) = F_{\gcd(n, m)}$$

7. São os números que tornam menos eficiente o algoritmo de euclides

7.5.2 Fibonacci Coding

De acordo com o teorema de Zeckendorf, todo número natural pode ser representado de maneira única como soma de números de fibonacci não consecutivos: $N = F_{k_1} + F_{k_2} + \dots + F_{k_r}$ com $k_1 \geq k_2 + 2, \dots, k_r \geq 2$. Então dá pra escrever o código como $d_0 d_1 \dots d_s 1$, em que $d_i = 1$ se F_{i+2} é usado e um 1 indica o término do código.

Por exemplo, $1 = F_2 = (11)_F$, $2 = F_3 = (011)_F$, $9 = (100011)_F$

Dá pra usar o seguinte algoritmo:

1. Itere até achar o maior menor ou igual a n
2. Subtraia-o de n e coloque o 1 correspondente
3. Repita até o 0
4. Adicione o 1 para indicar o final

Para decodificar é trivial.

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix}$$

Pondo $n = 2k$, obtemos as equações $F_{2k+1} = F_k^2 + F_{k+1}^2$ e $F_{2k} = F_k(2F_{k+1} - F_k)$. A partir delas também dá pra calcular F_n .

7.5.3 Periodicidade

Vamos analisar a sequência módulo m . Ela é periódica. O seu período é o pisano period $\pi(n)$. É desconhecido se $\pi(p^k) = p^{k-1}\pi(p)$ para todo primo p e k inteiro (então podemos assumir que isso sempre vale hahaha. Ou passamos o problema ou desprovamos uma conjectura muito conhecida. Os dois outcomes são bons).

Uma consequência do teorema chinês dos restos é que $\pi(p_1^{a_1} \dots p_k^{a_k}) = \pi(p_1^{a_1}) \dots \pi(p_k^{a_k})$.

Seja p primo. Se módulo 10 $p \equiv \pm 1$, então $\pi(p)|p-1$. Se $p \equiv \pm 3$, então $\pi(p)$ é o quociente entre $2(p+1)$ e um divisor ímpar.

É sabido que $\pi(n) \leq 6n$ com igualdade se, e só se, $n = 2 \cdot 5^r$ com $r \geq 1$.

Se $n = 2^k$, então $\pi(n) = \frac{3n}{2}$. Se $n = 5^k$, então $\pi(n) = 4n$.

$\pi(n)$	+1	+2	+3	+4	+5	+6	+7	+8	+9	+10	+11	+12
0+	1	3	8	6	20	24	16	12	24	60	10	24
12+	28	48	40	24	36	24	18	60	16	30	48	24
24+	100	84	72	48	14	120	30	48	40	36	80	24
36+	76	18	56	60	40	48	88	30	120	48	32	24
48+	112	300	72	84	108	72	20	48	72	42	58	120
60+	60	30	48	96	140	120	136	36	48	240	70	24
72+	148	228	200	18	80	168	78	120	216	120	168	48
84+	180	264	56	60	44	120	112	48	120	96	180	48
96+	196	336	120	300	50	72	208	84	80	108	72	72
108+	108	60	152	48	76	72	240	42	168	174	144	120
120+	110	60	40	30	500	48	256	192	88	420	130	120
132+	144	408	360	36	276	48	46	240	32	210	140	24

7.5.4 Problemas

Problema 7.5.1: Determine $F_0 + F_1 + \dots + F_n$

Solução: $F_{n+2} - 1$

Problema 7.5.2: Dado um certo resto f por $m = 10^{13}$, qual o primeiro n tal que $F_n \equiv r$?

Solução: A ideia é principal é baby-step-giant-step. Sejam $a, b|m$ com $\gcd(a, b) = 1$. Seja F o fibonacci tal que $F \equiv f \pmod{m}$. Então, $F \equiv f \pmod{a}$ e $F \equiv f \pmod{b}$. Encontre todas as ocorrências de $f \pmod{a}$ na sequência de fibonacci módulo a . Faça o mesmo pro b . Suponha que na posição i da primeira, temos algo que satisfaça e que na posição j da segunda também. Seja $t(m)$ o período da sequência de fibonacci módulo m . Temos $t(ab) = \text{lcm}(t(a), t(b))$. Então resolvendo a equação diofantina $i + t(a) \cdot x = j + t(b) \cdot y$, achamos uma posição em

ab que satisfaz o que queremos. Então é só bruteforçar o k em $t(ab) \cdot k + u$. (para pular $t(ab)$ posições é só multiplicar por uma matriz). No caso de $m = 10^{13}$ dá pra tomar $a = 5^9$ e $b = 2^{13}$.

7.6 Primos

7.6.1 Crivo de Eratóstenes

Acha todos os primos de 1 até n usando $O(n)$ memória e $O(n \log \log n)$ tempo

```

2855dc
int n;
vector<bool> is_prime(n + 1, true);
is_prime[0] = is_prime[1] = false;
for (int i = 2; i * i <= n; i++) {
    if (is_prime[i]) {
        for (int j = i * i; j <= n; j += i)
            is_prime[j] = false;
    }
}

```

É possível fazer o crivo usando apenas $O(\sqrt{n} + S)$ de memória utilizando divisão em blocos de tamanho S . Use $10^4 \leq S \leq 10^5$

```

593b41
int count_primes(int n) {
    // conta o numero de primos de 1 ateh n
    const int S = 10000;

    vector<int> primes;
    int nsqrt = sqrt(n);
    vector<char> is_prime(nsqrt + 2, true);
    for (int i = 2; i <= nsqrt; i++) {
        if (is_prime[i]) {
            primes.push_back(i);
            for (int j = i * i; j <= nsqrt; j += i)
                is_prime[j] = false;
        }
    }

    int result = 0;
    vector<char> block(S);
}

```

```

for (int k = 0; k * S <= n; k++) {
    fill(block.begin(), block.end(), true);
    int start = k * S;
    for (int p : primes) {
        int start_idx = (start + p - 1) / p;
        int j = max(start_idx, p) * p - start;
        for (; j < S; j += p)
            block[j] = false;
    }
    if (k == 0) block[0] = block[1] = false;
    for (int i = 0; i < S && start + i <= n; i++) {
        if (block[i]) result++;
    }
}
return result;
}

```

Como encontrar os números num intervalo pequeno de tamanho 10^7 $[L, R]$ mas com $R \leq 10^{12}$? Encontre os primos até $\sqrt{R}$ e marque todos os compostos no segmento $[L, R]$. Isso roda em $O((R - L + 1) \log \log(R) + \sqrt{R} \log \log \sqrt{R})$.

```

91fe31
vector<char> segmentedSieve(long long L, long long R) {
    // generate all primes up to sqrt(R)
    long long lim = sqrt(R);
    vector<char> mark(lim + 1, false);
    vector<long long> primes;
    for (long long i = 2; i <= lim; ++i) {
        if (!mark[i]) {
            primes.emplace_back(i);
            for (long long j = i * i; j <= lim; j += i)
                mark[j] = true;
        }
    }

    vector<char> isPrime(R - L + 1, true);
    for (long long i : primes)
        for (long long j = max(i * i, (L + i - 1) / i * i);
             j <= R; j += i)
            isPrime[j - L] = false;
    if (L == 1) isPrime[0] = false;
}

```

```

return isPrime;
}

```

Na verdade uma versão disso em $O((R - L + 1) \log(R) + \sqrt{R})$ não exige a pre-geração dos primos:

```

d1bf54
vector<char> segmentedSieveNoPreGen(long long L, long long R)
{
    vector<char> isPrime(R - L + 1, true);
    long long lim = sqrt(R);
    for (long long i = 2; i <= lim; ++i)
        for (long long j = max(i * i, (L + i - 1) / i * i);
             j <= R; j += i)
            isPrime[j - L] = false;
    if (L == 1) isPrime[0] = false;
    return isPrime;
}

```

Outra versão do crivo é em $O(n)$. O problema é que usa $32\times$ a memória (então só faz sentido pra $n \leq 10^7$). E também não é tão "mais rápido" que o crivo normal. É até mais lento que o crivo em blocos.

O benefício de usar ele é que com ele dá pra calcular facilmente a fatoração de um número, pois a array $lp[i]$ calcula o menor fator primo de todos os números no range $[2; n]$.

```

f222d8
const int N = 10000000;
vector<int> lp(N + 1);
vector<int> pr;

for (int i = 2; i <= N; ++i) {
    if (lp[i] == 0) {
        lp[i] = i;
        pr.push_back(i);
    }
    for (int j = 0; i * pr[j] <= N; ++j) {
        lp[i * pr[j]] = pr[j];
        if (pr[j] == lp[i]) {
            break;
        }
    }
}

```

}

Crivo mais poderoso que já vi. Ele funcionou num problema que pedia a soma dos divisores de números $\leq 10^{16}$ (olhar como resolvi na seção de funções multiplicativas) que precisou dos primos de 1 até 10^8 .

0b9b48

```
#define MAX 1000000000
#define LMT 10000

int flag[MAX / 64];
int primes[5761460], total;

#define chkC(n) (flag[n >> 6] & (1 << ((n >> 1) & 31)))
#define setC(n) (flag[n >> 6] |= (1 << ((n >> 1) & 31)))

void sieve() {
    int i, j, k;
    flag[0] |= 0;
    for (i = 3; i < LMT; i += 2)
        if (!chkC(i))
            for (j = i * i, k = i << 1; j < MAX; j += k)
                setC(j);
    primes[(j = 0)++] = 2;
    for (i = 3; i < MAX; i += 2)
        if (!chkC(i)) primes[j++] = i;
    total = j;
}
```

7.6.2 Testes de primalidade

Clássico $O(\sqrt{n})$

f8d9d4

```
bool isPrime(int x) {
    for (int d = 2; d * d <= x; d++) {
        if (x % d == 0) return false;
    }
    return x >= 2;
}
```

MillerRabin: o determinístico passa tranquilo pra $t \leq 500$ casos e $n \leq 2^{63} - 1$

8899f6

```
using u64 = uint64_t;
using u128 = __uint128_t;

u64 binpower(u64 base, u64 e, u64 mod) {
    u64 result = 1;
    base %= mod;
    while (e) {
        if (e & 1) result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}

bool check_composite(u64 n, u64 a, u64 d, int s) {
    u64 x = binpower(a, d, n);
    if (x == 1 || x == n - 1) return false;
    for (int r = 1; r < s; r++) {
        x = (u128)x * x % n;
        if (x == n - 1) return false;
    }
    return true;
};

// NAO DETERMINISTICO
bool MillerRabin(u64 n, int iter = 5) { // returns true if n
    is probably prime, else returns false.
    if (n < 4) return n == 2 || n == 3;

    int s = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        s++;
    }

    for (int i = 0; i < iter; i++) {
        int a = 2 + rand() % (n - 3);
        if (check_composite(n, a, d, s)) return false;
    }
}
```

```

    }
    return true;
}

// DETERMINISTICO
bool MillerRabin(u64 n) { // returns true if n is prime,
    else returns false.
    if (n < 2) return false;

    int r = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        r++;
    }

    for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
        37}) {
        if (n == a) return true;
        if (check_composite(n, a, d, r)) return false;
    }
    return true;
}

```

7.6.3 Fatoração

Para $n \leq 10^{12}$ dá pra usar um crivo até 10^6 e gerar uma lista de primos e ir testando.

Para $n \leq 10^{18}$, usar pollard

```

        6b3f96
typedef long long unsigned int llui;
typedef long long int lli;
typedef long double float64;

llui mul_mod(llui a, llui b, llui m) {
    llui y = (llui)((float64)a * (float64)b / m + (float64)1
        / 2);
    y = y * m;
    llui x = a * b;
    llui r = x - y;
}

```

```

    if ((lli)r < 0) {
        r = r + m;
        y = y - 1;
    }
    return r;
}

llui C, a, b;
llui gcd() {
    llui c;
    if (a > b) {
        c = a;
        a = b;
        b = c;
    }
    while (1) {
        if (a == 1LL) return 1LL;
        if (a == 0 || a == b) return b;
        c = a;
        a = b % a;
        b = c;
    }
}

llui f(llui a, llui b) {
    llui tmp;
    tmp = mul_mod(a, a, b);
    tmp += C;
    tmp %= b;
    return tmp;
}

llui pollard(llui n) {
    if (!(n & 1)) return 2;
    C = 0;
    llui iteracoes = 0;
    while (iteracoes <= 1000) {
        llui x, y, d;
        x = y = 2;
        d = 1;
    }
}

```

```

    while (d == 1) {
        x = f(x, n);
        y = f(f(y, n), n);
        llui m = (x > y) ? (x - y) : (y - x);
        a = m;
        b = n;
        d = gcd();
    }
    if (d != n) return d;
    iteracoes++;
    C = rand();
}

return 1;
}

llui pot(llui a, llui b, llui c) {
    if (b == 0) return 1;
    if (b == 1) return a % c;
    llui resp = pot(a, b >> 1, c);
    resp = mul_mod(resp, resp, c);
    if (b & 1) resp = mul_mod(resp, a, c);
    return resp;
}

// Rabin–Miller primality testing algorithm
bool isPrime(llui n) {
    llui d = n - 1;
    llui s = 0;
    if (n <= 3 || n == 5) return true;
    if (!(n & 1)) return false;
    while (!(d & 1)) {
        s++;
        d >>= 1;
    }
    for (llui i = 0; i < 32; i++) {
        llui a = rand();
        a <<= 32;
        a += rand();
        a %= (n - 3);
        a += 2;

```

```

        llui x = pot(a, d, n);
        if (x == 1 || x == n - 1) continue;
        for (llui j = 1; j <= s - 1; j++) {
            x = mul_mod(x, x, n);
            if (x == 1) return false;
            if (x == n - 1) break;
        }
        if (x != n - 1) return false;
    }
    return true;
}

map<llui, int> factors;
// Precondition: factors is an empty map, n is a positive
// integer
// Postcondition: factors[p] is the exponent of p in prime
// factorization of n
void fact(llui n) {
    if (!isPrime(n)) {
        llui fac = pollard(n);
        fact(n / fac);
        fact(fac);
    } else {
        map<llui, int>::iterator it;
        it = factors.find(n);
        if (it != factors.end()) {
            (*it).second++;
        } else {
            factors[n] = 1;
        }
    }
}
}

```

7.7 Funções comuns de teoria dos números

7.7.1 Função totiente de Euler

Seja $\phi(n)$ a função totiente de euler.

Código para calcular $\phi(n)$ em $O(\sqrt{n})$:

```

int phi(int n) {

```

fb8d57

```

int result = n;
for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) {
        while (n % i == 0)
            n /= i;
        result -= result / i;
    }
}
if (n > 1) result -= result / n;
return result;
}

```

Código para calcular $\phi(n)$ de 1 a n em $O(n \log \log n)$ com crivo:

0815d0

```

void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}

```

Fatos conhecidos:

$$\phi(p_1^{\alpha_1} \dots p_k^{\alpha_k}) = p_1^{\alpha_1} \dots p_k^{\alpha_k} \left(1 - \frac{1}{p_1}\right) \dots \left(1 - \frac{1}{p_k}\right)$$

$$\sum_{d|n} \phi(d) = n$$

Para calcular $x^n \bmod m$ rápido (mesmo que $\gcd(x, m) \neq 1$) podemos usar:

$$x^n \equiv x^{\phi(m) + [n \bmod \phi(m)]} \pmod{m}$$

7.7.2 Funções quantidade e soma de divisores

Seja $n = p_1^{e_1} \dots p_k^{e_k}$. Então

$$d(n) = \prod_{i=1}^k (e_i + 1)$$

$$\sigma(n) = \prod_{i=1}^k \frac{p_i^{e_i+1} - 1}{p_i - 1}$$

É possível calculá-las em $O(\sqrt{n})$ ingenuamente. Mas se você usar o crivo super-poderoso e ir subindo no vetor de primos, dá até pra calcular a soma dos divisores de vários $n \leq 10^{16}$. (dá pra usar crivo normal também, ele só não vai chegar a 10^{16}).

7.7.3 Função de mobius

$\mu(n) = 0$ se $p^2 | n$ para algum p . Caso contrário, $\mu(n) = (-1)^k$ em que k é o número de fatores primos distintos de n . Por exemplo, $\mu(1) = 1$, $\mu(2) = -1$, $\mu(3) = -1$, $\mu(4) = 0$, $\mu(6) = 1$. Calcula com crivo:

b2a00f

```

const int MAXN = 2e6 + 100;

std::vector<int> prime;
bool is_composite[MAXN];
int mu[MAXN];

void sieve(int n) {
    std::fill(is_composite, is_composite + n, false);
    mu[1] = 1;
    for (int i = 2; i < n; ++i) {
        if (!is_composite[i]) {
            prime.push_back(i);
            mu[i] = -1; // i is prime
        }
        for (int j = 0; j < prime.size() && i * prime[j] < n; ++j) {
            is_composite[i * prime[j]] = true;
            if (i % prime[j] == 0) {
                mu[i * prime[j]] = 0; // prime[j] divides i
            }
        }
    }
}

```

```

        break;
    } else {
        mu[i * prime[j]] = -mu[i]; // prime[j] does
                                not divide i
    }
}
}
}
}
}

```

7.7.4 Funções aritméticas multiplicativas

Funções nos inteiros tais que $f(ab) = f(a)f(b)$ quando $\gcd(a, b) = 1$. Na grande maioria dos casos, é fácil adaptar o crivo acima. O único problema é caso fique difícil de fazer o caso em que o primo divide i . O que você pode fazer é criar uma array cnt indicando o expoente do menor fator primo. Aqui um exemplo em que $f(p^k) = k$:

d90ff

```

std::vector<int> prime;
bool is_composite[MAXN];
int func[MAXN], cnt[MAXN];

void sieve(int n) {
    std::fill(is_composite, is_composite + n, false);
    func[1] = 1;
    for (int i = 2; i < n; ++i) {
        if (!is_composite[i]) {
            prime.push_back(i);
            func[i] = 1;
            cnt[i] = 1;
        }
        for (int j = 0; j < prime.size() && i * prime[j] < n; ++j) {
            is_composite[i * prime[j]] = true;
            if (i % prime[j] == 0) {
                func[i * prime[j]] = func[i] / cnt[i] * (cnt[i] + 1);
                cnt[i * prime[j]] = cnt[i] + 1;
                break;
            } else {

```

```

                func[i * prime[j]] = func[i] * func[prime[j]];
                cnt[i * prime[j]] = 1;
            }
        }
    }
}

```

7.8 Inversos modulares

Para achar o inverso multiplicativo de a módulo m , uma condição necessária pra ele existir é que $\gcd(a, m) = 1$. E como já falei em 7.3.1, basta usar o algoritmo estendido de euclides.

Se não quiser usar ele, dá pra usar exponenciação binária pra calcular $a^{\phi(m)-1}$. Em particular, se m for primo dá pra fazer isso em a^{m-2} .

Problema 7.8.1: Ache inversos módulo p primo.

Solução 1:

7d081d

```

// acha inversos de 1 a n mod p primo
// acha em O(n) tempo. Requer O(n) memo

int n = 10, p = 1000000007;
int inv[n + 1];
inv[1] = 1;
for (int i = 2; i <= n; i++)
    inv[i] = 1LL * (p - p / i) * inv[p % i] % p;

```

Solução 2:

73dea2

```

// acha inversos de a[1], a[2], ..., a[n] mod p primo
// acha em O(n + log p) tempo. Requer O(n) memo

#include "bits/stdc++.h"
using namespace std;

const long long P = 1000000007;
long long qpow(long long a, long long b) {
    long long ans = 1;
    while (b) {

```

```

        if (b & 1) ans = ans * a % P;
        a = a * a % P;
        b >>= 1;
    }
    return ans;
}

long long n, a[1000010], pre[1000010], suf[1000010], pr = 1;
// prefix, suffix, product
int main() {
    cin >> n;
    pre[0] = suf[n + 1] = 1;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    for (int i = 1; i <= n; i++) {
        pre[i] = pre[i - 1] * a[i] % P;
        suf[n + 1 - i] = suf[n + 2 - i] * a[n + 1 - i] % P;
        pr = pr * a[i] % P;
    }
    pr = qpow(pr, P - 2);
    for (int i = 1; i <= n; i++) {
        cout << (pre[i - 1] * suf[i + 1] % P) * pr % P <<
            endl;
    }
}

```

7.9 Congruências modulares

Sejam a, b e n inteiros dados. Queremos achar os x inteiros em $[0, n - 1]$ tais que

$$a \cdot x \equiv b \pmod{n}$$

Seja $g = \gcd(a, n)$. Seja $a = a'g$, $n = n'g$. Se g não dividir b , não existem soluções. Seja $b = gb'$. A equação original implica

$$a' \cdot x \equiv b' \pmod{n'}$$

Como $\gcd(a', n') = 1$, então só existe uma solução. Então, ao todo, existem g soluções dadas por

$$x_i \equiv x' + i \cdot n' \pmod{n}, i = 0 \dots g - 1$$

Também é possível resolvê-la utilizando o algoritmo estendido de euclides com a equação

$$a \cdot x + n \cdot k = b$$

em que x e k são desconhecidos.

7.9.1 Teorema Chinês dos Restos

Sejam $m_1, \dots, m_k$ coprimos. Sejam a_i inteiros. Queremos achar a tal que $a \equiv a_i \pmod{m_i}$ para todo $1 \leq i \leq k$.

A solução é única mod $m_1 m_2 \dots m_k$.

Para construí-la, seja $M_i = \prod_{j \neq i} m_j$ e $N_i = M_i^{-1} \pmod{m_i}$. Então, módulo $m_1 \dots m_k$, a solução é

$$a \equiv \sum_{i=1}^k a_i M_i N_i$$

756102

```

struct Congruence {
    long long a, m;
};

long long chinese_remainder_theorem(vector<Congruence> const
    & congruences) {
    long long M = 1;
    for (auto const& congruence : congruences) {
        M *= congruence.m;
    }

    long long solution = 0;
    for (auto const& congruence : congruences) {
        long long a_i = congruence.a;
        long long M_i = M / congruence.m;
        long long N_i = mod_inv(M_i, congruence.m);
        solution = (solution + a_i * M_i % M * N_i) % M;
    }
    return solution;
}

```


7.9.2 Algoritmo de Garner

É possível usar o teorema chinês dos restos para representar um número muito gigante a utilizando apenas os a_i . Utilizamos a representação

$$a = x_1 + x_2 p_1 + x_3 p_1 p_2 + \cdots + x_k p_1 \cdots p_{k-1}$$

com $x_i \in [0, p_i)$

O que o algoritmo de Garner faz é computar os dígitos x_i . É possível utilizar esse algoritmo para calcular em $O(k^2)$ os x_i . Eis uma implementação **em java**:

```

410684
final int SZ = 100;
int pr[] = new int[SZ];
int r[][] = new int[SZ][SZ];

void init() {
    for (int x = 1000 * 1000 * 1000, i = 0; i < SZ; ++x)
        if (BigInteger.valueOf(x).isProbablePrime(100))
            pr[i++] = x;

    for (int i = 0; i < SZ; ++i)
        for (int j = i + 1; j < SZ; ++j)
            r[i][j] = BigInteger.valueOf(pr[i]).modInverse(
                BigInteger.valueOf(pr[j])).intValue();
}

class Number {
    int a[] = new int[SZ];

    public Number() {}

    public Number(int n) {
        for (int i = 0; i < SZ; ++i)
            a[i] = n % pr[i];
    }

    public Number(BigInteger n) {
        for (int i = 0; i < SZ; ++i)
            a[i] = n.mod(BigInteger.valueOf(pr[i])).intValue();
    }
}

```

```

}

public Number add(Number n) {
    Number result = new Number();
    for (int i = 0; i < SZ; ++i)
        result.a[i] = (a[i] + n.a[i]) % pr[i];
    return result;
}

public Number subtract(Number n) {
    Number result = new Number();
    for (int i = 0; i < SZ; ++i)
        result.a[i] = (a[i] - n.a[i] + pr[i]) % pr[i];
    return result;
}

public Number multiply(Number n) {
    Number result = new Number();
    for (int i = 0; i < SZ; ++i)
        result.a[i] = (int) ((a[i] * 1l * n.a[i]) % pr[i]);
    return result;
}

public BigInteger bigIntegerValue(boolean
    can_be_negative) {
    BigInteger result = BigInteger.ZERO, mult =
        BigInteger.ONE;
    int x[] = new int[SZ];
    for (int i = 0; i < SZ; ++i) {
        x[i] = a[i];
        for (int j = 0; j < i; ++j) {
            long cur = (x[i] - x[j]) * 1l * r[j][i];
            x[i] = (int) ((cur % pr[i] + pr[i]) % pr[i]);
        }
        result = result.add(mult.multiply(BigInteger.
            valueOf(x[i])));
        mult = mult.multiply(BigInteger.valueOf(pr[i]));
    }
}

```

```

    if (can_be_negative)
        if (result.compareTo(mult.shiftRight(1)) >= 0)
            result = result.subtract(mult);

    return result;
}

```

7.9.3 Fatorial módulo p

Dá pra precalcular em $O(p)$ os fatoriais e em $O(\log_p n)$ o fatorial de um determinado número. Isso usa $O(p)$ de memória.

```

int factmod(int n, int p) {
    vector<int> f(p);
    f[0] = 1;
    for (int i = 1; i < p; i++)
        f[i] = f[i - 1] * i % p;

    int res = 1;
    while (n > 1) {
        if ((n / p) % 2) res = p - res;
        res = res * f[n % p] % p;
        n /= p;
    }
    return res;
}

```

Truque de mestre! Se $p \approx 10^9$, dá pra hardcodar um vetor fatmod tal que fatmod[i] é $(Si)!$ módulo p em que S é um tamanho de bloco. Dá pra usar $S = 10^6$ pro vetor ter 10^3 caras e isso ser algo colocável no código.

Então ficamos com:

```

const long long int mod = 1e9 + 7;
const long long int maxn = 1e9;
const int tempo = 1e6;
long long int fatmod[mod / tempo + 1] = {
    /* com outro programa calcular os numeros e copiar e
       colar aqui*/};

```

```

long long int computa_fatmod(int n) {
    if (n >= mod) return 0;
    long long int atual = fatmod[n / tempo];
    for (int i = (n / tempo) * tempo + 1; i <= n; i++) {
        atual *= i;
        atual %= mod;
    }
    return atual;
}

```

7.9.4 Fórmula de Legendre

Qual o expoente de p na fatoração prima de $n!$? Sem calcular $n!$, podemos calcular isso em $O(\log_p n)$ com a seguinte fórmula:

$$v_p(n!) = \sum_{i=1}^{\infty} \left\lfloor \frac{n}{p^i} \right\rfloor$$

```

int multiplicity_factorial(int n, int p) {
    int count = 0;
    do {
        n /= p;
        count += n;
    } while (n);
    return count;
}

```

7.9.5 Logaritmo Discreto

Inteiro x satisfazendo

$$a^x \equiv b \pmod{m}$$

para inteiros a, b, m dados. Será resolvido com o algoritmo **baby-step giant-step** de complexidade $O(\sqrt{m})$. Assumiremos que $0^0 \equiv 1$.

Forma mais simples sem usar unordered map roda em $O(\sqrt{m} \log m)$ (assumiremos nas duas versões $\gcd(a, m) = 1$):

```

int powmod(int a, int b, int m) {
    int res = 1;

```

```

while (b > 0) {
    if (b & 1) {
        res = (res * 11l * a) % m;
    }
    a = (a * 11l * a) % m;
    b >>= 1;
}
return res;
}

int solve(int a, int b, int m) {
    a %= m, b %= m;
    int n = sqrt(m) + 1;
    map<int, int> vals;
    for (int p = 1; p <= n; ++p)
        vals[powmod(a, p * n, m)] = p;
    for (int q = 0; q <= n; ++q) {
        int cur = (powmod(a, q, m) * 11l * b) % m;
        if (vals.count(cur)) {
            int ans = vals[cur] * n - q;
            return ans;
        }
    }
    return -1;
}

```

Usando unordered map:

```

670ed1
// Returns minimum x for which a ^ x % m = b % m, a and m
// are coprime.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int n = sqrt(m) + 1;

    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 11l * a) % m;

    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++q) {

```

```

        vals[cur] = q;
        cur = (cur * 11l * a) % m;
    }

    for (int p = 1, cur = 1; p <= n; ++p) {
        cur = (cur * 11l * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur];
            return ans;
        }
    }
    return -1;
}

```

Caso em que $\gcd(a, m) \neq 1$. Se g não divide b , então não há solução. Se $g|b$, então tomando $a = g\alpha$, $b = g\beta$, $m = g\nu$. Daí, precisamos resolver $\alpha a^{x-1} \equiv \beta \pmod{\nu}$. Mas o nosso algoritmo pode ser facilmente estendido para $ka^x \equiv b$:

```

dcf2d0
// Returns minimum x for which a ^ x % m = b % m.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int k = 1, add = 0, g;
    while ((g = gcd(a, m)) > 1) {
        if (b == k) return add;
        if (b % g) return -1;
        b /= g, m /= g, ++add;
        k = (k * 11l * a / g) % m;
    }

    int n = sqrt(m) + 1;
    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 11l * a) % m;

    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++q) {

```

```

for (int p = 1, cur = k; p <= n; ++p) {
    cur = (cur * 1ll * an) % m;
    if (vals.count(cur)) {
        int ans = n * p - vals[cur] + add;
        return ans;
    }
}
return -1;
}

```

7.9.6 Raízes primitivas

g é dita uma raiz primitiva módulo n se todo número coprimo a n é congruente a uma potência de g módulo n . Se g é uma raiz primitiva e $g^k \equiv a$, então k é dito o índice ou logaritmo discreto de a na base g .

Uma raiz primitiva módulo n existe se e só se:

1. $n = 1, 2, 4$ ou
2. $n = p^k$, p primo ímpar, $k \geq 1$
3. $n = 2p^k$, p primo ímpar, $k \geq 1$

Se existe alguma raiz primitiva módulo n , então existem $\phi(\phi(n))$ delas. O seguinte código acha um gerador em $O(\text{Ans} \cdot \log \phi(n) \cdot \log n)$. Sabe-se que $g \in O(\log^6 p)$. Há um detalhe: o código abaixo **só funciona** para p primo. Mas pra funcionar pra qualquer número basta trocar a definição da variável phi:

```

c6d472
int powmod(int a, int b, int p) {
    int res = 1;
    while (b)
        if (b & 1)
            res = int(res * 1ll * a % p), --b;
        else
            a = int(a * 1ll * a % p), b >>= 1;
    return res;
}

int generator(int p) {
    vector<int> fact;
    int phi = p - 1, n = phi;

```

```

for (int i = 2; i * i <= n; ++i)
    if (n % i == 0) {
        fact.push_back(i);
        while (n % i == 0)
            n /= i;
    }
if (n > 1) fact.push_back(n);

for (int res = 2; res <= p; ++res) {
    bool ok = true;
    for (size_t i = 0; i < fact.size() && ok; ++i)
        ok &= powmod(res, phi / fact[i], p) != 1;
    if (ok) return res;
}
return -1;
}

```

7.9.7 Raiz discreta

Dado um primo n e dois inteiros a, k , achar todos os x tais que $x^k \equiv a \pmod{n}$.

Solução: achar uma raiz primitiva g módulo n (em $O(\text{Ans} \cdot \log^2 n)$). Depois, se $x \equiv g^y$, basta resolver $(g^k)^y \equiv a$ em y usando o logaritmo discreto.

Problema 7.9.1: Encontrar todas as soluções a partir de uma.

Solução: suponha que conhecemos a solução $x_0 = g^{y_0} \pmod{n}$. Então, todas as soluções são da forma

$$x \equiv g^{y_0 + i \frac{\phi(n)}{\gcd(k, \phi(n))}}$$

com i variando em $\mathbb{Z}$.

O seguinte código mostra todas as soluções:

```

076e87
int gcd(int a, int b) { return a ? gcd(b % a, a) : b; }

int powmod(int a, int b, int p) {
    int res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a % p;
    }
    a = a * a % p;

```

```

        b >= 1;
    }
    return res;
}

// Finds the primitive root modulo p
int generator(int p) {
    vector<int> fact;
    int phi = p - 1, n = phi;
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            fact.push_back(i);
            while (n % i == 0)
                n /= i;
        }
    }
    if (n > 1) fact.push_back(n);

    for (int res = 2; res <= p; ++res) {
        bool ok = true;
        for (int factor : fact) {
            if (powmod(res, phi / factor, p) == 1) {
                ok = false;
                break;
            }
        }
        if (ok) return res;
    }
    return -1;
}

// This program finds all numbers x such that  $x^k = a \pmod n$ 
int main() {
    int n, k, a;
    scanf("%d %d %d", &n, &k, &a);
    if (a == 0) {
        puts("1\n0");
        return 0;
    }
}

```

```

int g = generator(n);

// Baby-step giant-step discrete logarithm algorithm
int sq = (int)sqrt(n + .0) + 1;
vector<pair<int, int>> dec(sq);
for (int i = 1; i <= sq; ++i)
    dec[i - 1] = {powmod(g, i * sq * k % (n - 1), n), i};
sort(dec.begin(), dec.end());
int any_ans = -1;
for (int i = 0; i < sq; ++i) {
    int my = powmod(g, i * k % (n - 1), n) * a % n;
    auto it = lower_bound(dec.begin(), dec.end(),
        make_pair(my, 0));
    if (it != dec.end() && it->first == my) {
        any_ans = it->second * sq - i;
        break;
    }
}
if (any_ans == -1) {
    puts("0");
    return 0;
}

// Print all possible answers
int delta = (n - 1) / gcd(k, n - 1);
vector<int> ans;
for (int cur = any_ans % delta; cur < n - 1; cur +=
    delta)
    ans.push_back(powmod(g, cur, n));
sort(ans.begin(), ans.end());
printf("%d\n", ans.size());
for (int answer : ans)
    printf("%d ", answer);
}

```

7.10 Sistemas Numéricos

7.10.1 Ternário balanceado

Usa os dígitos 1, 0 e $Z (= -1)$. Para calcular a partir da base 3, usemos o exemplo $64_{10} = 02101_3$. Você sempre processa da direita pra esquerda. O 1 e 0 são skipados. E também toda vez que encontramos um 2, transformamos ele em Z e adicionamos 1 ao próx dígito (ou seja, existe um carry). Então $1Z101 = 64_{10}$.

7.10.2 Código de Gray

Sistema binário em que valores consecutivos diferem em um bit. Os gray codes para números de 3 bits são 000, 001, 011, 010, 110, 111, 101, 100. Então $G(4) = 110_2 = 6$. É fácil calcular $G(n)$ com operações binárias. Basta usar

```
22e7f9
int g(int n) { return n ^ (n >> 1); }
```

Problema 7.10.1: Dado $g(n)$, recuperar n .

Solução:

```
3a6f06
int rev_g(int g) {
    int n = 0;
    for (; g; g >>= 1)
        n ^= g;
    return n;
}
```

Algumas coisas legais:

1. os códigos de gray de n bits determinam um ciclo hamiltoniano num hiper-cubo de n dimensões
2. dá pra resolver o problema das torres de hanoi. Seja n o número de discos. Comece com $G(0)$ e vá indo de $G(i)$ para $G(i+1)$. Interprete como se o i -ésimo bit do gray code atual representasse o i -ésimo disco. Lembrando que só um bit muda, podemos interpretar a mudança no i -ésimo bit como uma mudança no i -ésimo disco. Note que existe exatamente uma opção de movimento para cada disco em cada etapa. Existem duas opções para o menor disco mas sempre tem a seguinte estratégia. Se n for ímpar, a sequência de menor quantidade de movimentos é $f \rightarrow t \rightarrow r \rightarrow f \rightarrow t \rightarrow r \rightarrow \dots$ em que f é a rod inicial, t é a red final e r a restante. E se n for ímpar ela é $f \rightarrow r \rightarrow r \rightarrow f \rightarrow r \rightarrow t \rightarrow \dots$.

3. Sendo $G(N)$ a sequência de códigos de Gray para N bits, $G(N)^R$ ela revertida e $aG(N)$ ela concatenada do bit de valor a , então

$$G(N) = 0G(N-1) \cup 1G(N-1)^R$$

Implementação da torre de hanoi (discos de 0 a $n-1$, pilhas de 0 a 2, move tudo da pilha 0 para a 2)

```
ca1667
int g(int n) { return n ^ (n >> 1); }

void swap_peg(int& x) {
    if (x == 1)
        x = 2;
    else if (x == 2)
        x = 1;
}

int main() {
    int n;
    cin >> n;
    for (int i = 1;; i++) {
        int gi = g(i - 1);
        int gip1 = g(i);
        int bitpos = gi ^ gip1;
        int pos_dude = __builtin_ctz(bitpos);
        if (pos_dude == n) break;
        int from_peg = (i & (i - 1)) % 3;
        int to_peg = ((i | (i - 1)) + 1) % 3;
        if (n % 2 == 0) {
            swap_peg(from_peg);
            swap_peg(to_peg);
        }
        cout << "Disco " << pos_dude << " vai de " <<
            from_peg << " para " << to_peg << "\n";
    }
}
```

7.11 Manipulação de bits

```

5aff4a
bool isPowerOfTwo(unsigned int n) { return n && !(n & (n - 1)); }

int countSetBits(int n) {
    int count = 0;
    while (n) {
        n = n & (n - 1);
        count++;
    }
    return count;
}

// n & (n+1) limpa os trailing ones
// n | (n+1) seta o último bit zero
// n & (-n) extrai o último bit

// __builtin_popcount(unsigned int) numero de bits um
// __builtin_ffs(int) índice do bit 1 mais a direita
// __builtin_clz(unsigned int) -> qnt de leading zeros
// __builtin_ctz(unsigned int) -> qnt de trailing zeros
// __builtin_parity() -> paridade do número de uns
// NOTA: algumas dessas podem ser lentas no GCC se nao
//        ativar um target específico de compilador com
// #pragma GCC target("popcnt")

```

7.11.1 Iterar por todas as submáscaras de uma máscara

```

8a2859
// O(3^n)
for (int s = m;; s = (s - 1) & m) {
    // poe o codigo aqui se quiser considerar o caso s=0
    // se n, bota dps do if
    if (s == 0) break;
}

```

}

7.12 Aritmética de precisão arbitrária

Para representar números muito grandes, podemos usar um vector de int em que cada elemento representa um dígito e isso na base 10^9

```

69c051
typedef vector<int> lnum;
const int base = 1000 * 1000 * 1000;

// printar
printf("%d", a.empty() ? 0 : a.back());
for (int i = (int)a.size() - 2; i >= 0; --i)
    printf("%09d", a[i]);

// leitura (string)
for (int i = (int)s.length(); i > 0; i -= 9)
    if (i < 9)
        a.push_back(atoi(s.substr(0, i).c_str()));
    else
        a.push_back(atoi(s.substr(i - 9, 9).c_str()));

// leitura (vetor de char)
for (int i = (int)strlen(s); i > 0; i -= 9) {
    s[i] = 0;
    a.push_back(atoi(i >= 9 ? s + i - 9 : s));
}

// se a entrada contiver leading zeros
while (a.size() > 1 && a.back() == 0)
    a.pop_back();

// a+= b
int carry = 0;
for (size_t i = 0; i < max(a.size(), b.size()) || carry; ++i)
{
    if (i == a.size()) a.push_back(0);
    a[i] += carry + (i < b.size() ? b[i] : 0);
    carry = a[i] >= base;
    if (carry) a[i] -= base;
}

```

```

// a -= b
int carry = 0;
for (size_t i = 0; i < b.size() || carry; ++i) {
    a[i] -= carry + (i < b.size() ? b[i] : 0);
    carry = a[i] < 0;
    if (carry) a[i] += base;
}
while (a.size() > 1 && a.back() == 0)
    a.pop_back();

// multiplicacao por b < base
int carry = 0;
for (size_t i = 0; i < a.size() || carry; ++i) {
    if (i == a.size()) a.push_back(0);
    long long cur = carry + a[i] * 111 * b;
    a[i] = int(cur % base);
    carry = int(cur / base);
}
while (a.size() > 1 && a.back() == 0)
    a.pop_back();

// multiplicacao por outro vector
lnum c(a.size() + b.size());
for (size_t i = 0; i < a.size(); ++i)
    for (int j = 0, carry = 0; j < (int)b.size() || carry; ++j) {
        long long cur = c[i + j] + a[i] * 111 * (j < (int)b.size() ? b[j] : 0) + carry;
        c[i + j] = int(cur % base);
        carry = int(cur / base);
    }
while (c.size() > 1 && c.back() == 0)
    c.pop_back();

// divisao por b < base (a /= b)
int carry = 0;
for (int i = (int)a.size() - 1; i >= 0; --i) {
    long long cur = a[i] + carry * 111 * base;
    a[i] = int(cur / b);

```

```

    carry = int(cur % b);
}
while (a.size() > 1 && a.back() == 0)
    a.pop_back();

```

Também podemos fatorar em fatores primos e trabalhar com eles.

Outra ideia é usar o algoritmo de Garner.

Uma ideia para variáveis de ponto flutuante é guardar um int com o expoente e um double com um valor em $[0.1, 1)$

7.12.1 IO do int128

```

d9afcb
__int128 read() {
    __int128 x = 0, f = 1;
    char ch = getchar();
    while (ch < '0' || ch > '9') {
        if (ch == '-') f = -1;
        ch = getchar();
    }
    while (ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}
void print(__int128 x) {
    if (x < 0) {
        putchar('-');
        x = -x;
    }
    if (x > 9) print(x / 10);
    putchar(x % 10 + '0');
}
bool cmp(__int128 x, __int128 y) { return x > y; }

```

7.13 FFT

Seja $A(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$ e $B(x)$ definido analogamente. Queremos saber os coeficientes de $A(x) \cdot B(x)$ em $O(n \log n)$.

Para isso, $\text{DFT}(A) = \text{DFT}(a_0, \dots, a_{n-1}) = (y_0, \dots, y_{n-1}) = (A(w_n^0), \dots, A(w_n^{n-1}))$. Ademais, defina $\text{InverseDFT}(y_0, \dots, y_{n-1}) =$

$(a_0, \dots, a_{n-1})$. Note que $AB = \text{InverseDFT}(\text{DFT}(AB)) = \text{InverseDFT}(\text{DFT}(A)\text{DFT}(B))$ (em que a multiplicação de dentro é component-wise). Sendo assim, a ideia é fazer a DFT e a InverseDFT em $O(n \log n)$, para daí obter AB .

Essa FFT não é nada boa para multiplicações envolvendo coeficientes altos. Vão ter muitos erros de imprecisão. Geralmente se não tiver gente perto de 10^{15} (pra double) ou 10^{18} (pra long double) tá de boa.

```

96096f
using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd>& a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j) swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i + j], v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd& x : a)
            x /= n;
    }
}

```

```

    }
}

vector<int> multiply(vector<int> const& a, vector<int> const
& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end()
);
    int n = 1;
    while (n < a.size() + b.size())
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}

```

7.13.1 Number theoretic transform

Objetivo: multiplicar polinômios agora com os coeficientes módulo p .

Solução: para $p = 73400233$, $w_{2^{20}} = 5$ é uma 2^{20} -ésima raiz primitiva. Podemos, então, utilizar a seguinte FFT:

```

e35a40
/**
 * NTT eh o mesmo que FFT, so que com um *n-th root*
 * diferente
 * MOD tem que ser um primo do tipo p = c * 2^k + 1
 * Com isso, a n-th root existe pra n = 2^k.
 * Essa n-th root é g^c, onde g é um *primitive root* de 2^k
 */

namespace NTT {

```

```

const int mod = 998244353; // 998244353 7340033
const int root = 363395222; // 15311432 5
const int root_1 = 704923114; // 469870224 4404020
const int root_pw = 1 << 19; // 1 << 23; 1 << 20;
// ~5*10^5 ~8*10^6 ~10^6

int fast_pow(int a, int b, int m) {
    int ans = 1;
    while (b) {
        if (b & 1) ans = 1LL * ans * a % m;
        a = 1LL * a * a % m;
        b >>= 1;
    }
    return ans;
}

int inverse(int x, int m) { return fast_pow(x, m - 2, m); }

/* INICIO */
/** So quando as constantes acima nao forem suficientes */

map<int, int> factor(int n) {
    map<int, int> ans;
    for (int i = 2; i * i <= n; ++i) {
        while (n % i == 0) {
            ans[i]++;
            n /= i;
        }
    }
    if (n > 1) ans[n]++;
    return ans;
}

int Phi(int n) {
    auto fact = factor(n);
    int ans = 1;

    for (auto [a, b] : fact) {
        ans *= fast_pow(a, b, n + 1) - fast_pow(a, b - 1, n
+ 1);
    }
}

```

```

    return ans;
}

int prim_root(int n) {
    int phi = Phi(n);
    auto fact = factor(phi);

    for (int res = 2; res <= n; ++res) {
        if (__gcd(res, n) > 1) continue;
        bool ok = true;
        for (auto [a, b] : fact) {
            ok &= fast_pow(res, phi / a, n) != 1;
            if (!ok) break;
        }
        if (ok) return res;
    }
    return -1;
}

// Generates NTT constants for any Mod and prints it on the
screen
void generate(int Mod) {

    int n = 1;
    int aux = Mod - 1;
    while ((aux & 1) == 0)
        aux >>= 1, n <<= 1;
    int c = aux;

    // g = primitive root de Mod
    int g = prim_root(Mod);
    if (g == -1) {
        printf("No constants could be found, cant find
        primitive root of %d\n", n);
        return;
    }
    int root = fast_pow(g, c, Mod);
    int root_1 = inverse(root, Mod);
    printf("mod = %d, root = %d, root_1 = %d root_pw = %d\n"
, Mod, root, root_1, n);
}

```

```

    assert(fast_pow(root, n, Mod) == 1);
    // g^c
}
/* FIM */

inline void ntt(vector<int>& a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j) swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        int wlen = invert ? root_1 : root;

        for (int i = len; i < root_pw; i <<= 1)
            wlen = 1LL * wlen * wlen % mod;

        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i + j], v = 1LL * a[i + j + len / 2] * w % mod;
                a[i + j] = u + v < mod ? u + v : u + v - mod;
                a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + mod;
                w = 1LL * w * wlen % mod;
            }
        }
    }

    if (invert) {
        int n_1 = inverse(n, mod);
        for (int& x : a)

```

```

        x = 1LL * x * n_1 % mod;
    }
}

vector<int> multiply(vector<int> const& a, vector<int> const
& b) {
    vector<int> ca(a.begin(), a.end()), cb(b.begin(), b.end
());
    int n = 1;
    while (n < (int)max(a.size(), b.size()))
        n <<= 1;
    n <<= 1;

    ca.resize(n);
    cb.resize(n);

    ntt(ca, false);
    ntt(cb, false);

    for (int i = 0; i < n; i++)
        ca[i] = 1LL * ca[i] * cb[i] % mod;
    ntt(ca, true);

    return ca;
}; // namespace NTT

```

Para módulos da forma $p = c2^k + 1$ (com p primo), g^c é uma 2^k -ésima raiz da unidade, em que g é uma raiz primitiva módulo p .

Atenção: Se em algum momento ela tentar calcular um fft de um vetor de tamanho maior que 2^k , vai bugar!

Como calcular NTT com módulo M arbitrário? Uma opção é usar teorema chinês dos restos em primos da forma $c2^k + 1$

Uma outra opção é decompor os polinômios $A(x)$ e $B(x)$ em $A = A_1 + A_2 \cdot C$ e $B = B_1 + B_2 \cdot C$ em que $C \approx \sqrt{M}$ (de modo que os coeficientes de cada um deles sejam pequenos) e daí

$$A \cdot B = A_1 \cdot B_1 + (A_1 \cdot B_2 + A_2 \cdot B_1) \cdot C + (A_2 \cdot B_2) \cdot C^2$$

ou seja, com isso, os coeficientes calculados (de cada produto $A_i B_j$) não passam de $n \cdot M$.

Problema 7.13.1: Número de pares de números com uma dada soma.

Solução: Eleve ao quadrado o polinômio de frequências.

Problema 7.13.2: Produto interno entre array a e shifts cíclicos de array b

Solução: Reverte a, completa com zeros, appenda b em b e calcula o produto entre os novos a e b. Na array c, $c[k] = \text{produto escalar entre a e o left shift cíclico } k-(n-1) \text{ de b (com k indo de } n-1 \text{ a } 2n-2)$.

```

2dd44c
// NAO E exatamente esse problema
// recebo duas fitas de DNA |s| >= |r|
// determinar r.size() - maior produto interno
// ou seja eu vou deslizando r em s (sem passar dos limites)
#include <bits/stdc++.h>

using namespace std;

using cd = complex<double>;
const double PI = acos(-1);
typedef long long int lli;

void fft(vector<cd>& a, bool invert) {}

vector<lli> multiply(vector<lli> const& a, vector<lli> const
    & b) {}

int conversion_map[200];

int main() {
    string s;
    string r;
    cin >> s;
    cin >> r;
    conversion_map['A'] = 0;
    conversion_map['C'] = 1;
    conversion_map['T'] = 2;
    conversion_map['G'] = 3;

    vector<lli> polys[4];
    vector<lli> polys_r[4];
    vector<lli> prods[4];
    for (int i = 0; i < s.size(); i++) {

```

```

        int atual = conversion_map[s[i]];
        for (int j = 0; j < 4; j++) {
            polys[j].push_back(atual == j);
        }

    for (int i = 0; i < r.size(); i++) {
        int atual = conversion_map[r[i]];
        for (int j = 0; j < 4; j++) {
            polys_r[j].push_back(atual == j);
        }

        for (int j = 0; j < 4; j++) {
            reverse(polys_r[j].begin(), polys_r[j].end());
            prods[j] = multiply(polys[j], polys_r[j]);
            if (prods[j].size() < s.size() - 1) prods[j].resize(
                s.size() - 1);
        }

        int max_ans = -1;

        for (int i = r.size() - 1; i <= s.size() - 1; i++) {
            int curr_ans = 0;
            for (int j = 0; j < 4; j++) {
                curr_ans += prods[j][i];
            }
            if (curr_ans > max_ans) {
                max_ans = curr_ans;
            } else if (curr_ans < max_ans)
                continue;
        }

        cout << r.size() - max_ans << "\n";
    }
}

```

Problema 7.13.3: Temos duas fitas de booleanos a e b. Queremos encontrar o número de maneiras de aplicar uma na outra de modo que nenhum par de uns estejam na mesma posição.

Solução: análoga a 7.13.2.

Problema 7.13.4: Dado um texto T , um padrão P (os 2 de letra minúscula),

calcular todas as ocorrências do padrão no texto.

Solução: Criamos

$$A(x) = a_0 + \dots + a_{n-1}x^{n-1}, n = |T|$$

com $a_i = e^{i\alpha_i}$ onde $\alpha_i = \frac{2\pi T[i]}{26}$

$$B(x) = b_0 + \dots + b_{m-1}x^{m-1}, m = |P|$$

com $b_i = e^{-i\beta_i}$ onde $\beta_i = \frac{2\pi P[m-i-1]}{26}$

Agora, considere $C(x) = A(x) \cdot B(x)$. Se $c_{m-1-i} = m$, é porque teve um match a partir de $T[i]$.

Problema 7.13.5: Permitindo wildcards no padrão (letras que dão match com qualquer uma)

Solução: basta setar $b_i = 0$ sempre que $P[m-i-1] = *$. Aí se o padrão tiver x wildcards, então $c_{m-1-i} = m - x$ indica que a partir do índice $T[i]$ coincidem.

7.13.2 FFT 2D

```

284bd1
const int B = 12; // nesse problema os pols envolvidos
                tinham no max x^800 y^800
const int N = 1 << B;

cd roots[N];
int inv[N];

cd get_root(int k, int n) { return roots[k * (N / n)]; }

void precalc() {
    for (int i = 0; i < N; i++) {
        double ang = 2 * PI * i / N;
        roots[i] = cd(cos(ang), sin(ang));
    }
}

bool bit(int mask, int i) { return (mask >> i) & 1; }

void precalc_inv(int n) {

```

```

    int b = 0;
    while ((1 << b) < n)
        ++b;
    assert((1 << b) == n);
    assert(b <= B);

    inv[0] = 0;
    int hb = -1;
    for (int i = 1; i < n; ++i) {
        if (bit(i, hb + 1)) {
            ++hb;
        }
        inv[i] = inv[i ^ (1 << hb)] ^ (1 << (b - hb - 1));
    }
}

// ****copia e cola fft aq

void fft_2d(vector<vector<cd>>& a, bool rev) {
    precalc_inv((a[0].size()));
    for (auto& row : a) {
        fft(row, rev);
    }
    precalc_inv((a.size()));
    for (int j = 0; j < a.front().size(); j++) {
        vector<cd> col;
        for (int i = 0; i < a.size(); i++)
            col.push_back(a[i][j]);
        fft(col, rev);
        for (int i = 0; i < a.size(); i++)
            a[i][j] = col[i];
    }
}

vector<vector<cd>> mult(vector<vector<cd>> x, vector<vector<
cd>> y) {
    int b_rows = 0;
    while ((1 << b_rows) <= max(x.size(), y.size()))
        ++b_rows;
    ++b_rows;

```

```

int b_cols = 0;
while ((1 << b_cols) <= max(x.front().size(), y.front().size()))
    ++b_cols;
++b_cols;

x.resize(1 << b_rows);
y.resize(1 << b_rows);
for (auto& row : x) {
    row.resize(1 << b_cols, 0);
}
for (auto& row : y) {
    row.resize(1 << b_cols, 0);
}

fft_2d(x, 0);
fft_2d(y, 0);

for (int i = 0; i < x.size(); i++)
    for (int j = 0; j < x[i].size(); j++)
        x[i][j] *= y[i][j];

fft_2d(x, 1);

return x;
}

```

7.13.3 Fast Walsh–Hadamard

No FFT usual, se tivermos multiplicando $P(x) \cdot Q(x)$, podemos enxergar isso como o incremento de $a_i b_j x^{i+j}$ no resultado.

No entanto, é possível também fazer os seguintes três tipos de incrementações:

1. $a_i b_j x^{i \oplus j}$ (xor)
2. $a_i b_j x^{i \wedge j}$ (and)
3. $a_i b_j x^{i \vee j}$ (or)

Ou seja, meio que temos uma convolução lógica. O funcionamento é assim: você tem um polinômio de grau n (potência de 2) e você sabe construir uma matriz T_n que satisfaz o seguinte:

$$T_n(P\$Q) = T_n(P) \cdot T_n(Q)$$

em que P e Q são os polinômios que aqui são enxergados como vetores de n entradas, o símbolo $\$$ se refere à convolução lógica escolhida (entre as três que forneci) e o $\cdot$ representa o produto interno usual.

Assim, basta multiplicar por T_n^{-1} na esquerda para acharmos nossa convolução lógica $P\$Q$.

A propósito, vale que $(P\$Q)\$R = P\$(Q\$R)$, independente da $\$$ entre as três que forneci.

Espero que você nunca tenha que usar que:

$$T_1^\oplus = (1), \quad T_n^\oplus = \begin{pmatrix} T_{n/2}^\oplus & T_{n/2}^\oplus \\ T_{n/2}^\oplus & -T_{n/2}^\oplus \end{pmatrix}, \quad (T_n^\oplus)^{-1} = \frac{T_n^\oplus}{n}$$

$$T_1^\wedge = (1), \quad T_n^\wedge = \begin{pmatrix} 0 & T_{n/2}^\wedge \\ T_{n/2}^\wedge & -T_{n/2}^\wedge \end{pmatrix}, \quad (T_n^\wedge)^{-1} = \begin{pmatrix} -T_{n/2}^\wedge & T_{n/2}^\wedge \\ T_{n/2}^\wedge & 0 \end{pmatrix}$$

$$T_1^\vee = (1), \quad T_n^\vee = \begin{pmatrix} T_{n/2}^\vee & T_{n/2}^\vee \\ T_{n/2}^\vee & 0 \end{pmatrix}, \quad (T_n^\vee)^{-1} = \begin{pmatrix} \left(T_{n/2}^\vee\right)^{-1} & -\left(T_{n/2}^\vee\right)^{-1} \\ \left(T_{n/2}^\vee\right)^{-1} & 0 \end{pmatrix}$$

2dc31e

```

const ll rev = inverse(2);

void FWT(vector<ll>& a, int n) {
    for (int d = 1; d < n; d <= 1)
        for (int m = d < 1, i = 0; i < n; i += m)
            for (int j = 0; j < d; j++) {
                int x = a[i + j], y = a[i + j + d];
                a[i + j] = (x + y) % mod, a[i + j + d] = (x
                    - y + mod) % mod;
                // xor : a[i+j]=x+y, a[i+j+d]=x-y;
                // and : a[i+j]=x+y;
                // or : a[i+j+d]=x+y;
            }
}

void UFWT(vector<ll>& a, int n) {
    for (int d = 1; d < n; d <= 1)
        for (int m = d < 1, i = 0; i < n; i += m)

```

```

        for (int j = 0; j < d; j++) {
            int x = a[i + j], y = a[i + j + d];
            a[i + j] = 1LL * (x + y) * rev % mod,
                a[i + j + d] = (1LL * (x - y) * rev %
                    mod + mod) % mod;
            // xor:a[i+j]=(x+y)/2,a[i+j+d]=(x-y)/2;
            // and:a[i+j]=x-y;
            // or:a[i+j+d]=y-x;
        }
    }

// tambem da pra usar pra or (eu acho :3)
vector<ll> mul_xor(const vector<ll>& v1, const vector<ll>&
    v2) {
    vector<ll> p1(v1.begin(), v1.end());
    vector<ll> p2(v2.begin(), v2.end());
    vector<ll> res;
    int novosize = 1;
    while (p1.size() + p2.size() > novosize) {
        novosize <= 1;
    }
    p1.resize(novosize);
    p2.resize(novosize);
    res.resize(novosize);
    FWI(p1, novosize);
    FWI(p2, novosize);
    for (int i = 0; i < novosize; i++)
        res[i] = (p1[i] * p2[i]) % mod;
    UFWI(res, novosize);
    if (novosize > size_lim) res.resize(size_lim);
    return res;
}

// size_lim = 1<<16 no problema que resolvi
// isso aqui multiplica uma lista de vetores
vector<ll> mul_and(vector<vector<ll>>& vector_list) {
    vector<ll> la_respuesta(size_lim, 1);
    for (int i = 0; i < vector_list.size(); i++) {
        vector_list[i].resize(size_lim);
        FWI(vector_list[i], size_lim);
    }
}

```

```

        for (int j = 0; j < size_lim; j++)
            la_respuesta[j] = (la_respuesta[j] * vector_list
                [i][j]) % mod;
    }
    UFWI(la_respuesta, size_lim);
    return la_respuesta;
}

```

7.14 Métodos numéricos

7.14.1 Busca ternária

373e7a

```

double ternary_search(double l, double r) {
    double eps = 1e-9; // set the error limit here
    while (r - l > eps) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        double f1 = f(m1); // evaluates the function at m1
        double f2 = f(m2); // evaluates the function at m2
        if (f1 < f2)
            l = m1;
        else
            r = m2;
    }
    return f(l); // return the maximum of f(x) in [l, r]
}

```

7.14.2 Integração de Simsom

4b333b

```

const int N = 1000 * 1000; // number of steps (already
    multiplied by 2)

double simpson_integration(double a, double b) {
    double h = (b - a) / N;
    double s = f(a) + f(b); // a = x_0 and b =
        x_{2n}
    for (int i = 1; i <= N - 1; ++i) { // Refer to final
        Simpson's formula
    }
}

```

```

double x = a + h * i;
s += f(x) * ((i & 1) ? 4 : 2);
}
s *= h / 3;
return s;
}

```

7.15 Miscelânea

7.15.1 Números harmônicos

$$H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$$

$$\lim_{n \rightarrow \infty} H_n - \ln(n) = \gamma \approx 0.5772$$

Problema 7.15.1: Dado n inteiro positivo, qual um upper bound pro tamanho do conjunto

$$D_n = \left\{ \left\lfloor \frac{n}{k} \right\rfloor \mid 1 \leq k \leq n \right\}$$

Solução: $2\sqrt{n}$. Isso porque se $i > \sqrt{n}$, $\left\lfloor \frac{n}{i} \right\rfloor < \frac{n}{i} < \sqrt{n}$.

Problema 7.15.2: Seja $q \in S_n$. Seja l o menor inteiro positivo tal que $\lfloor n/l \rfloor = q$. Seja r o maior inteiro positivo tal que $\lfloor n/r \rfloor = q$. Então,

$$r = \left\lfloor \frac{n}{\left\lfloor \frac{n}{l} \right\rfloor} \right\rfloor$$

Solução: A prova segue da aplicação do algoritmo da divisão em l e num l' que satisfaz a propriedade que l e r satisfazem (i.e. $l \leq l' \leq r$). E isso acaba provando (sse) que $l' \leq n/q$. Então o r seria $\lfloor n/q \rfloor$ e substituir q por $\lfloor n/l \rfloor$ nos dá o que queríamos.

Baseado nesses fatos, o seguinte for itera pelo conjunto D_n .

5305c7

```

// O(sqrt(n))
for (ll l = 1, r = 1; (n / l); l = r + 1) {
    r = (n / (n / l));
    // q = (n/l), process the range [l, r]
}

```

Sejam $m, x \in \mathbb{R}$ e $n > 0$ inteiro. Então,

$$\left\lfloor \frac{\lfloor x/m \rfloor}{n} \right\rfloor = \left\lfloor \frac{x}{mn} \right\rfloor$$

$$\left\lceil \frac{\lceil x/m \rceil}{n} \right\rceil = \left\lceil \frac{x}{mn} \right\rceil$$

7.15.2 Funções Multiplicativas II

Um monoide é um conjunto equipado de uma operação binária e de uma identidade. A operação binária deve ser associativa. Seja S um monoide. Uma função $f : \mathbb{N}^* \rightarrow S$ é dita aritmética multiplicativa se

$$f(a \cdot b) = f(a) \cdot f(b), \quad \forall a, b \in \mathbb{N}^* \text{ com } \gcd(a, b) = 1$$

Em situações assim, é só obter cada $f(p^k)$.

Para o caso particular em que S é um grupo abeliano (visto como um $\mathbb{Z}$ -módulo), então se $f, g : \mathbb{N}^* \rightarrow S$ (repare que não falei nada sobre serem multiplicativas), temos que

$$g(n) = \sum_{d|n} f(d), \quad \forall n \in \mathbb{N}^* \implies f(n) = \sum_{d|n} \mu(d)g(n/d), \quad \forall n \in \mathbb{N}^*$$

(em que os somatórios iteram pelos divisores positivos). Isso é a chamada **inversão de Möbius**.

A **convolução de Dirichlet** entre duas funções $f, g : \mathbb{N}^* \rightarrow \mathbb{C}$ é uma nova função $f * g : \mathbb{N}^* \rightarrow \mathbb{C}$:

$$(f * g)(n) = \sum_{d|n} f(d)g(n/d)$$

Sejam $f, g, h : \mathbb{N}^* \rightarrow \mathbb{C}$. Valem as seguintes propriedades

$$f * g = g * f$$

$$f * (g * h) = (f * g) * h$$

$$f * (g + h) = f * g + f * h$$

$$f, g \text{ multiplicativas} \implies f * g \text{ multiplicativa}$$

Denotamos por $\varepsilon : \mathbb{N}^* \rightarrow \mathbb{C}$ a função que leva 1 em 1 e qualquer $n \neq 1$ em 0. Ela é daora de mencionar, porque $f * \varepsilon = f$ para toda f aritmética. Além disso, se $f(1) \neq 0$, existe uma função $f^{-1} : \mathbb{N}^* \rightarrow \mathbb{C}$ tal que $f * f^{-1} = \varepsilon$. Outras funções interessantes de serem mencionadas:

$$1. \ 1(n) := 1, \quad \forall n \in \mathbb{N}^*$$

2. $\text{Id}_k(n) = n^k, \quad \forall n \in \mathbb{N}^*$
3. $1 * \mu = \varepsilon$
4. $g = f * 1 \iff f = g * \mu$
5. $\sigma_k = \text{Id}_k * 1$ (soma dos divisores elevados a k)
6. $\tau = 1 * 1$ (número de divisores)
7. $\varphi = \text{Id} * \mu$ (totiente de euler)
8. λ é a função de Lliouville. É 1 se for um número par de primos e -1 , se ímpar.
9. $\lambda * |\mu| = \varepsilon$
10. $\lambda * 1 = 1_{\text{Sq}}$
11. $\text{Id}_k * (\text{Id}_k \mu) = \varepsilon$
12. $\tau^3 * 1 = (\tau * 1)^2$
13. J_k é a função totiente de Jordan. É o número de k tuplas de inteiros positivos $\leq n$ e que, juntos com n , formam um conjunto coprimo com $k + 1$ elementos.
14. $J_k * 1 = \text{Id}_k$
15. $(\text{Id}_s J_r) * J_s = J_{s+r}$
16. Λ é a função de Mangoldt. É $\log p$ se n for potência de um primo p , 0 caso contrário.
17. $\Lambda * 1 = \log$
18. $\omega(n)$ é o número de fatores primos distintos de n . $\Omega(n)$ é o número total de fatores primos de n .
19. $|\mu| * 1 = 2^\omega$
20. $\Omega * \mu = 1_{\mathcal{P}}$, a função característica de potências de primos.
21. $\omega * \mu = 1_{\mathbb{P}}$, a função característica de primos.

O inverso de Dirichlet pode ser calculado recursivamente:

$$f^{-1}(1) = \frac{1}{f(1)}, \quad f^{-1}(n) = \frac{-1}{f(1)} \sum_{d|n, d \neq n} f(n/d) f^{-1}(d) \text{ para } n > 1$$

O inverso de Dirichlet de uma função multiplicativa é multiplicativa. $(f * g)^{-1} = g^{-1} * f^{-1}$.

Seja h uma função completamente multiplicativa. Então, $(f * g)h = (fh) * (gh)$.

Soma parcial de funções multiplicativas

Técnica 1: Convolução

Seja $f : \mathbb{N}^* \rightarrow \mathbb{C}$ multiplicativa. Então, definimos $s_f : \mathbb{N}^* \rightarrow \mathbb{C}$ como

$$s_f(x) = \sum_{k=1}^x f(k)$$

Como calcular $s_f(n)$ rápido? Imagina que eu encontrei g tal que você sabe calcular $s_{f*g}(n)$ e $s_g(n)$ em $O(1)$. Fazendo continha, obtemos que

$$s_f(n) = \frac{s_{f*g}(n) - \sum_{d=2}^n s_f\left(\left\lfloor \frac{n}{d} \right\rfloor\right) g(d)}{g(1)}$$

Note que se eu já tiver computado todos os s_f até determinado ponto, conseguimos calcular s_f em $O(\sqrt{n})$.

Assim, a complexidade total fica $O(\sum_{i=1}^{\sqrt{n}} \sqrt{i}) + O(\sum_{i=1}^{\sqrt{n}} \sqrt{\frac{n}{i}}) = O(n^{3/4})$

Mas até agora nem usamos que f é multiplicativa. Podemos usar isso para calcular os k primeiros valores de f (e, portanto, de s_f) em $O(k)$ usando um crivo linear,

de modo que a complexidade total vá para $O(k) + O\left(\sum_{i=1}^{\frac{n}{k}} \sqrt{\frac{n}{i}}\right) = O(k) + O\left(\frac{n}{\sqrt{k}}\right)$

Acontece que, pondo $k = n^{2/3}$, essa complexidade é otimizada e vai para $O(n^{2/3})$.

A princípio, não temos nem garantia de que uma função assim exista. Mas dá pra pensar assim: quais funções dá pra calcular a soma parcial de boa? Bom, algumas coisas que podemos utilizar são

$$1. \varepsilon = \mu * 1$$

$$2. \text{Id} = \phi * 1$$

Técnica 2: DP

Você quer calcular rápido $\sum_{k=1}^n f(k)$ pra $n \approx 10^9$.

Aqui um código pra $f(p^q) = \binom{q+k+d}{d}$. Teoricamente roda em $O(n^{2/3})$.

A ideia é fazer uma $dp(n, x) = \sum_{i=1, \text{spf}(i) \geq x}^n f(i)$

$$dp(n, x) = \begin{cases} 0 & \text{se } x = 0 \\ 1 & \text{se } p > x \\ dp(n, p+1) & \text{se } p \text{ não é primo} \\ \sum_{i=0 \text{ ate } p^i \leq x} dp\left(\left\lfloor \frac{x}{p^i} \right\rfloor, p+1\right) f(p^i) & \text{caso contrário} \end{cases}$$

8b90f7

```
#include <bits/stdc++.h>
#define ll long long
#define pb push_back
#define mp make_pair
#define F first
#define S second
#define pii pair<int, int>
#define pll pair<ll, ll>
#define pcc pair<char, char>
#define vi vector<int>
#define vl vector<ll>
#define sd(x) scanf("%d", &x)
#define slld(x) scanf("%lld", &x)
#define pd(x) printf("%d", x)
#define plld(x) printf("%lld", x)
#define pds(x) printf("%d ", x)
#define pllds(x) printf("%lld ", x)
#define pdn(x) printf("%d\n", x)
#define plldn(x) printf("%lld\n", x)
using namespace std;
ll powmod(ll base, ll exponent, ll mod) {
    ll ans = 1;
    if (base < 0) base += mod;
    while (exponent) {
        if (exponent & 1) ans = (ans * base) % mod;
        base = (base * base) % mod;
        exponent /= 2;
    }
}
```

```

    }
    return ans;
}
ll gcd(ll a, ll b) {
    if (b == 0)
        return a;
    else
        return gcd(b, a % b);
}
const int INF = 2e9;
const ll INFLL = 4e18;
const int small_lim = 1e6 + 1;
const int mod = 1e9 + 7;
const int big_lim = 1e3 + 1;
ll primes_till_i[small_lim];
ll primes_till_bigger_i[big_lim];
vl sieved_primes[small_lim];
vl sieved_primes_big[big_lim];
vi prime;
int N, k, d;
void sieve() {
    vi lpf(small_lim);
    ll pw;
    for (int i = 2; i < small_lim; i++) {
        if (!lpf[i]) {
            prime.pb(i);
            lpf[i] = i;
        }
        for (int j : prime) {
            if ((j > lpf[i]) || (j * i >= small_lim)) break;
            lpf[j * i] = j;
        }
    }
    for (int i = 2; i < small_lim; i++) {
        primes_till_i[i] = primes_till_i[i - 1] + (lpf[i] ==
            i);
    }
}
ll count_primes(ll n, int ind) {
    if (ind < 0) return n - 1;
}
```

```

    if (1ll * prime[ind] * prime[ind] > n) {
        if (n < small_lim) return primes_till_i[n];
        if (primes_till_bigger_i[N / n]) return
            primes_till_bigger_i[N / n];
        int l = -1, r = ind;
        while (r - l > 1) {
            int mid = (l + r) >> 1;
            if (1ll * prime[mid] * prime[mid] > n)
                r = mid;
            else
                l = mid;
        }
        return primes_till_bigger_i[N / n] = count_primes(n,
            l);
    }
    int sz;
    if (n < small_lim)
        sz = sieved_primes[n].size();
    else
        sz = sieved_primes_big[N / n].size();
    ll ans;
    if (sz <= ind) {
        ans = count_primes(n, ind - 1);
        ans -= count_primes(n / prime[ind], ind - 1);
        ans += ind;
        if (n < small_lim)
            sieved_primes[n].pb(ans);
        else
            sieved_primes_big[N / n].pb(ans);
    }
    if (n < small_lim)
        return sieved_primes[n][ind];
    else
        return sieved_primes_big[N / n][ind];
}
ll count_primes(ll n) {
    if (n < small_lim) return primes_till_i[n];
    if (primes_till_bigger_i[N / n]) return
        primes_till_bigger_i[N / n];
    return count_primes(n, prime.size() - 1);
}

```

```

}

const int ncrlim = 3.5e6;

int fact[ncrlim];
int invfact[ncrlim];

void init_fact() {
    fact[0] = 1;
    for (int i = 1; i < ncrlim; i++)
        fact[i] = (1ll * fact[i - 1] * i) % mod;
    invfact[ncrlim - 1] = powmod(fact[ncrlim - 1], mod - 2,
        mod);
    for (int i = ncrlim - 1; i > 0; i--)
        invfact[i - 1] = (1ll * invfact[i] * i) % mod;
}

int ncr(int n, int r) {
    if (r > n || r < 0) return 0;
    int ans = fact[n];
    ans = (1ll * ans * invfact[n - r]) % mod;
    ans = (1ll * ans * invfact[r]) % mod;
    return ans;
}

ll calculate_dp(ll n, int ind) {
    if (n == 0) return 0;
    if (prime[ind] > n) return 1;
    ll ans = 1, temp;
    if (1ll * prime[ind] * prime[ind] > n) {
        temp = ncr(k + d, d);
        temp *= count_primes(n) - ind;
        ans += temp;
        ans %= mod;
        return ans;
    }
    ans = 0;
    ll gg = 1;
    ll mult = d;
    while (gg <= n) {

```

```
        temp = calculate_dp(n / gg, ind + 1);
        temp *= ncr(mult, d);
        ans += temp;
        ans %= mod;
        mult += k;
        gg *= prime[ind];
    }
    return ans;
}

int main() {
    sieve();
    init_fact();
    int t;
    sd(t);
    while (t--) {
        sd(N);
        sd(k);
        sd(d);
        plldn(calculate_dp(N, 0));
        for (int i = 1; i < big_lim; i++) {
            primes_till_bigger_i[i] = 0;
            sieved_primes_big[i].clear();
        }
    }
    return 0;
}
```

8 Combinatória

8.1 Fatoriais e binomiais

Problema 8.1.1: Como calcular $C(n, k) = \binom{n}{k} = \frac{n!}{k!(n-k)!}$?

Solução: Você pode calcular cada fatorial. Você pode calcular o produto a partir de um certo ponto no numerador e depois dividir pelo fatorial que sobrou. Você pode utilizar a recorrência $C(n, k) = C(n-1, k-1) + C(n-1, k)$. Você pode ter os fatoriais precomputados e calcular em $O(1)$.

Propriedades:

$$\binom{n}{k} = \binom{n}{n-k}$$

$$\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1}$$

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

$$\sum_{k=0}^n \binom{n}{k} = 2^n$$

$$\sum_{m=0}^n \binom{m}{k} = \binom{n+1}{k+1}$$

$$\sum_{k=0}^m \binom{n+k}{k} = \binom{n+m+1}{m}$$

$$\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}$$

$$\sum_{k=0}^n k \binom{n}{k} = n \cdot 2^{n-1}$$

$$\sum_{k=0}^n \binom{n-k}{k} = F_{n+1}$$

$$(a+b)^n = \sum_{k=0}^n \binom{n}{k} a^k b^{n-k}$$

$$\binom{m+n}{r} = \sum_{k=0}^r \binom{m}{k} \binom{n}{r-k}$$

$$\sum_{k=0}^m (-1)^k \binom{m}{k} = (-1)^m \binom{n-1}{m}$$

Obs.: Se o "denominador" do coeficiente binominal for maior que o "numerador", definimo-lo como zero.

Problema 8.1.2: Calcular $C(n, k) \pmod{m}$.

Solução: fatorar m em potências de primo e juntar tudo com o teorema chinês dos restos. Para achar $C(n, k) \pmod{p^e}$, seja $c(x)$ o maior inteiro tal que $p^{c(x)} | x!$.

Seja $g(x) = \frac{x!}{p^{c(x)}}$. Você pode calcular os g e os c com dp por exemplo. Aí no binomial você teria que lidar com

$$\frac{g(n)}{g(k)g(n-k)} p^{c(n)-c(k)-c(n-k)}$$

Como os g agora são coprimos com p dá pra calcular de boa o mod deles por potência de primo. E, quanto à outra parte, é só multiplicar no final.

Problema 8.1.3: $C(n, k) \pmod{m}$ pra n grande e m pequeno.

Solução 1 (m primo): Usa as técnicas do problema 2, que leva $O(m \log_m n)$.

Solução 2 (m primo): Usa o teorema de Lucas. Esse teorema diz que se $n = n_l p^l + \dots + n_0$ e $k = k_l p^l + \dots + k_0$ forem a expressão de n e k na base p , então

$$\binom{n}{k} \equiv \prod_{i=0}^l \binom{n_i}{k_i} \pmod{p}$$

Solução 3 (m livre de quadrados, fatores pequenos): Usa o teorema de lucas e combina tudo com o teorema chinês dos restos.

Solução 4 (m qualquer): Usa o teorema de lucas generalizado e junta com o teorema chinês dos restos. Seja $(k!)_p$ o produto de todos os inteiros menores ou iguais a k não divisíveis por p . Suponha que queremos achar $\binom{n}{m}$ (com $r = n - m \geq 0$) módulo p^f . Escreva $n = n_0 + n_1 p + \dots + n_s p^s$ em base p . Seja N_j o menor resíduo positivo de $\lfloor n/p^j \rfloor \pmod{p^f}$ (de modo que $N_j = n_j + \dots + n_{j+f-1} p^{f-1}$). Defina de maneira análoga m_j, M_j, r_j, R_j . Seja e_j o número de índices $i \geq j$ tais que $n_i < m_i$. Então:

$$\frac{1}{p^{e_0}} \binom{n}{m} \equiv (\pm 1)^{e_{f-1}} \prod_{i=0}^s \frac{(N_i!)_p}{(M_i!)_p (R_i!)_p} \pmod{p^f}$$

onde o ± 1 é -1 , exceto no caso $p = 2$ e $f \geq 3$.

Problema 8.1.4: Você quer ir de (a, b) para (c, d) indo apenas pra cima/baixo/esq/dir. Qual o número de caminhos com a menor distância?

Solução: Seja $\Delta_x = |a - c|$ e $\Delta_y = |b - d|$. Então, a resposta é $\binom{\Delta_x + \Delta_y}{\Delta_x}$.

Problema 8.1.5: Você tem uma array de $n = 10^5$ e vai processar $m = 10^5$ queries que adicionam $\binom{k}{k}$, $\binom{k+1}{k}$, $\dots$ no intervalo $[l, r]$. Sabendo que $k \leq 100$, dizer como a array fica no final.

Solução: Faça mais 101 arrays de "subtrações" de prefixo. Diga que $vet[0][i] = a[i] - a[i - 1]$ e $vet[i][j] = vet[i - 1][j] - vet[i - 1][j - 1]$. Se uma query envolve um k arbitrário, incremente $vet[k][l]$ e decamente $vet[k][r + 1]$. Depois de em todas as queries fazer isso, para cada k , reconstrua a array original a partir de $vet[k]$. Você vai obter várias arrays modificadas. Cada uma delas considera todas as queries para determinado k agindo no vetor. Então você soma todas elas e ajusta.

Problema 8.1.6: Você quer saber para quantos/quais k o número $\binom{n}{k}$ é ímpar.

Solução: Pelo teorema de Lucas, para ser ímpar, a cada bit 1 de k , não pode corresponder um bit 0 de n . Em outras palavras, cada bit 0 de n corresponderá a um bit 0 de k . E pra cada bit 1 de n , o bit correspondente de k não importa. Daí, você pode iterar de 0 até n e pegar os k que satisfazem o que queremos com isso. E para pegar a quantidade é só 2^{n_1} onde n_1 é o número de bits 1 da representação decimal de n .

Problema 8.1.7: Dada uma string de parêntesis, como determinar o número de substrings (não necessariamente contíguas) iguais a $()$, $(())$, $((()))$, etc?

Solução: Para a string $(((((...))))))$ com x (e y), a resposta é $\binom{x+y}{x}$ (por ex via vandermonde). Agora, itere pelos (da string e para cada um deles, conte o número de substrings em que este é o último opening bracket. Para isso, suponha que tem x opening brackets (estritamente) antes dele e y closing brackets depois. Aí como esse (atual deve estar incluído na resposta, então queremos contar as substrings de $(((((...))))$. Então ponha uma lista no último opening bracket e vá somando as escolhas de $r - 1$ opening brackets e r closing brackets e some. Pela mesma lógica de vandermonde, agora a resposta é $\binom{x+y}{x+1}$.

Problema 8.1.8: Tem uma string s de $n \leq 10^6$ letras de a ate z. Cada conjunto de letras contíguas é uma "colônia" da especie de bactéria correspondente a essa letra (então número de espécies é menor ou igual a 26 e pode ter mais de uma colônia de uma dada espécie). A cada instante ocorre um ataque. Um ataque é tipo "aabb" vira "aaab" e é sempre um caractere. A pergunta é o número de possíveis estados que podem ocorrer. Por exemplo, se for "aab", temos "aab", "aaa", "abb", "bbb", então a resposta é 5;

Solução: Converta cada colonia para um caractere por ex "aaaabbbccc" vira "abc". É óbvio que uma string t de n caracteres é um estado válido se, e só se, ao eu converter cada colonia dela pra um caractere, isso é uma substring do "abc". Ou seja, dada uma substring de "abc", basta eu ver quantas strings t são tais

que a compressão dessa string t seja essa substring. Digamos que essa substring tenha m caracteres. É fácil contar as strings t porque são o número de soluções de $x_1 + \dots + x_m = n$ com $x_i \geq 1$. Ou seja, $\binom{n-1}{m-1}$. Então temos que contar o número de substrings (não necessariamente contíguas) de "abc" (isto é, da compressão de s) que não possuem dois caracteres consecutivos iguais e que tenham "m" caracteres para cada valor de "m". Dá pra fazer isso com uma $dp[c][l]$ - numero de substrings de tamanho l que terminam em c . $dp[c][l] = 1 + soma[c != ci] dp[c, l-1]$

Problema 8.1.9: Tem n pontos com coordenadas inteiras no plano. Pra cada ponto, ou você põe uma reta horizontal passando por ele, ou uma vertical, ou você não faz nada. Dadas as coordenadas, qual o número de desenhos?

Solução: Imagina que você tem um grafo em que os pontos são vértices e tem uma aresta se e só se tiverem o mesmo x ou o mesmo y . A resposta é claramente o produto das respostas pra cada componente. Em uma dada componente, se nela não tiver nenhum "ciclo" (não ciclo, mas 4 pontos que formam um retângulo paralelo aos eixos coordenados), então a resposta pra ela é $2^{x+y} - 1$. Se tiver, é 2^{x+y} , em que x é a quantidade de coordenadas x distintas e y analogamente.

Problema 8.1.10: Palitos e bolinhas. Você tem $x_i \geq 0$ e quer saber o número de soluções de $x_1 + \dots + x_n = N$.

Solução: $\binom{N+n-1}{n-1}$

Problema 8.1.11: Palitos e bolinhas estendido. Você tem $0 \leq x_i \leq r_i$ e quer saber o número de soluções de $x_1 + \dots + x_n = N$.

Solução: Argumenta por inclusão e exclusão no palitos e bolinhas normal. Vai chegar em

$$\sum_{S \subseteq \{1, 2, \dots, n\}} (-1)^{|S|} \binom{N+n-1 - \sum_{i \in S} r_i + 1}{n-1}$$

No caso particular em que $r_i = r$, temos

$$\sum_{k=0}^{\lfloor N/(r+1) \rfloor} (-1)^k \binom{n}{k} \binom{N - k(r+1) + n - 1}{n-1}$$

Problema 8.1.12: Você tem um sanduíche de $N \leq 10^{18}$ metros e quer cortar no menor número de pedaços com comprimento positivo que é tal que é possível que cada pedaço fique com tamanho $\leq K \leq 10^{18} \bmod M \leq 10^6$

Solução: O número de pedaços q satisfaz $\lceil N/q \rceil \leq K$. Na verdade queremos o menor q que satisfaz isso. Esse q é $\lceil N/K \rceil$. Se o tamanho do sanduíche fosse qK , a resposta seria 1. Mas o tamanho dele é N . Note que $qK - N$, ou seja, estamos descendo do menor múltiplo de K maior ou igual a N para o próprio N . Então essa diminuição é menor que K . Ou seja, basta resolvermos $x_1 + \dots + x_q = qK - N$ em que $x_i < K$ (mas esta é uma restrição desnecessária, visto que $qK - N < K$),

onde x_i representa o quanto tiramos do sanduíche i . Por isso, a resposta final é $\binom{qK-N+q-1}{q-1}$. Para calcular isso, temos binomiais muito grandes e arbitrários. Agora fatora o M e usa o teorema de lucas generalizado em cada potência de primo e junta tudo com o teorema chinês dos restos.

8.2 Números de catalão

Incluindo o zero,

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

de modo que os primeiros números sejam 1, 1, 2, 5, 14, 42, 132, 429, ...

O número C_n é a solução para:

1. Número de sequências de parêntesis balanceadas
2. Número de árvores binárias enraizadas completas com $n+1$ folhas. Uma árvore enraizada é cheia se cada vértice ou não tem filho ou tem dois filhos.
3. Número de maneiras de parentesear completamente $n+1$ fatores.
4. Número de triangulações de um polígono convexo de $n+2$ lados.
5. Número de maneiras de conectar $2n$ pontos num círculo para formar n cordas disjuntas.
6. Número de árvore binárias completas não isomórficas com n nós internos (aqueles que têm algum filho)
7. Número de caminhos de $(0,0)$ até (n,n) que não passam por cima da diagonal
8. Número de permutações de tamanho n que podem ser stack sorted, isto é. não tem $i < j < k$ satisfazendo $p_i < p_j < p_k$.
9. Número de partições non-crossing de um conjunto de n
10. Número de maneiras de cobrir a escala $1 \dots n$ usando n retângulos (a escada consiste de n colunas, onde a i -ésima possui altura i)

elementos

Recursivamente, podemos definir $C_0 = C_1 = 1$ e para $n \geq 2$:

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$$

8.3 Princípio da Inclusão e Exclusão

Sejam n um inteiro positivo e $A_1, \dots, A_n$ conjuntos. Temos

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{\emptyset \neq J \subset \{1,2,\dots,n\}} (-1)^{|J|-1} \left| \bigcap_{j \in J} A_j \right|$$

Problema 8.3.1: Dado n e r , determine o número de inteiros de 1 a r relativamente primos com n .

Solução: Sejam $p_1, \dots, p_k$ os fatores primos de n . O número de caras divisível por x no intervalo é $\lfloor r/x \rfloor$. Então faz inclusão exclusão com os primos. Demoramos $O(\sqrt{n})$ para fatorar e iteramos pelos 2^k subconjuntos de p 's.

65c916

```
int solve(int n, int r) {
    vector<int> p;
    for (int i = 2; i * i <= n; ++i)
        if (n % i == 0) {
            p.push_back(i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1) p.push_back(n);

    int sum = 0;
    for (int msk = 1; msk < (1 << p.size()); ++msk) {
        int mult = 1, bits = 0;
        for (int i = 0; i < (int)p.size(); ++i)
            if (msk & (1 << i)) {
                ++bits;
                mult *= p[i];
            }

        int cur = r / mult;
        if (bits % 2 == 1)
            sum += cur;
        else
            sum -= cur;
    }

    return r - sum;
}
```

}

Essa solução é $O(\sqrt{n})$.

Problema 8.3.2: Dados n números a_i e um número r , queremos os inteiros no intervalo $[1, r]$ que são múltiplos de pelo menos um a_i .

Solução: É quase idêntico ao problema anterior. Só que dada uma máscara precisamos pegar o lcm dos caras que compõem ela, o que leva $O(n \log r)$, de modo que a complexidade final seja $O(2^n n \log r)$.

Problema 8.3.3: Número de strings que satisfazem um padrão: são dados n padrões de mesmo comprimento que consistem de letras de a a z ou interrogações. A tarefa é contar o número de strings que dão match em exatamente k dos padrões (primeiro problema) e em pelo menos k dos padrões (segundo problema).

Solução: É fácil verificar o número de strings que satisfazem um determinado conjunto de padrões. Mas agora, fixe X um subconjunto dos padrões que consiste de k padrões. Temos que contar o número de strings que satisfazem apenas X e não satisfazem nenhum outro padrão. Para isso, usamos o princípio da inclusão e exclusão: percorra todos os Y que contenham X e faça inclusão e exclusão da seguinte maneira:

$$\text{ans}(X) = \sum_{Y \supseteq X} (-1)^{|Y|-k} f(Y)$$

onde $f(Y)$ é o número de strings que dão match em Y . E para pegar a resposta final:

$$\text{ans} = \sum_{X: |X|=k} \text{ans}(X)$$

Essa solução tem assintótica de $O(3^k \cdot k)$. Para melhorá-la note que cada $\text{ans}(X)$ compartilha computações acerca dos conjuntos Y . Dá pra transformar a fórmula nisso:

$$\text{ans} = \sum_{Y: |Y| \geq k} (-1)^{|Y|-k} \cdot \binom{|Y|}{k} \cdot f(Y)$$

de modo que tenhamos uma assintótica $O(2^k \cdot k)$. Para a segunda versão do problema, note que a parte multiplicando $f(Y)$ seria

$$(-1)^{|Y|-k} \binom{|Y|}{k} + \dots + (-1)^{|Y|-|Y|} \binom{|Y|}{|Y|}$$

mas uma de nossas identidades combinatórias simplifica isso para

$$(-1)^{|Y|-k} \binom{|Y|-1}{|Y|-k}$$

então podemos calcular tudo em $O(2^k \cdot k)$

$$\text{ans} = \sum_{Y: |Y| \geq k} (-1)^{|Y|-k} \binom{|Y|-1}{|Y|-k} f(Y)$$

Problema 8.3.4: Tem um campo $n \times m$ e k de suas células são paredes intransponíveis. Um robô tá na célula $(1, 1)$ e pode ir apenas para direita ou esquerda. Eventualmente ele precisa ir para a célula (n, m) evitando todos os obstáculos. Qual o número de maneiras que ele pode fazer isso? Assuma $n, m \leq 10^9$ e $k \leq 100$.

Solução: Coloque $(1, 1)$ no início e (n, m) no final da array de obstáculos. Seja $d[i]$ o número de maneiras de ir do ponto inicial até o obstáculo i da array sem pisar em nenhum outro obstáculo. Nossa resposta seria o último elemento de d . Primeiro, calcule normalmente o número de maneiras de ir da cela inicial até a cela i . Agora, desconte para cada $0 < t < i$ o número de maneiras de chegar até i passando por algum(ns) obstáculos, dentre os quais o primeiro foi o t . Esse número é $dp[t]$ multiplicado pelo número de caminhos para chegar até i . A complexidade final seria $O(k^2)$.

Problema 8.3.5: Número de quádruplas coprimas. Sejam $a_1, \dots, a_n$ números dados. Calcule o número de maneiras de escolher quatro números de modo que seu gcd é 1.

Solução: Computaremos o número de quádruplas em que o gcd é maior que um. Itere por todos os grupos de quatro números divisíveis por d . Temos:

$$\text{ans} = \sum_{d \geq 2} (-1)^{\deg(d)-1} f(d)$$

onde $\deg(d)$ é o número de primos na fatoração de d e $f(d)$ é o número de quádruplas divisíveis por d . Para calcular $f(d)$ só precisa contar o número de múltiplos de d e escolher quatro deles.

Problema 8.3.6: Número de triplas harmônicas. Seja $n \leq 10^6$. Você precisa computar o número de triplas $2 \leq a < b < c \leq n$ que satisfazem uma das seguintes condições:

1. ou $\gcd(a, b) = \gcd(a, c) = \gcd(b, c) = 1$
2. ou $\gcd(a, b) > 1, \gcd(a, c) > 1, \gcd(b, c) > 1$

Primeiramente, considere o problema ao contrário i.e. contar o número de triplas não harmônicas. Uma tripla não harmônica é composta por um par de coprimos e um terceiro número que não é coprimo com pelo menos um cara do par. Então o número de triplas não harmônicas que contêm i é igual ao número de inteiros de 2 até n que são coprimos com i multiplicado pelo número de inteiros que não são

coprimos com i . Mas temos que dividir por 2 por que fizemos contagens duplas para valores diferentes de i . Para calcular o número de coprimos a i no intervalo de 2 a n , uma solução mais rápida do que foi mencionada seria a seguinte:

1. Encontre todos os números em $[2, n]$ tais que sua fatoração não inclui nenhum fator primo duas vezes. Durante o crivo, podemos manter uma array $deg[i]$ que diz o número de primos na fatoração de i e uma array $good[i]$ que marca se cada i contém cada fator no máximo uma vez (1 ou 0). Ao iterar de 2 a n , se chegarmos em um número que tem $deg = 0$, então ele é primo e seu deg vira 1. Durante o crivo, vamos iterar i de 2 a n . Quando formos processar um primo, iteramos por todos os seus múltiplos e aumentamos o seu $deg[i]$. Se algum desses múltiplos for múltiplo do quadrado de i , colocamos seu $good$ como falso.
2. Precisamos calcular a resposta para todo i de 2 até n - os números não coprimos com i . Para isso, iteramos por um produto de primos da sua fatoração e adicionamos ou subtraímos seus múltiplos. Então digamos que estejamos processando um número i com $good[i] = 1$. Itere por todos os números que são múltiplos de i e adicione ou subtraia $\lfloor N/i \rfloor$ da $cnt[]$ de acordo com a paridade de $deg[i]$.

d14afa

```
int n;
bool good[MAXN];
int deg[MAXN], cnt[MAXN];

long long solve() {
    memset(good, 1, sizeof good);
    memset(deg, 0, sizeof deg);
    memset(cnt, 0, sizeof cnt);

    long long ans_bad = 0;
    for (int i = 2; i <= n; ++i) {
        if (good[i]) {
            if (deg[i] == 0) deg[i] = 1;
            for (int j = 1; i * j <= n; ++j) {
                if (j > 1 && deg[i] == 1)
                    if (j % i == 0)
                        good[i * j] = false;
                else
                    ++deg[i * j];
            }
        }
        ans_bad += (cnt[i] - 1) * 1ll * (n - 1 - cnt[i]);
    }

    return (n - 1) * 1ll * (n - 2) * (n - 3) / 6 - ans_bad / 2;
}
```

```
        cnt[i * j] += (n / i) * (deg[i] % 2 == 1 ?
        +1 : -1);
    }
}
ans_bad += (cnt[i] - 1) * 1ll * (n - 1 - cnt[i]);
}

return (n - 1) * 1ll * (n - 2) * (n - 3) / 6 - ans_bad / 2;
}
```

A assintótica dessa solução é $O(n \log n)$.

Problema 8.3.7: Número de permutações caóticas de n elementos (aquelas que não contêm pontos fixos):

Solução: É possível calcular arredondando $n!/e$ para o inteiro mais próximo ou então calculando de maneira exata com a fórmula:

$$n! - \binom{n}{1}(n-1)! + \binom{n}{2}(n-2)! + \cdots \pm \binom{n}{n}(n-n)!$$

Para chegar nisso, dava pra ter usado inclusão e exclusão nos conjuntos A_k de permutações de tamanho n com um ponto fixo na posição k .

8.4 Teoria dos grupos

Seja G um grupo finito que age no conjunto X .

Dado $x \in X$, a órbita de x , denotada por $G \cdot x$, é o subconjunto de X que pode ser alcançado por ações de G em x i.e. $G \cdot x = \{g \cdot x : g \in G\}$. Note também que $x \in G \cdot y \iff G \cdot x = G \cdot y$. Com isso, é possível dividir X em suas órbitas. Ou seja, podemos dizer que X/G é o conjunto de todas as órbitas de X sobre a ação de G .

Defina também o estabilizador de $x \in X$ como o conjunto dos $g \in G$ tais que x é um de seus pontos fixos i.e. $G_x = \{g \in G : g \cdot x = x\}$. Em particular, note que $\forall x \in X$, G_x é um subgrupo de G . É fácil ver que este é um subgrupo normal, por isso há sentido em definir G/G_x .

Teorema da órbita e do estabilizador:

$$\forall x \in X, G \cdot x \cong G/G_x$$

Corolário: No caso finito, pelo teorema de lagrange, temos $|G \cdot x| \cdot |G_x| = |G|$

Lema de Burnside: Defina $X^g = \{x \in X : g \cdot x = x\}$. Então

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

Se imaginarmos G como um grupo de permutações π que agem em X , podemos trocar o X^g por $I(\pi)$, o que representaria os elementos de x invariantes por determinada permutação.

Seja $C(\pi)$ o número de ciclos na decomposição em ciclos da permutação π . Seja Y um conjunto de cores. Seja Y^X o conjunto de funções $X \rightarrow Y$, isto é, que associam a cada elemento de X uma cor de Y .

Teorema da enumeração de Pólya:

$$|Y^X/G| = \frac{1}{|G|} \sum_{\pi \in G} |Y|^{C(\pi)}$$

Note que esse teorema é uma consequência do lema de Burnside, pois $|I(\pi)| = k^{C(\pi)}$.

Problema 8.4.1: Você tem k cores para colorir n bolinhas que vão formar um colar. Um colar é idêntico a outro se pode ser obtido por rotação. Qual a quantidade desses colares?

Solução: Seja $G \cong C_n$. Temos que G está agindo sobre colorações do conjunto X de bolas com um conjunto Y de cores i.e. G está agindo sobre Y^X . O número de colorações é $|Y^X/G|$, que é o que queremos calcular. $|G| = n$. $|Y| = k$. Seja $\pi \in G$ uma rotação simples do colar, de modo que queremos determinar $C(\pi^k)$. Seja $g = \gcd(n, k)$. Note que π^k é composta por ciclos de tamanho n/g e que existem g desses ciclos. Então nossa resposta é:

$$\frac{1}{n} \sum_{i=0}^{n-1} k^{\gcd(i, n)}$$

Dá pra reescrever isso considerando que agora passamos por todos os valores de $\gcd(i, n)$. Para cada divisor d de n , o número de i tais que $d = \gcd(i, n)$ é claramente $\phi(n/d)$. Por isso, outra forma de escrever seria

$$\frac{1}{n} \sum_{d|n} \phi(n/d) k^d$$

Problema 8.4.2: Sejam n, m inteiros positivos com $n < m$. Considere um retângulo $n \times m$ onde cada casinha é preta ou branca. Agora pegue o lado longo desse retângulo e junte com o outro formando o cilindro. Agora dobre o cilindro de maneira que os círculos de baixo/cima se conectem e tenhamos um toro. Qual o número de toros distintos que podemos obter? O toro pode ser girado em torno dos círculozinhos dele e também girado de fato.

Solução: Geralmente é difícil de obter um número explícito de classes de equivalência. Então a gente tem que fazer na marra. Pra isso, encontramos várias

permutações básicas que geram o grupo G . É fácil ver que G é gerado por π_1 , π_2 e π_3 , onde π_1 shifta ciclicamente todas as linhas, π_2 shifta ciclicamente todas as colunas e π_3 gira a folha 180 graus. Então lembrando que nesse caso temos $|Y| = 2$, podemos obter o número de classes de equivalência (resposta do problema) utilizando o teorema de Pólya:

03ec69

```
using Permutation = vector<int>;

void operator*=(Permutation& p, Permutation const& q) {
    Permutation copy = p;
    for (int i = 0; i < p.size(); i++)
        p[i] = copy[q[i]];
}

int count_cycles(Permutation p) {
    int cnt = 0;
    for (int i = 0; i < p.size(); i++) {
        if (p[i] != -1) {
            cnt++;
            for (int j = i; p[j] != -1;) {
                int next = p[j];
                p[j] = -1;
                j = next;
            }
        }
    }
    return cnt;
}

int solve(int n, int m) {
    Permutation p(n * m), p1(n * m), p2(n * m), p3(n * m);
    for (int i = 0; i < n * m; i++) {
        p[i] = i;
        p1[i] = (i % n + 1) % n + i / n * n;
        p2[i] = (i / n + 1) % m * n + i % n;
        p3[i] = (m - 1 - i / n) * n + (n - 1 - i % n);
    }

    set<Permutation> s;
    for (int i1 = 0; i1 < n; i1++) {
```

```

    for (int i2 = 0; i2 < m; i2++) {
        for (int i3 = 0; i3 < 2; i3++) {
            s.insert(p);
            p *= p3;
        }
        p *= p2;
    }
    p *= p1;
}

int sum = 0;
for (Permutation const& p : s) {
    sum += 1 << count_cycles(p);
}
return sum / s.size();
}

```

8.5 K-Combinações

Na lib padrão do c++, tem a função next_permutation:

```

66ac41
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec = {1, 2, 3};

    // Ordena o vetor em ordem crescente
    std::sort(vec.begin(), vec.end());

    do {
        // Imprime a permutacao atual
        for (int i : vec) {
            std::cout << i << " ";
        }
        std::cout << std::endl;
    } while (std::next_permutation(vec.begin(), vec.end()));
}

```

```

return 0;
}

```

Dados inteiros positivos N e K , determinar os subconjuntos de $\{1, \dots, N\}$ de tamanho exatamente K .

Problema 8.5.1: Determine a próxima K-combinação (considerando ordem lexicográfica) considerando a primeira como sendo $1, 2, \dots, K$

Solução: Em $O(K)$

```

38743b
bool next_combination(vector<int>& a, int n) {
    int k = (int)a.size();
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - k + i + 1) {
            a[i]++;
            for (int j = i + 1; j < k; j++)
                a[j] = a[j - 1] + 1;
            return true;
        }
    }
    return false;
}

```

Problema 8.5.2: Gere todas as K-combinações de modo que combinações adjacentes difiram por no máximo um elemento

Solução 1: Em $O(2^n)$. Usa o fato de que se você gerar os 2^n graycodes de N bits e pegar apenas os de K bits ativos, então a diferença entre caras consecutivos vai ser que de um pro outro um bit apaga e um bit acende.

```

b8c59f
int gray_code(int n) { return n ^ (n >> 1); }

int count_bits(int n) {
    int res = 0;
    for (; n; n >>= 1)
        res += n & 1;
    return res;
}

void all_combinations(int n, int k) {
    for (int i = 0; i < (1 << n); i++) {
        int cur = gray_code(i);
    }
}

```

```

        if (count_bits(cur) == k) {
            for (int j = 0; j < n; j++) {
                if (cur & (1 << j)) cout << j + 1;
            }
            cout << "\n";
        }
    }
}

```

Solução 2: Em $O\left(N \cdot \binom{N}{K}\right)$, mas com constante alta

```

97981e
vector<int> ans;

void gen(int n, int k, int idx, bool rev) {
    if (k > n || k < 0) return;

    if (!n) {
        for (int i = 0; i < idx; ++i) {
            if (ans[i]) cout << i + 1;
        }
        cout << "\n";
        return;
    }

    ans[idx] = rev;
    gen(n - 1, k - rev, idx + 1, false);
    ans[idx] = !rev;
    gen(n - 1, k - !rev, idx + 1, true);
}

void all_combinations(int n, int k) {
    ans.resize(n);
    gen(n, k, 0, false);
}

```

8.6 Extras

Problema 8.6.1: Encontre o número de maneiras de colocar K bispos num tabuleiro $n \times n$ de modo que não se ataquem

Solução: Numere as diagonais de maneira que as pretas tenham índices ímpares, as brancas tenham índices pares e as diagonais são numeradas em ordem do número de quadrados nela. Aqui um exemplo para um tabuleiro 5×5

1	2	5	6	9
2	5	6	9	8
5	6	9	8	7
6	9	8	7	4
9	8	7	4	3

Seja $d[i][j]$ o número de maneiras de colocar j bispos em diagonais com índices até o i e que têm a mesma cor da diagonal i . Dessa maneira, i varia de 0 a $2n - 1$ e j varia de 0 a k .

Podemos calcular $d[i][j]$ usando apenas os valores de $d[i-2]$. Tem duas maneiras de calcular. Ou coloca todos os j bispos em diagonais anteriores, de modo que tenham $d[i-2][j]$ maneiras de que isso aconteça. Ou então colocamos um bispo na diagonal i e $j-1$ bispos em diagonais anteriores. O número de maneiras de fazer isso é igual ao número de quadrados na diagonal i menos $j-1$, já que $j-1$ bispos colocados em diagonais anteriores bloquearão um quadrado na diagonal atual. O número de quadrados na diagonal i é fácil de calcular:

```

b2ef50
int squares(int i) {
    if (i & 1)
        return i / 4 * 2 + 1;
    else
        return (i - 1) / 4 * 2 + 2;
}

```

O caso base é simples: $d[i][0] = d[1][1] = 1$. Tendo calculado os $D[i][j]$ a resposta é a soma dos $D[2n-1][i] * D[2n-2][k-i]$.

```

a84f3e
int bishop_placements(int N, int K) {
    if (K > 2 * N - 1) return 0;

    vector<vector<int>> D(N * 2, vector<int>(K + 1));
    for (int i = 0; i < N * 2; ++i)
        D[i][0] = 1;
    D[1][1] = 1;
    for (int i = 2; i < N * 2; ++i)

```

```

    for (int j = 1; j <= K; ++j)
        D[i][j] = D[i - 2][j] + D[i - 2][j - 1] * (
            squares(i) - j + 1);

    int ans = 0;
    for (int i = 0; i <= K; ++i)
        ans += D[N * 2 - 1][i] * D[N * 2 - 2][K - i];
    return ans;
}

```

Problema 8.6.2: Encontrar a próxima sequência de parêntesis balanceada.

```

// inicialmente começa com (((...)))
// vai para a proxima em O(n)
// lembrando que sao C_n ao todo
bool next_balanced_sequence(string& s) {
    int n = s.size();
    int depth = 0;
    for (int i = n - 1; i >= 0; i--) {
        if (s[i] == '(')
            depth--;
        else
            depth++;

        if (s[i] == '(' && depth > 0) {
            depth--;
            int open = (n - i - 1 - depth) / 2;
            int close = n - i - 1 - open;
            string next = s.substr(0, i) + ')' + string(open, '(') + string(close, ')');
            s.swap(next);
            return true;
        }
    }
    return false;
}

```

Problema 8.6.3: Encontrar índice dessa sequência na lista de sequências

Solução: Seja $d[i][j]$ onde i é o tamanho da sequência (semi-balanceada, isto é, tem parêntese que falta fechar) e j é o balanço atual (diferença entre opening e closing brackets) o número de sequências que correspondem a esse i e esse j .

Inicialmente, $d[0][0] = 1$, $d[0][j] = 0$, $j > 0$ e depois

$$d[i][j] = d[i - 1][j - 1] + d[i - 1][j + 1]$$

Então em $O(n^2)$ fazemos essa dp. Para calcular o índice de uma dada sequência, façamos o seguinte. Primeiro fazemos o caso em que só tem um tipo de parêntesis. Temos um contador depth de profundidade que diz o quão nested nós estamos e iteramos pelos caracteres da sequência. Se $s[i] == ($, então incrementamos depth. Se o caractere atual for $)$, então incrementamos $d[2n - i - 1][depth + 1]$ na resposta e decrementamos depth.

Agora, se tiverem k tipos diferentes de brackets, então quando observamos o caractere atual $s[i]$, temos que iterar por todos os tipos de brackets que são menores que o caractere atual e tentar colocar esse bracket na sua posição atual (obtendo um novo balance $ndepth = depth \pm 1$) e daí adicionamos o número de maneiras de terminar a sequência (tamanho $2n - i - 1$, balanço ndepth) à resposta:

$$d[2n - i - 1][ndepth] \cdot k \frac{2n - i - 1 - ndepth}{2}$$

Problema 8.6.4: Encontrando a k -ésima sequência.

Solução: Seja n o número de pares de brackets na sequência. No problema anterior computamos $d[i][j]$. Primeiro, com apenas um tipo de bracket: nós iteramos pelos caracteres na string que queremos gerar. Aqui também tem um contador depth. Em cada posição queremos decidir se usamos um opening ou closing bracket. Para ser um opening tem que ter $d[2n - i - 1][depth + 1] \geq k$. Incrementamos o contador depth e vamos para o próximo. Caso contrário decrementamos k por $d[2n - i - 1][depth + 1]$, colocamos um closing bracket e continuamos

```

// 3d8508
string kth_balanced(int n, int k) {
    vector<vector<int>>> d(2 * n + 1, vector<int>(n + 1, 0));
    d[0][0] = 1;
    for (int i = 1; i <= 2 * n; i++) {
        d[i][0] = d[i - 1][1];
        for (int j = 1; j < n; j++)
            d[i][j] = d[i - 1][j - 1] + d[i - 1][j + 1];
        d[i][n] = d[i - 1][n - 1];
    }

    string ans;
    int depth = 0;
    for (int i = 0; i < 2 * n; i++) {

```

```

        if (depth + 1 <= n && d[2 * n - i - 1][depth + 1] >=
            k) {
            ans += '(';
            depth++;
        } else {
            ans += ')';
            if (depth + 1 <= n) k -= d[2 * n - i - 1][depth
                + 1];
            depth--;
        }
    }
    return ans;
}

```

Agora, se tiverem k tipos de brackets, multiplicamos o valor de $d[2n - i - 2n - i - 1 - ndepth]$ por $k^{\frac{2n - i - 1 - ndepth}{2}}$ e levamos em conta o fato de que podem existir tipos de brackets diferentes para o próximo caractere.

Aqui uma implementação para dois tipos de bracket

```

e4365d
string kth_balanced2(int n, int k) {
    vector<vector<int>> d(2 * n + 1, vector<int>(n + 1, 0));
    d[0][0] = 1;
    for (int i = 1; i <= 2 * n; i++) {
        d[i][0] = d[i - 1][1];
        for (int j = 1; j < n; j++)
            d[i][j] = d[i - 1][j - 1] + d[i - 1][j + 1];
        d[i][n] = d[i - 1][n - 1];
    }

    string ans;
    int shift, depth = 0;

    stack<char> st;
    for (int i = 0; i < 2 * n; i++) {

        // '('
        shift = ((2 * n - i - 1 - depth - 1) / 2);
        if (shift >= 0 && depth + 1 <= n) {
            int cnt = d[2 * n - i - 1][depth + 1] << shift;

```

```

            if (cnt >= k) {
                ans += '(';
                st.push('(');
                depth++;
                continue;
            }
            k -= cnt;
        }

        // ')'
        shift = ((2 * n - i - 1 - depth + 1) / 2);
        if (shift >= 0 && depth && st.top() == '(') {
            int cnt = d[2 * n - i - 1][depth - 1] << shift;
            if (cnt >= k) {
                ans += ')';
                st.pop();
                depth--;
                continue;
            }
            k -= cnt;
        }

        // '['
        shift = ((2 * n - i - 1 - depth - 1) / 2);
        if (shift >= 0 && depth + 1 <= n) {
            int cnt = d[2 * n - i - 1][depth + 1] << shift;
            if (cnt >= k) {
                ans += '[';
                st.push('[');
                depth++;
                continue;
            }
            k -= cnt;
        }

        // ']'
        ans += ']';
        st.pop();
        depth--;
    }
}

```

```

return ans;
}

```

Problema 8.6.5: Contando o número de grafos rotulados (isto é, cada vértice tem um rótulo de 1 a n) de n vértices. (não consideramos multi-edges nem directed edges).

Solução: Para cada conexão, decidimos se fazemos ela ou não: $G_n = 2^{n(n-1)/2}$;

Problema 8.6.6: Grafos rotulados conexos.

Solução: Digamos que a resposta é C_n . Primeiros focamos nos desconexos. Na verdade, primeiro focamos nos desconexos e enraizados. Um grafo enraizado é um em que eu enfatizo um vértice como sendo uma raiz (temos n possibilidades de escolher a raiz então é só dividir o número de grafos enraizados por n). O vértice raiz irá aparecer na componente conexa de tamanho $1, 2, \dots, n-1$. Existem $k \binom{n}{k} C_k G_{n-k}$ grafos tais que o vértice raiz está numa componente conexa com k vértices (existem n escolhe k maneiras de escolher k vértices para a componente, as quais podem ser conectadas de C_k maneiras, o vértice raiz pode ser qualquer um dos k vértices e os outros $n-k$ vértices podem ser conectados/desconectados de qualquer maneira). Então

$$C_n = G_n - \frac{1}{n} \sum_{k=1}^{n-1} k \binom{n}{k} C_k G_{n-k}$$

Problema 8.6.7: Grafos rotulados com k componentes conexas.

Solução: Seja $D[i][j]$ o número de grafos rotulados com i vértices e j componentes (para cada $i \leq n$ e cada $j \leq k$). Tomamos o último vértice (índice n). Esse vértice pertence a alguma componente. Se o tamanho dessa componente é s , então existem $\binom{n-1}{s-1}$ maneiras de escolher esse conjunto de vértices e C_s maneiras de conectá-los. Depois de remover essa componente do grafo temos $n-s$ vértices restando com $k-1$ componentes conexas. Daí,

$$D[n][k] = \sum_{s=1}^n \binom{n-1}{s-1} C_s D[n-s][k-1]$$

Problema 8.6.8: Números de Delanoy. Qual o número de caminhos de $(0,0)$ para (m,n) , indo pra cima, pra direita ou na diagonal? Como resolver a recorrência $D(n,m) = 1$ se $n = 0$ ou $m = 0$ e $D(n,m) = D(n-1,m) + D(m-1,n-1) + D(n,m-1)$ caso contrário?

Solução:

$$D(n,m) = \sum_{k=0}^{\min(m,n)} \binom{m}{k} \binom{n}{k} 2^k$$

Problema 8.6.9: Números de Schröder. Qual o número de caminhos S_n na mesma situação do problema acima só que agora em nenhum momento passam por cima da diagonal principal? Quantos dominós (retângulos 1×2 e 2×1) podemos arranjar em um formado de diamante asteca?

Solução: Com $S_0 = 1$ e $S_1 = 2$, satisfazem as recorrências

$$S_n = 3S_{n-1} + \sum_{k=1}^{n-2} S_k S_{n-k-1}$$

e

$$S_n = \frac{6n-3}{n+1} S_{n-1} - \frac{n-2}{n+1} S_{n-2}$$

O número de dominós é $2^{n(n+1)/2}$. A conexão com os números de Schröder é que um determinante com esses números dá isso, mas isso é irrelevante pra maratona provavelmente.

9 Teoria dos Jogos

9.1 Jogos em grafos arbitrários

Suponha que dois jogadores jogam um jogo num grafo arbitrário G . Os jogadores se movem em turnos e vão para um vértice adjacente. Dependendo do jogo, a pessoa que não puder se mover vai perder ou ganhar.

Consideramos um grafo dirigido arbitrário. A tarefa é determinado, dado um estado inicial, quem vai ganhar o jogo se os dois tiverem estratégias ótimas (ou então determinar que haverá um empate). Será encontrada uma solução para todos os vértices iniciais do grafo em $O(m)$ onde $m = \text{num de arestas}$.

Um vértice é ganhador se um player começando nele ganha o jogo se jogar de maneira ótima. Analogamente denotaremos o outro por perdedor. Para todos os vértices sem outgoing edges já sabemos sua condição de vitória/derrota. Além disso:

1. Se um vértice tem uma aresta que vai para um perdedor, então ele é ganhador
2. Se todas as arestas que saem de um vértice são vencedoras, então o vértice é perdedor
3. Se em algum ponto ainda existirem vértices indefinidos, então é um vértice de empate

Então dá pra fazer um algoritmo em $O(nm)$: roda pra todos os vértices aplicando essas regras. Mas dá pra reduzir pra $O(m)$.

Iteramos por todos os vértices que sabemos o estado inicial. Para cada um deles fazemos uma DFS que se move considerando as arestas ao contrário. Ela não vai entrar em vértices que já tem vitória/derrota definida. Se a pesquisa for de um vértice perdedor para um indefinido, então marca o indefinido como ganhador e continua a DFS dele. Se nós formos de um ganhador para um indefinido, então chegamos se todas as arestas desse indefinido vão para ganhadores. Dá pra fazer isso em $O(1)$ guardando o número de arestas que vão para vencedores para cada vértice.

f2f3d3

```
vector<vector<int>> adj_rev;

vector<bool> winning;
vector<bool> losing;
vector<bool> visited;
vector<int> degree;
```

```
void dfs(int v) {
    visited[v] = true;
    for (int u : adj_rev[v]) {
        if (!visited[u]) {
            if (losing[v])
                winning[u] = true;
            else if (--degree[u] == 0)
                losing[u] = true;
            else
                continue;
            dfs(u);
        }
    }
}
```

Problema 9.1.1: Polícia e ladrão. Tem um tabuleiro $m \times n$. Algumas das células não podem ser entradas. As coordenadas iniciais do policial e do ladrão são conhecidas. Uma das células é a saída. Se o policial e o ladrão estão na mesma cela em qualquer momento, o policial ganha. Se o ladrão estiver na saída (sem que o policial esteja lá também), ele ganha. O policial pode andar nas 8 direções e o ladrão em 4. O policial e o ladrão alternam os movimentos, mas também podem skipar um movimento se quiserem.

Solução: construímos um grafo. O estado do jogo é definido pelas coordenadas do policial, do ladrão e de quem é a vez. Ou seja, um vértice do grafo é (P, T, P_{turn}) . Precisamos determinar quais vértices são perdedores e ganhadores. Se é a vez do policial, então o vértice é um winning se as coordenadas dos dois coincidirem e é losing se não for winning E o policial está na exit. Se é a vez do ladrão, então é losing se as coordenadas coincidirem e winning se não for losing e o ladrão estiver no exit vertex.

Sobre a implementação, é importante decidir se você vai construir o grafo explicitamente ou on the fly. O pro é que tem menos chance de ter erro e vai ser bem mais fácil de codar. O contra é que vai ser muito lento do que usando on the fly.

Aqui uma approach construindo explicitamente:

4bfc4f

```
struct State {
    int P, T;
    bool Pstep;
};
```



```

vector<State> adj_rev[100][100][2]; // [P][T][Pstep]
bool winning[100][100][2];
bool losing[100][100][2];
bool visited[100][100][2];
int degree[100][100][2];

void dfs(State v) {
    visited[v.P][v.T][v.Pstep] = true;
    for (State u : adj_rev[v.P][v.T][v.Pstep]) {
        if (!visited[u.P][u.T][u.Pstep]) {
            if (losing[v.P][v.T][v.Pstep])
                winning[u.P][u.T][u.Pstep] = true;
            else if (--degree[u.P][u.T][u.Pstep] == 0)
                losing[u.P][u.T][u.Pstep] = true;
            else
                continue;
            dfs(u);
        }
    }
}

int main() {
    int n, m;
    cin >> n >> m;
    vector<string> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];

    for (int P = 0; P < n * m; P++) {
        for (int T = 0; T < n * m; T++) {
            for (int Pstep = 0; Pstep <= 1; Pstep++) {
                int Px = P / m, Py = P % m, Tx = T / m, Ty =
                    T % m;
                if (a[Px][Py] == '*' || a[Tx][Ty] == '*')
                    continue;

                bool& win = winning[P][T][Pstep];
                bool& lose = losing[P][T][Pstep];
                if (Pstep) {
                    win = Px == Tx && Py == Ty;

```

```

                    lose = !win && a[Tx][Ty] == 'E';
                } else {
                    lose = Px == Tx && Py == Ty;
                    win = !lose && a[Tx][Ty] == 'E';
                }
                if (win || lose) continue;

                State st = {P, T, !Pstep};
                adj_rev[P][T][Pstep].push_back(st);
                st.Pstep = Pstep;
                degree[P][T][Pstep]++;

                const int dx[] = {-1, 0, 1, 0, -1, -1, 1,
                                    1};
                const int dy[] = {0, 1, 0, -1, -1, 1, -1,
                                    1};
                for (int d = 0; d < (Pstep ? 8 : 4); d++) {
                    int PPx = Px, PPy = Py, TTx = Tx, TTy =
                        Ty;
                    if (Pstep) {
                        PPx += dx[d];
                        PPy += dy[d];
                    } else {
                        TTx += dx[d];
                        TTy += dy[d];
                    }

                    if (PPx >= 0 && PPx < n && PPy >= 0 &&
                        PPy < m && a[PPx][PPy] != '*' &&
                        TTx >= 0 && TTx < n && TTy >= 0 &&
                        TTy < m && a[TTx][TTy] != '*') {
                        adj_rev[PPx * m + PPy][TTx * m + TTy]
                            [!Pstep].push_back(st);
                        ++degree[P][T][Pstep];
                    }
                }
            }
        }
    }
}

```

```

for (int P = 0; P < n * m; P++) {
    for (int T = 0; T < n * m; T++) {
        for (int Pstep = 0; Pstep <= 1; Pstep++) {
            if ((winning[P][T][Pstep] || losing[P][T][
                Pstep]) && !visited[P][T][Pstep])
                dfs({P, T, (bool)Pstep});
        }
    }
}

int P_st, T_st;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (a[i][j] == 'P')
            P_st = i * m + j;
        else if (a[i][j] == 'T')
            T_st = i * m + j;
    }
}

if (winning[P_st][T_st][true]) {
    cout << "Police catches the thief" << endl;
} else if (losing[P_st][T_st][true]) {
    cout << "The thief escapes" << endl;
} else {
    cout << "Draw" << endl;
}
}

```

9.2 Teorema de Sprague-Grundy e Nim

O teorema descreve o jogo de dois players inicial i.e. aqueles em que os movimentos disponíveis e winning/losing depende apenas do estado do jogo. Ou seja, a única diferença entre os jogadores é quem se move primeiro.

Também assumimos que o jogo tem informação perfeita i.e. os jogadores sabem perfeitamente as regras e movimentos possíveis. Também assumimos que é finito i.e. depois de uma certa quantidade de movimentos um dos jogadores acabará numa losing position da qual não pode se mover. Então não há empate nesse jogo.

Esses jogos podem ser descritos completamente por um DAG.

9.2.1 Nim

Esse é um jogo que obedece tudo o que foi dito acima. Além disso, qualquer jogo imparcial two-layer de informação perfeita pode ser reduzido para o jogo de nim. Então estudar o nim resolve todos os jogos.

Você tem várias pilhas com várias pedras. Num movimento, um jogador deve escolher uma pilha e retirar uma quantidade inteira positiva de pedras dessa pilha. Ele perde se não tem mais pedras.

Teorema (Nim): O jogador atual tem uma estratégia vencedora se e só se a xor-sum das pilhas é não zero i.e. $s = a_1 \oplus \dots \oplus a_n \neq 0$. Além disso, o movimento vencedor de um dado estado consiste em reduzir alguma pilha de tamanho x para o tamanho $s \oplus x$.

Corolário: todo estado do nim pode ser substituído por outro com o mesmo xor. Quando estamos analisando um nim com várias pilhas podemos observar isso como substituindo por uma única pilha de tamanho s .

9.2.2 Teorema de Sprague-Grundy

Agora suponha que podemos adicionar pedras a uma pilha escolhida. As regras exatas de como e quando pode ocorrer esse aumento não nos interessam, mas o que importa é que essas regras devem deixar nosso jogo acíclico.

Lema: A adição de acréscimos a pedras não muda a maneira como estados de vitória/derrota são determinados. Ou seja, podemos ignorá-los.

Teorema: Considere um estado v de um jogo imparcial e sejam v_i os estados alcançáveis dele (com $i \in \{1, \dots, k\}$, $k \geq 0$). A esse estado podemos designar um jogo de nim completamente equivalente com uma pilha de tamanho x . O número x é dito o valor de Grundy ou nim-value do estado v . Esse número pode ser achado de maneira recursiva:

$$x = \text{mex}\{x_1, \dots, x_k\}$$

em que x_i são os valores de Grundy para os estados v_i e o mex pega o menor valor não negativo que não está no conjunto.

Aplicação: Para calcular o valor de Grundy de um estado, você deve

1. Pegar todas as transições a partir desse estado
2. Cada transição pode levar à soma de jogos independentes. Calcule o valor de Grundy para cada jogo independente e xor-sum neles.
3. Depois de calcular o valor de Grundy para cada transição, pega o mex dos números
4. Se o valor for zero, então o estado atual é losing. Se não for, é winning.

Dica: em tarefas específicas estudar a tabela de valores em busca de padrões é uma ótima ideia. Em muitos jogos onde uma análise teórica pode parecer difícil, os valores de Grundy são periódicos ou então tem uma forma compreensível. Em 99,9% dos casos dá pra provar o padrão por indução. Entretanto, ainda está aberto o problema de se existem regularidades em alguns casos.

Problema 9.2.1: Considere uma fita de tamanho $1 \times n$. Em um movimento, o jogador deve colocar uma cruz. Mas é proibido colocar duas cruzes uma ao lado da outra. O jogador sem movimentos válidos perde.

Solução: Seja g o valor de Grundy. É trivial ver que

$$g(n) = \text{mex}(\{g(n-2)\} \cup \{g(i-2) \oplus g(n-i-1) | 2 \leq i \leq n-1\})$$

Com isso, temos uma solução $O(n^2)$. Na verdade, $g(n)$ tem período de tamanho 34 começando com $n = 52$.

10 Álgebra Linear

10.1 Equações lineares

Podemos resolver o sistema $Ax = b$ com o seguinte código (onde o vector de vector é a matriz a só que com a última coluna sendo b , o vector de resposta vai ser alterado e será retornado 0, 1 ou infinito correspondendo ao número de soluções do sistema):

```
d847fe
const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be
                    // infinity or a big number

int gauss(vector<vector<double>> a, vector<double>& ans) {
    int n = (int)a.size();
    int m = (int)a[0].size() - 1;

    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n; ++col) {
        int sel = row;
        for (int i = row; i < n; ++i)
            if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
        if (abs(a[sel][col]) < EPS) continue;
        for (int i = col; i <= m; ++i)
            swap(a[sel][i], a[row][i]);
        where[col] = row;

        for (int i = 0; i < n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j = col; j <= m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
        ++row;
    }

    ans.assign(m, 0);
    for (int i = 0; i < m; ++i)
        if (where[i] != -1) ans[i] = a[where[i]][m] / a[
            where[i]][i];
}
```

```

for (int i = 0; i < n; ++i) {
    double sum = 0;
    for (int j = 0; j < m; ++j)
        sum += ans[j] * a[i][j];
    if (abs(sum - a[i][m]) > EPS) return 0;
}

for (int i = 0; i < m; ++i)
    if (where[i] == -1) return INF;
return 1;
}
```

Sendo n o número de linhas de A e m o de colunas, a complexidade desse código é $O(\min(n, m) \cdot mn)$

Dá pra resolver um desse mod 2 32 vezes mais rápido:

```
262530
int gauss(vector<bitset<N>> a, int n, int m, bitset<N>& ans)
{
    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n; ++col) {
        for (int i = row; i < n; ++i)
            if (a[i][col]) {
                swap(a[i], a[row]);
                break;
            }
        if (!a[row][col]) continue;
        where[col] = row;

        for (int i = 0; i < n; ++i)
            if (i != row && a[i][col]) a[i] ^= a[row];
        ++row;
    }
    // The rest of implementation is the same as above
}
```

10.2 Determinante e posto

Usaremos as ideias da subseção anterior. Calcularemos o determinante em $O(N^3)$

a8fc27

```

const double EPS = 1E-9;
int n;
vector<vector<double>> a(n, vector<double>(n));

double det = 1;
for (int i = 0; i < n; ++i) {
    int k = i;
    for (int j = i + 1; j < n; ++j)
        if (abs(a[j][i]) > abs(a[k][i])) k = j;
    if (abs(a[k][i]) < EPS) {
        det = 0;
        break;
    }
    swap(a[i], a[k]);
    if (i != k) det = -det;
    det *= a[i][i];
    for (int j = i + 1; j < n; ++j)
        a[i][j] /= a[i][i];
    for (int j = 0; j < n; ++j)
        if (j != i && abs(a[j][i]) > EPS)
            for (int k = i + 1; k < n; ++k)
                a[j][k] -= a[i][k] * a[j][i];
}

cout << det;

```

10.2.1 Decomposição LU

Uma matriz A $n \times n$ inversível sempre admite uma fatoração $A = LU$ (em que L é lower e U é upper triangular, SPG com diagonal principal de $L = 1$). Ela é única se, e só se, todos seus (determinantes dos) menores principais são não nulos. Código em java:

19e17f

```

static BigInteger det(BigDecimal a[][], int n) {
    try {

        for (int i = 0; i < n; i++) {
            boolean nonzero = false;
            for (int j = 0; j < n; j++)

```

```

                if (a[i][j].compareTo(new BigDecimal(
                    BigInteger.ZERO)) > 0)
                    nonzero = true;
            if (!nonzero)
                return BigInteger.ZERO;
        }

        BigDecimal scaling[] = new BigDecimal[n];
        for (int i = 0; i < n; i++) {
            BigDecimal big = new BigDecimal(BigInteger.ZERO);
            for (int j = 0; j < n; j++)
                if (a[i][j].abs().compareTo(big) > 0)
                    big = a[i][j].abs();
            scaling[i] = (new BigDecimal(BigInteger.ONE)).
                divide(big, 100, BigDecimal.ROUND_HALF_EVEN);
        }

        int sign = 1;

        for (int j = 0; j < n; j++) {
            for (int i = 0; i < j; i++) {
                BigDecimal sum = a[i][j];
                for (int k = 0; k < i; k++)
                    sum = sum.subtract(a[i][k].multiply(a[k]
                        ][j]));
                a[i][j] = sum;
            }

            BigDecimal big = new BigDecimal(BigInteger.ZERO);
            for (int i = j; i < n; i++) {
                BigDecimal sum = a[i][j];
                for (int k = 0; k < j; k++)
                    sum = sum.subtract(a[i][k].multiply(a[k]
                        ][j]));
                a[i][j] = sum;
                BigDecimal cur = sum.abs();

```

```

        cur = cur.multiply(scaling[i]);
        if (cur.compareTo(big) >= 0) {
            big = cur;
            imax = i;
        }
    }

    if (j != imax) {
        for (int k = 0; k < n; k++) {
            BigDecimal t = a[j][k];
            a[j][k] = a[imax][k];
            a[imax][k] = t;
        }

        BigDecimal t = scaling[imax];
        scaling[imax] = scaling[j];
        scaling[j] = t;

        sign = -sign;
    }

    if (j != n - 1)
        for (int i = j + 1; i < n; i++)
            a[i][j] = a[i][j].divide(a[j][j], 100,
                BigDecimal.ROUND_HALF_EVEN);
    }

    BigDecimal result = new BigDecimal(1);
    if (sign == -1)
        result = result.negate();
    for (int i = 0; i < n; i++)
        result = result.multiply(a[i][i]);

    return result.divide(BigDecimal.valueOf(1), 0,
        BigDecimal.ROUND_HALF_EVEN).toBigInteger();
} catch (Exception e) {
    return BigInteger.ZERO;
}
}

```

É possível determinar o posto de uma matriz em $O(n^3)$ com eliminação gaussiana

19a3f4

```

const double EPS = 1E-9;

int compute_rank(vector<vector<double>> A) {
    int n = A.size();
    int m = A[0].size();

    int rank = 0;
    vector<bool> row_selected(n, false);
    for (int i = 0; i < m; ++i) {
        int j;
        for (j = 0; j < n; ++j) {
            if (!row_selected[j] && abs(A[j][i]) > EPS)
                break;
        }

        if (j != n) {
            ++rank;
            row_selected[j] = true;
            for (int p = i + 1; p < m; ++p)
                A[j][p] /= A[j][i];
            for (int k = 0; k < n; ++k) {
                if (k != j && abs(A[k][i]) > EPS) {
                    for (int p = i + 1; p < m; ++p)
                        A[k][p] -= A[j][p] * A[k][i];
                }
            }
        }
    }

    return rank;
}

```

10.3 Algebrin em $\mathbb{F}_2$

É possível enxergar $\mathbb{Z}$ como um $\mathbb{F}_2$ -espaço vetorial (adição de vetores é $\oplus$). Uma maneira de visualizar isso é imaginando os elementos desse espaço como uma lista infinita de zeros e uns (i.e. é isomorfo a $\mathbb{F}_2^\infty$).

Tem uma técnica para adicionar/checar pertencimento em um espaço vetorial desse tipo que funciona em $O(32)$ (ou 64).

```

e54101
int basis[d]; // basis[i] keeps the mask of the vector whose
               f value is i

int sz; // Current size of the basis

void insertVector(int mask) {
    for (int i = 0; i < d; i++) {
        if ((mask & 1 << i) == 0) continue; // continue if i
        != f(mask)

        if (!basis[i]) { // If there is no basis vector with
            the i'th bit set, then insert this
                        // vector into the basis
            basis[i] = mask;
            ++sz;
            return;
        }
        mask ^= basis[i]; // Otherwise subtract the basis
                          vector from this vector
    }
}

```

pra checar pertencimento, basta ver se mask virou zero.

Problema 10.3.1: (número de maneiras de escrever elemento) Dada uma array de n inteiros, quero saber de quantas maneiras eu posso escrever x como combinação linear dos a_i .

Solução: Seja d a dimensão do espaço gerado pelos a_i . Primeiro, se x nem tiver nesse espaço, retorno zero. Se não, note que posso escolher $a_{i_1}, \dots, a_{i_d}$ que são LI. Sobram $n - d$ caras. Para simplificar, vamos dizer que temos $b_1, \dots, b_d, c_1, \dots, c_{n-d}$ em que $b_j = a_{i_j}$. Queremos saber o número de sequências de escalares λ_i e λ'_i tais que

$$\sum_{j=1}^d \lambda_j b_j + \sum_{j=1}^{n-d} \lambda'_j c_j = x$$

No entanto, note que, para cada escolha dos λ'_j , podemos determinar λ_j únicos que satisfazem o que queremos. Então, a resposta é 2^{n-d} .

11 Miscelânea

12 Apêndice

12.1 Matemática

12.1.1 Constantes

d41d8c

```

/* Definitions of useful mathematical constants
 * M_E      — e
 * M_LOG2E  — log2(e)
 * M_LOG10E — log10(e)
 * M_LN2    — ln(2)
 * M_LN10   — ln(10)
 * M_PI     — pi
 * M_PI_2   — pi/2
 * M_PI_4   — pi/4
 * M_1_PI   — 1/pi
 * M_2_PI   — 2/pi
 * M_2_SQRTPI — 2/sqrt(pi)
 * M_SQRT2  — sqrt(2)
 * M_SQRT1_2 — 1/sqrt(2)
 */
#define M_E 2.71828182845904523536
#define M_LOG2E 1.44269504088896340736
#define M_LOG10E 0.434294481903251827651
#define M_LN2 0.693147180559945309417
#define M_LN10 2.30258509299404568402
#define M_PI 3.14159265358979323846
#define M_PI_2 1.57079632679489661923
#define M_PI_4 0.785398163397448309616
#define M_1_PI 0.318309886183790671538
#define M_2_PI 0.636619772367581343076
#define M_2_SQRTPI 1.12837916709551257390
#define M_SQRT2 1.41421356237309504880
#define M_SQRT1_2 0.707106781186547524401

```

12.1.2 Fórmulas e Teoremas

Teorema de Lagrange: Todo inteiro positivo pode ser representado como soma de 4 quadrados.

Teorema 'Eureka' de Gauss: Todo inteiro positivo pode ser expresso como a soma de três números triangulares.

Teorema de Zeckendorf: Todo inteiro positivo tem uma representação única como soma de números de Fibonacci tal que não existem dois números iguais ou consecutivos.

Fórmula de Euclides para ternas pitagóricas: Ternas pitagóricas primitivas são todas da forma

$$(n^2 - m^2, 2nm, n^2 + m^2)$$

em que $0 < m < n$, $(n, m) = 1$ e $2 \nmid nm$.

12.1.3 Números primos

Primos até 10.000

Existem 1229 números primos até 10.000.

2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103
 107 109 113 127 131 137 139 149 151 157 163 167 173 179 181 191 193 197 199 211
 223 227 229 233 239 241 251 257 263 269 271 277 281 283 293 307 311 313 317 331
 337 347 349 353 359 367 373 379 383 389 397 401 409 419 421 431 433 439 443 449
 457 461 463 467 479 487 491 499 503 509 521 523 541 547 557 563 569 571 577 587
 593 599 601 607 613 617 619 631 641 643 647 653 659 661 673 677 683 691 701 709
 719 727 733 739 743 751 757 761 769 773 787 797 809 811 821 823 827 829 839 853
 857 859 863

877 881 883 887 907 911 919 929 937 941 947 953 967 971 977 983 991 997 1009
 1013 1019 1021 1031 1033 1039 1049 1051 1061 1063 1069 1087 1091 1093 1097 1103
 1109 1117 1123 1129 1151 1153 1163 1171 1181 1187 1193 1201 1213 1217 1223 1229
 1231 1237 1249 1259 1277 1279 1283 1289 1291 1297 1301 1303 1307 1319 1321 1327
 1361 1367 1373 1381 1399 1409 1423 1427 1429 1433 1439 1447 1451 1453 1459 1471
 1481 1483 1487 1489 1493 1499 1511 1523 1531 1543 1549 1553 1559 1567 1571 1579
 1583 1597 1601 1607 1609 1613 1619 1621 1627 1637 1657 1663 1667 1669 1693 1697
 1699 1709 1721 1723 1733 1741 1747 1753 1759 1777 1783 1787 1789 1801 1811 1823
 1831 1847 1861 1867 1871 1873 1877 1879 1889 1901 1907 1913 1931 1933 1949 1951
 1973 1979 1987

1993 1997 1999 2003 2011 2017 2027 2029 2039 2053 2063 2069 2081 2083 2087
 2089 2099 2111 2113 2129 2131 2137 2141 2143 2153 2161 2179 2203 2207 2213 2221
 2237 2239 2243 2251 2267 2269 2273 2281 2287 2293 2297 2309 2311 2333 2339 2341
 2347 2351 2357 2371 2377 2381 2383 2389 2393 2399 2411 2417 2423 2437 2441 2447

2459 2467 2473 2477 2503 2521 2531 2539 2543 2549 2551 2557 2579 2591 2593 2609
 2617 2621 2633 2647 2657 2659 2663 2671 2677 2683 2687 2689 2693 2699 2707 2711
 2713 2719 2729 2731 2741 2749 2753 2767 2777 2789 2791 2797 2801 2803 2819 2833
 2837 2843 2851 2857 2861 2879 2887 2897 2903 2909 2917 2927 2939 2953 2957 2963
 2969 2971 2999 3001 3011 3019 3023 3037 3041 3049 3061 3067 3079 3083 3089 3109
 3119 3121 3137 3163 3167 3169 3181

3187 3191 3203 3209 3217 3221 3229 3251 3253 3257 3259 3271 3299 3301 3307
 3313 3319 3323 3329 3331 3343 3347 3359 3361 3371 3373 3389 3391 3407 3413 3433
 3449 3457 3461 3463 3467 3469 3491 3499 3511 3517 3527 3529 3533 3539 3541 3547
 3557 3559 3571 3581 3583 3593 3607 3613 3617 3623 3631 3637 3643 3659 3671 3673
 3677 3691 3697 3701 3709 3719 3727 3733 3739 3761 3767 3769 3779 3793 3797 3803
 3821 3823 3833 3847 3851 3853 3863 3877 3881 3889 3907 3911 3917 3919 3923 3929
 3931 3943 3947 3967 3989 4001 4003 4007 4013 4019 4021 4027 4049 4051 4057 4073
 4079 4091 4093 4099 4111 4127 4129 4133 4139 4153 4157 4159 4177 4201 4211 4217
 4219 4229 4231 4241 4243 4253 4259 4261 4271 4273 4283 4289 4297 4327 4337 4339
 4349 4357 4363 4373 4391 4397 4409

4421 4423 4441 4447 4451 4457 4463 4481 4483 4493 4507 4513 4517 4519 4523
 4547 4549 4561 4567 4583 4591 4597 4603 4621 4637 4639 4643 4649 4651 4657 4663
 4673 4679 4691 4703 4721 4723 4729 4733 4751 4759 4783 4787 4789 4793 4799 4801
 4813 4817 4831 4861 4871 4877 4889 4903 4909 4919 4931 4933 4937 4943 4951 4957
 4967 4969 4973 4987 4993 4999 5003 5009 5011 5021 5023 5039 5051 5059 5077 5081
 5087 5099 5101 5107 5113 5119 5147 5153 5167 5171 5179 5189 5197 5209 5227 5231
 5233 5237 5261 5273 5279 5281 5297 5303 5309 5323 5333 5347 5351 5381 5387 5393
 5399 5407 5413 5417 5419 5431 5437 5441 5443 5449 5471 5477 5479 5483 5501 5503
 5507 5519 5521 5527 5531 5557 5563 5569 5573 5581 5591 5623 5639 5641 5647 5651
 5653 5657 5659 5669 5683 5689 5693

5701 5711 5717 5737 5741 5743 5749 5779 5783 5791 5801 5807 5813 5821 5827
 5839 5843 5849 5851 5857 5861 5867 5869 5879 5881 5897 5903 5923 5927 5939 5953
 5981 5987 6007 6011 6029 6037 6043 6047 6053 6067 6073 6079 6089 6091 6101 6113
 6121 6131 6133 6143 6151 6163 6173 6197 6199 6203 6211 6217 6221 6229 6247 6257
 6263 6269 6271 6277 6287 6299 6301 6311 6317 6323 6329 6337 6343 6353 6359 6361
 6367 6373 6379 6389 6397 6421 6427 6449 6451 6469 6473 6481 6491 6521 6529 6547
 6551 6553 6563 6569 6571 6577 6581 6599 6607 6619 6637 6653 6659 6661 6673 6679
 6689 6691 6701 6703 6709 6719 6733 6737 6761 6763 6779 6781 6791 6793 6803 6823
 6827 6829 6833 6841 6857 6863 6869 6871 6883 6899 6907 6911 6917 6947 6949 6959
 6961 6967 6971 6977 6983 6991 6997

7001 7013 7019 7027 7039 7043 7057 7069 7079 7103 7109 7121 7127 7129 7151
 7159 7177 7187 7193 7207 7211 7213 7219 7229 7237 7243 7247 7253 7283 7297 7307
 7309 7321 7331 7333 7349 7351 7369 7393 7411 7417 7433 7451 7457 7459 7477 7481
 7487 7489 7499 7507 7517 7523 7529 7537 7541 7547 7549 7559 7561 7573 7577 7583

7589 7591 7603 7607 7621 7639 7643 7649 7669 7673 7681 7687 7691 7699 7703 7717
 7723 7727 7741 7753 7757 7759 7789 7793 7817 7823 7829 7841 7853 7867 7873 7877
 7879 7883 7901 7907 7919 7927 7933 7937 7949 7951 7963 7993 8009 8011 8017 8039
 8053 8059 8069 8081 8087 8089 8093 8101 8111 8117 8123 8147 8161 8167 8171 8179
 8191 8209 8219 8221 8231 8233 8237 8243 8263 8269 8273 8287 8291 8293 8297 8311
 8317 8329 8353 8363 8369 8377 8387

8389 8419 8423 8429 8431 8443 8447 8461 8467 8501 8513 8521 8527 8537 8539
 8543 8563 8573 8581 8597 8599 8609 8623 8627 8629 8641 8647 8663 8669 8677 8681
 8689 8693 8699 8707 8713 8719 8731 8737 8741 8747 8753 8761 8779 8783 8803 8807
 8819 8821 8831 8837 8839 8849 8861 8863 8867 8887 8893 8923 8929 8933 8941 8951
 8963 8969 8971 8999 9001 9007 9011 9013 9029 9041 9043 9049 9059 9067 9091 9103
 9109 9127 9133 9137 9151 9157 9161 9173 9181 9187 9199 9203 9209 9221 9227 9239
 9241 9257 9277 9281 9283 9293 9311 9319 9323 9337 9341 9343 9349 9371 9377 9391
 9397 9403 9413 9419 9421 9431 9433 9437 9439 9461 9463 9467 9473 9479 9491 9497
 9511 9521 9533 9539 9547 9551 9587 9601 9613 9619 9623 9629 9631 9643 9649 9661
 9677 9679 9689 9697 9719 9721 9733

9739 9743 9749 9767 9769 9781 9787 9791 9803 9811 9817 9829 9833 9839 9851
 9857 9859 9871 9883 9887 9901 9907 9923 9929 9931 9941 9949 9967 9973

Depois daí os próximos primos são 10007, 10009, 10037, 10039.

Existem em torno de 50 milhões de primos até 10^9 . Uns primos logo antes de 10^9 são 999999883, 999999893, 999999929, 999999937. Uns primos logo depois são 1000000007, 1000000009, 1000000021, 1000000033, 1000000087, 1000000093, 1000000097, 1000000103.

Seja $\pi(x)$ a quantidade de primos $\leq x$. Temos que

$$\frac{x}{\ln x} \left(1 + \frac{1}{\ln x} \right) < \pi(x) < \frac{x}{\ln x} \left(1 + \frac{1}{\ln x} + \frac{2.51}{(\ln x)^2} \right)$$

em que a primeira desigualdade vale para $x \geq 599$ e a segunda para $x \geq 355991$. Mas uma bound útil e que vale para $x \geq 55$ é que

$$\frac{x}{\ln x + 2} < \pi(x) < \frac{x}{\ln x - 4}$$

Seja p_n o enésimo número primo. Temos $p_n \approx n \ln n$. Uma aproximação melhor é a seguinte

$$\frac{p_n}{n} = \ln n + \ln \ln n - 1 + \frac{(\ln \ln n)^2 - 6 \ln \ln n + 11}{2(\ln n)^2} + o\left(\frac{1}{(\ln n)^2}\right)$$

12.2 Big-O

Os servidores do USACO fazem mais ou menos 10^8 operações por segundo e isso pode subir para $\approx 5 \cdot 10^8$ se as constantes forem boas.

Tabela de valores conservadores de n e complexidades possíveis. Pode ser que se eu escrevi uma determinada complexidade para determinada faixa de n , na verdade dava pra ser uma complexidade ainda pior.

n	Complexidades possíveis
$n \leq 10$	$O(n!), O(n^7), O(n^6)$
$n \leq 20$	$O(2^n \cdot n), O(n^5)$
$n \leq 80$	$O(n^4)$
$n \leq 400$	$O(n^3)$
$n \leq 7500$	$O(n^2)$
$n \leq 7 \cdot 10^4$	$O(n \cdot \sqrt{n})$
$n \leq 5 \cdot 10^5$	$O(n \log n)$
$n \leq 5 \cdot 10^6$	$O(n)$
$n \leq 10^{18}$	$O(\log^2 n), O(\log n), O(1)$

Teorema mestre: Seja $T(n)$ o tempo levado por um algoritmo com input de tamanho n . Seja $f(n)$ o tempo levado pelo topo da recorrência. Suponha que

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

onde $b > 1$ (é como se dividisse em a subproblemas e em cada um deles eu trabalhe com uma input menor por um fator de $1/b$).

Então, sendo $c' = \log_b(a)$, teremos:

1. Se $f(n) = O(n^c)$ com $c < c'$ com diferença polinomial entre $f(n)$ e $n^{c'}$ i.e. $f(n)/n^{c'} = \Theta(n^d)$ para algum d real, então $T(n) = \Theta(n^{c'})$
2. Se $f(n) = \Theta(n^{c'} \log^k n)$ para algum $k \geq 0$, então $T(n) = \Theta(n^{c'} \log^{k+1} n)$
3. Se $f(n) = \Omega(n^c)$ com $c > c'$, então não dá pra tirar conclusão. Mas no caso particular em que $af\left(\frac{n}{b}\right) \leq kf(n)$ para alguma constante $k < 1$ e n grande o suficiente (isso é chamado de condição de regularidade), então $T(n) = \Theta(f(n))$

É possível estender esse segundo caso

1. Se $f(n) = \Theta(n^{c'} \log^k n)$ com $k > -1$, então $T(n) = \Theta(n^{c'} \log^{k+1} n)$
2. Se $f(n) = \Theta(n^{c'} \log^k n)$ com $k = -1$, então $T(n) = \Theta(n^{c'} \log \log n)$
3. Se $f(n) = \Theta(n^{c'} \log^k n)$ com $k < -1$, então $T(n) = \Theta(n^{c'})$

Tem uma versão "linear" do teorema mestre no sentido de que

$$T(n) = aT(n-b) + f(n)$$

em que $a, b > 0$ são constantes e $f(n) = O(n^k)$ com $k \geq 0$. Temos os seguintes casos

1. Se $a < 1$, $T(n) = O(n^k)$ ou simplesmente $T(n) = O(f(n))$
2. Se $a = 1$, então $T(n) = O(n^{k+1})$ ou simplesmente $T(n) = O(nf(n))$
3. Se $a > 1$, então $T(n) = O(a^{n/b} f(n))$

12.3 Python...

Espero que você não tenha que usar isso.

078ca2

```
if judge:
    import io, os
    input = io.BytesIO(os.read(0, os.fstat(0).st_size)).
        readline

import sys

sys.stdout.write('Hello')
sys.setrecursionlimit(10**9)
```